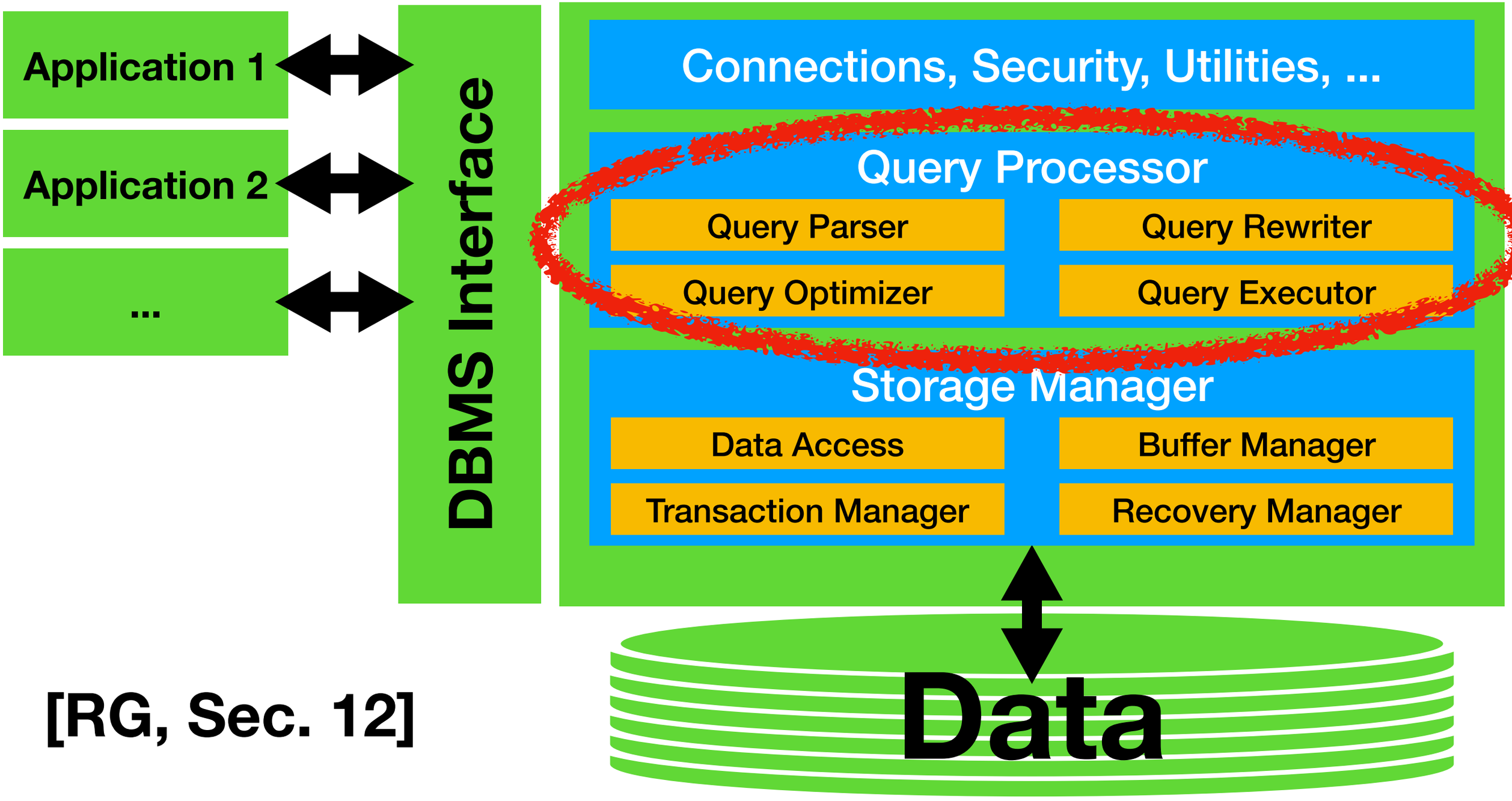
Operator Implementations

Immanuel Trummer
itrummer@cornell.edu
www.itrummer.org

Database Management Systems (DBMS)



[RG, Sec. 12]

Query Processing

- Input query is parsed (**Parser**) and simplified (**Rewriter**)
- **Query optimizer** generates optimized execution plan
- Executing plan (**Executor**) produces query result

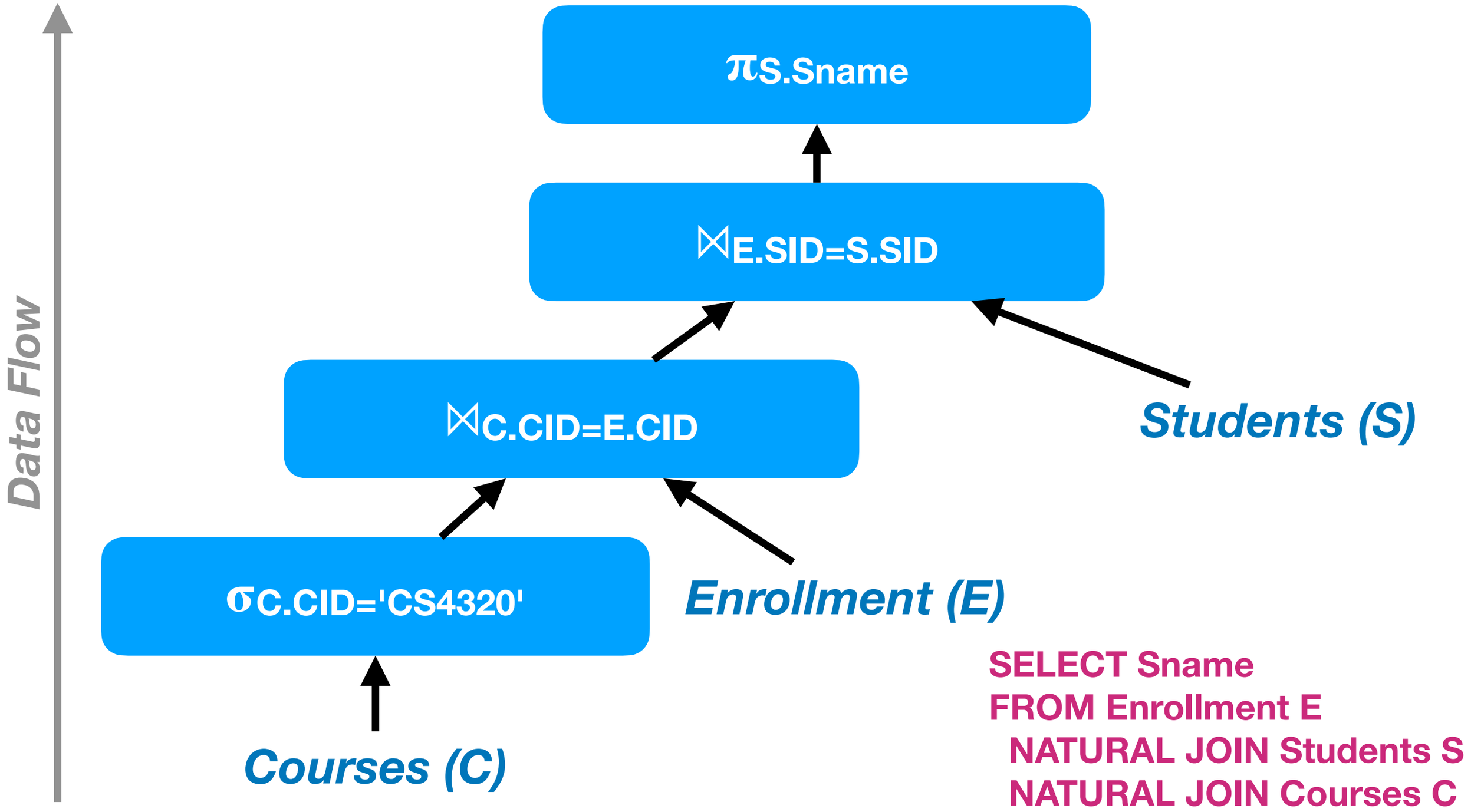
Query Plans

- Describe **how** to generate required data
- Typically represented as a **tree**
- Each leaf node represents a database **table**
- Each inner node represents an **operation**
- Tree edges represent **data flow**

Operators

- Query plans use fixed set of **standard operators**
- Consumes relation(s) and produces one **relation**
- **Filter operator (σ)**: discard rows based on condition
- **Projection operator (π)**: discard columns
- **Join operator ($\bowtie$)**: find matching tuple pairs
- ...

Example Plan




```
SELECT Sname
FROM Enrollment E
  NATURAL JOIN Students S
  NATURAL JOIN Courses C
WHERE C.CID = 'CS4320'
```

Outlook

- Will discuss how to implement **standard operators**
- Often have **multiple** implementations of same operator
- Can choose **most efficient** implementation at each point
- Will also discuss about **cost estimation**
  - Assumption: cost ~ **number of pages** read/written

Filter Operator (σ)

How to Filter?

- Want to retrieve table rows satisfying a **predicate**
- Simplest option: **scan** all pages, check each entry
- Option if sorted: **binary search** for specific predicates
- Can use **indexes** if available (right table, right key)

Costing Example

SELECT * FROM Enrollment WHERE CID='CS4320'

Property	Value
Nr. Students	60,000
Nr. Enrollments/Student	10
Size/Enrollment Entry	10 Bytes
Bytes/Page	1,000 Bytes

Calculate scan cost!

Nr. Entries/Page	?
Nr. Enrollment Pages	?
Total Scan Cost	?

Costing Example

SELECT * FROM Enrollment WHERE CID='CS4320'

Property	Value
Nr. Students	60,000
Nr. Enrollments/Student	10
Size/Enrollment Entry	10 Bytes
Bytes/Page	1,000 Bytes

Calculate scan cost!

Nr. Entries/Page	100
Nr. Enrollment Pages	6,000
Total Scan Cost	6,000

Example

P1	
	P2
Holly	P3
Olivia	P4

P2	
	P10
Dora	P11
Felix	P12

P3	
	P13
Kyle	P14
Mia	P15

P4	
	P16
Rosa	P17
Tia	P18

P10	
Alan	P25,3
Bob	P42,1
Chan	P29,3

P11	
Dora	P24,2
David	P36,1
Ester	P62,3

P12	
Felix	P21,3
Gert	P91,1
Harry	P74,2

P13	
Holly	P23,1
Ida	P47,2
Jana	P62,1

P14	
Kyle	P76,1
Lana	P22,3
Levi	P56,3

P15	
Mia	P36,1
Milo	P54,2
Nicola	P38,2

P16	
Olivia	P44,1
Paul	P35,2
Philip	P58,1

P17	
Rosa	P29,1
Ryan	P32,2
Sergei	P53,1

P18	
Tia	P41,1
Victor	P47,1
Zemin	P82,3

Index by Student Name

Example

Index entries (reference index pages)

P1	
	P2
Holly	P3
Olivia	P4

P2	
	P10
Dora	P11
Felix	P12

P3	
	P13
Kyle	P14
Mia	P15

P4	
	P16
Rosa	P17
Tia	P18

P10	
Alan	P25,3
Bob	P42,1
Chan	P29,3

P11	
Dora	P24,2
David	P36,1
Ester	P62,3

P12	
Felix	P21,3
Gert	P91,1
Harry	P74,2

P13	
Holly	P23,1
Ida	P47,2
Jana	P62,1

P14	
Kyle	P76,1
Lana	P22,3
Levi	P56,3

P15	
Mia	P36,1
Milo	P54,2
Nicola	P38,2

P16	
Olivia	P44,1
Paul	P35,2
Philip	P58,1

P17	
Rosa	P29,1
Ryan	P32,2
Sergei	P53,1

P18	
Tia	P41,1
Victor	P47,1
Zemin	P82,3

Data entries (reference data pages)

Index by Student Name

Example

Index entries (reference index pages)

P1	
	P2
Holly	P3
Olivia	P4

P2	
	P10
Dora	P11
Felix	P12

P3	
	P13
Kyle	P14
Mia	P15

P4	
	P16
Rosa	P17
Tia	P18

P10	
Alan	P25,3
Bob	P42,1
Chan	P29,3

P11	
Dora	P24,2
David	P36,1
Ester	P62,3

P12	
Felix	P21,3
Gert	P91,1
Harry	P74,2

P13	
Holly	P23,1
Ida	P47,2
Jana	P62,1

P14	
Kyle	P76,1
Lana	P22,3
Levi	P56,3

P15	
Mia	P36,1
Milo	P54,2
Nicola	P38,2

P16	
Olivia	P44,1
Paul	P35,2
Philip	P58,1

P17	
Rosa	P29,1
Ryan	P32,2
Sergei	P53,1

P18	
Tia	P41,1
Victor	P47,1
Zemin	P82,3

Data entries (reference data pages)

Searching for Student "Alan"

Index by Student Name

Example

Index entries (reference index pages)

P1	
	P2
Holly	P3
Olivia	P4

1

P2	
	P10
Dora	P11
Felix	P12

P3	
	P13
Kyle	P14
Mia	P15

P4	
	P16
Rosa	P17
Tia	P18

P10	
Alan	P25,3
Bob	P42,1
Chan	P29,3

P11	
Dora	P24,2
David	P36,1
Ester	P62,3

P12	
Felix	P21,3
Gert	P91,1
Harry	P74,2

P13	
Holly	P23,1
Ida	P47,2
Jana	P62,1

P14	
Kyle	P76,1
Lana	P22,3
Levi	P56,3

P15	
Mia	P36,1
Milo	P54,2
Nicola	P38,2

P16	
Olivia	P44,1
Paul	P35,2
Philip	P58,1

P17	
Rosa	P29,1
Ryan	P32,2
Sergei	P53,1

P18	
Tia	P41,1
Victor	P47,1
Zemin	P82,3

Data entries (reference data pages)

Searching for Student "Alan"

Index by Student Name

Example

Index entries (reference index pages)

P1	
	P2
Holly	P3
Olivia	P4

1

P2	
	P10
Dora	P11
Felix	P12

2

P3	
	P13
Kyle	P14
Mia	P15

P4	
	P16
Rosa	P17
Tia	P18

P10	
Alan	P25,3
Bob	P42,1
Chan	P29,3

P11	
Dora	P24,2
David	P36,1
Ester	P62,3

P12	
Felix	P21,3
Gert	P91,1
Harry	P74,2

P13	
Holly	P23,1
Ida	P47,2
Jana	P62,1

P14	
Kyle	P76,1
Lana	P22,3
Levi	P56,3

P15	
Mia	P36,1
Milo	P54,2
Nicola	P38,2

P16	
Olivia	P44,1
Paul	P35,2
Philip	P58,1

P17	
Rosa	P29,1
Ryan	P32,2
Sergei	P53,1

P18	
Tia	P41,1
Victor	P47,1
Zemin	P82,3

Data entries (reference data pages)

Searching for Student "Alan"

Index by Student Name

Example

Index entries
(reference index pages)

P1

	P2
Holly	P3
Olivia	P4

1

P2

	P10
Dora	P11
Felix	P12

2

P3

	P13
Kyle	P14
Mia	P15

P4

	P16
Rosa	P17
Tia	P18

P10

Alan	P25,3
Bob	P42,1
Chan	P29,2

3

P11

Dora	P24,2
David	P36,1
Ester	P62,3

P12

Felix	P21,3
Gert	P91,1
Harry	P74,2

P13

Holly	P23,1
Ida	P47,2
Jana	P62,1

P14

Kyle	P76,1
Lana	P22,3
Levi	P56,3

P15

Mia	P36,1
Milo	P54,2
Nicola	P38,2

P16

Olivia	P44,1
Paul	P35,2
Philip	P58,1

P17

Rosa	P29,1
Ryan	P32,2
Sergei	P53,1

P18

Tia	P41,1
Victor	P47,1
Zemin	P82,3

Data entries
(reference data pages)

Searching for Student "Alan"

Index by Student Name

Costing Example

SELECT * FROM Enrollment WHERE CID='CS4320'

Property	Value
Nr. Students	60,000
Nr. Courses	100
Nr. Enrollments/Student	10
Size/Enrollment Entry	10 Bytes
Bytes/Page	1,000 Bytes
Tree Height	3

Clustered Tree Index on CID - Calculate Access Cost!

Nr. Entries/Page	100
Nr. Enrollment Pages	6,000
Nr. Inner Node Visits	?
Nr. Leaf Node Visits	?
Total Cost	?

Costing Example

SELECT * FROM Enrollment WHERE CID='CS4320'

Property	Value
Nr. Students	60,000
Nr. Courses	100
Nr. Enrollments/Student	10
Size/Enrollment Entry	10 Bytes
Bytes/Page	1,000 Bytes
Tree Height	3

Clustered Tree Index on CID - Calculate Access Cost!

Nr. Entries/Page	100
Nr. Enrollment Pages	6,000
Nr. Inner Node Visits	2
Nr. Leaf Node Visits	?
Total Cost	?

Costing Example

SELECT * FROM Enrollment WHERE CID='CS4320'

Property	Value
Nr. Students	60,000
Nr. Courses	100
Nr. Enrollments/Student	10
Size/Enrollment Entry	10 Bytes
Bytes/Page	1,000 Bytes
Tree Height	3

Clustered Tree Index on CID - Calculate Access Cost!

Nr. Entries/Page	100
Nr. Enrollment Pages	6,000
Nr. Inner Node Visits	2
Nr. Leaf Node Visits	60
Total Cost	?

Costing Example

SELECT * FROM Enrollment WHERE CID='CS4320'

Property	Value
Nr. Students	60,000
Nr. Courses	100
Nr. Enrollments/Student	10
Size/Enrollment Entry	10 Bytes
Bytes/Page	1,000 Bytes
Tree Height	3

Clustered Tree Index on CID - Calculate Access Cost!

Nr. Entries/Page	100
Nr. Enrollment Pages	6,000
Nr. Inner Node Visits	2
Nr. Leaf Node Visits	60
Total Cost	62

Costing Example

SELECT * FROM Enrollment WHERE CID='CS4320'

Property	Value
Nr. Students	60,000
Nr. Courses	100
Nr. Enrollments/Student	10
Size/Enrollment Entry	10 Bytes
Bytes/Page	1,000 Bytes
Tree Height	3

Unclustered Tree Index on CID - Access Cost:

Nr. Entries/Page	100
Nr. Enrollment Pages	6,000
Nr. Inner Node Visits	2
Nr. Leaf Node Visits	60
Nr. Data Pages Read	6,000
Total Cost	6,062

Costing Example

SELECT * FROM Enrollment WHERE CID='CS4320'

Property	Value
Nr. Students	60,000
Nr. Courses	100
Nr. Enrollments/Student	10
Size/Enrollment Entry	10 Bytes
Bytes/Page	1,000 Bytes
Tree Height	3

Unclustered Tree Index on CID - Access Cost:

Nr. Entries/Page	100
Nr. Enrollment Pages	6,000
Nr. Inner Node Visits	2
Nr. Leaf Node Visits	60
Nr. Data Pages Read	6,000
Total Cost	6,062

Example Summary

Implementation	Cost
Scan	6,000
Clustered Index	62
Unclustered Index	6,062

Insights

- Index or sort orders **can** speed up filtering
- However, may **not always** be more efficient
- Need to **calculate cost** of alternatives and compare
- (That's the job of the **query optimizer** ...)

Join Operators (⋈)

Join Operators

- Often one of the **most expensive** operations
- Lots of research on different **join operators**
- Some are **more generic** and apply to any join predicate
- Some are **faster** in specific situations
- Some need **less memory** than others
- ...

Page Nested Loop Join

- Load one page after the other from **first table**
- **For each page** from first table:
  - Load one page after the other from **second table**
  - For all tuples in memory: **check** and **add** to result
- Terminology: first table = **outer table**

Notations

- **LoadPage**(P): Load page P
- **Pages**(T): Pages of table T
- **Tuples**(P): Tuples of page P

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

For ep in **Pages**(E):

  LoadPage(ep)

  For sp in **Pages**(S):

    LoadPage(sp)

    For et in **Tuples**(ep), st in **Tuples**(sp):

      If (et.Sid=st.Sid):

        Output(et ⋈ st)

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

For ep in **Pages**(E):



LoadPage(ep)

For sp in **Pages**(S):

LoadPage(sp)

For et in **Tuples**(ep), st in **Tuples**(sp):

If (et.Sid=st.Sid):

Output(et ⋈ st)

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```



Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
```

LoadPage(ep) ⟵ *For each page in E*

```
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

*Cost = pages in E * load cost*

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)                    <-- For each page in E
  For sp in Pages(S):
    LoadPage(sp)                  <-- For each page in E and each page in S
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

*Cost = pages in E * load cost + pages in E * pages in S * load cost*

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
```

For each page in E

```
  LoadPage(ep)
  For sp in Pages(S):
```

For each page in E and each page in S

```
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

*Cost = pages in E * load cost +*
*pages in E * pages in S * load cost +*
*tuples in E * tuples in S * evaluation cost*

Page Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

For each page in E

For each page in E and each page in S

*Cost = pages in E * load cost +*
*pages in E * pages in S * load cost +*
*tuples in E * tuples in S * evaluation cost*

Example

#Enrollment Pages	1,000
#Student Pages	100
Page Nested Loop Cost (Enrollment as Outer!)	?

Example

#Enrollment Pages	1,000
#Student Pages	100
Page Nested Loop Cost (Enrollment as Outer!)	**101,000**

1,000+100 * 1,000 = 101,000

How Much Memory?

- Need space to store current **page from outer table**
- Need space to store **current page from inner table**
- Need **one buffer page** to store output (before disk write)

How Much Memory?

- Need space to store current **page from outer table** *1*
- Need space to store **current page from inner table** *+1*
- Need **one buffer page** to store output (before disk write) *+1*

= 3

How to Improve Join Operator?

Block Nested Loop Join

- **Page** nested loop: read inner table for each outer **page**
- **Block** nested loop: read inner table for each outer **block**
  - More efficient as block contains **multiple** pages

More Notations

- **PageBlocks**(T, b): Blocks of b pages from T

- **LoadPages**(B): Load pages from block B

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

For ep in **PageBlocks**(E, b):

  LoadPages(ep)

  For sp in **Pages**(S):

    LoadPage(sp)

    For et in **Tuples**(ep), st in **Tuples**(sp):

      If (et.Sid=st.Sid):

        Output(et ⋈ st)

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

For ep in **PageBlocks**(E, b):



  LoadPages(ep)

  For sp in **Pages**(S):

    LoadPage(sp)

    For et in **Tuples**(ep), st in **Tuples**(sp):

      If (et.Sid=st.Sid):

        Output(et ⋈ st)

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```



Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
```

← *For each page in E*

```
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

*Cost = pages in E * load cost*

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

For each page in E

For each block in E and each page in S

*Cost = pages in E * load cost + blocks in E * pages in S * load cost*

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

For each page in E

For each block in E and each page in S

*Cost = pages in E * load cost + blocks in E * pages in S * load cost*

Block Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in PageBlocks(E, b):
  LoadPages(ep)
  For sp in Pages(S):
    LoadPage(sp)
    For et in Tuples(ep), st in Tuples(sp):
      If (et.Sid=st.Sid):
        Output(et ⋈ st)
```

For each page in E

For each block in E and each page in S

*Cost = pages in E * load cost + blocks in E * pages in S * load cost*

Example

#Enrollment Pages	1,000
#Student Pages	100
#Outer Block Pages	10
Block Nested Loop Cost (Enrollment as Outer!)	?

Example

#Enrollment Pages	1,000
#Student Pages	100
#Outer Block Pages	10
Block Nested Loop Cost (Enrollment as Outer!)	**11,000**

1,000+1,000/10 * 100 = 11,000

How Much Memory?

- Need enough space to store **blocks from outer relation**
- Need space to store **one page from inner** relation
- Need one **page to store output** (before writing to disk)

How Much Memory?

- Need enough space to store **blocks from outer relation** ***B***
- Need space to store **one page from inner** relation ***+1***
- Need one **page to store output** (before writing to disk) ***+1***

= B+2

Index Nested Loop Join

- Idea: have **index on join column** and equality predicate
- **Iterate over pages** of non-indexed (outer) table
- For each outer tuple, **use index** to find matching tuples

More Notations

- **Index**(Predicate): Entries satisfying predicate

- **Tuple**(P, i): i-th tuple on page P

Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For et in Tuples(ep):
    For <sp, i> in Index(et.Sid=st.Sid):
      LoadPage(sp)
      Output(et ⋈ Tuple(sp, i))
```

Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$



```
For ep in Pages(E):
  LoadPage(ep)
  For et in Tuples(ep):
    For <sp, i> in Index(et.Sid=st.Sid):
      LoadPage(sp)
      Output(et ⋈ Tuple(sp, i))
```

Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

For ep in **Pages**(E):



  LoadPage(ep)

  For et in **Tuples**(ep):

    For <sp, i> in **Index**(et.Sid=st.Sid):

      LoadPage(sp)

      Output(et ⋈ **Tuple**(sp, i))

Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

For ep in **Pages**(E):

  LoadPage(ep)



  For et in **Tuples**(ep):

    For <sp, i> in **Index**(et.Sid=st.Sid):

      LoadPage(sp)

      Output(et ⋈ **Tuple**(sp, i))

Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For et in Tuples(ep):
    For <sp, i> in Index(et.Sid=st.Sid):
      LoadPage(sp)
      Output(et ⋈ Tuple(sp, i))
```

Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

For ep in **Pages**(E):

  LoadPage(ep)

  For et in **Tuples**(ep):

    For <sp, i> in **Index**(et.Sid=st.Sid):

      LoadPage(sp)

      Output(et ⋈ **Tuple**(sp, i))

Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For et in Tuples(ep):
    For <sp, i> in Index(et.Sid=st.Sid):
      LoadPage(sp)
      Output(et ⋈ Tuple(sp, i))
```



Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For et in Tuples(ep):
    For <sp, i> in Index(et.Sid=st.Sid):
      LoadPage(sp)
      Output(et ⋈ Tuple(sp, i))
```

Index Nested Loop Join

$\bowtie_{E.Sid=S.Sid}$

```
For ep in Pages(E):
  LoadPage(ep)
  For et in Tuples(ep):
    For <sp, i> in Index(et.Sid=st.Sid):
      LoadPage(sp)
      Output(et ⋈ Tuple(sp, i))
```

Cost = pages in E* load cost +
index entries * load cost

How Much Memory?

- Need one page to store current **page from outer table**
- Need one page to store **current page from inner table**
- Need one page as **output buffer** (before disk write)

How Much Memory?

- Need one page to store current **page from outer table** *1*
- Need one page to store **current page from inner table** *+1*
- Need one page as **output buffer** (before disk write) *+1*

= 3

Alternatives for Equality Joins?

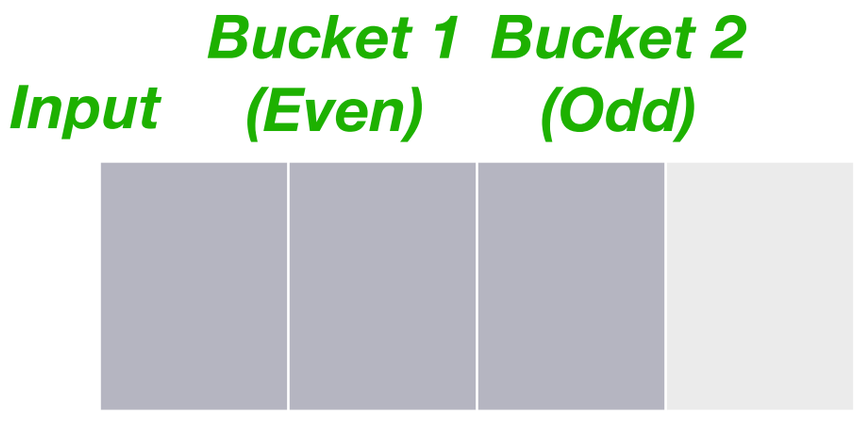
Outlook

- Will see **two operators** for equality join predicates
- Both **pre-process** input data
  - Goal: **collocate rows** with similar join column values
  - Methods: either use **sorting** or **hashing**

Hash Join

- Join **Phase 1**
  - **Partition data** by hash values in join columns
- Join **Phase 2**
  - Join **each partition** pair (same hash value)

Hash Join - Phase 1



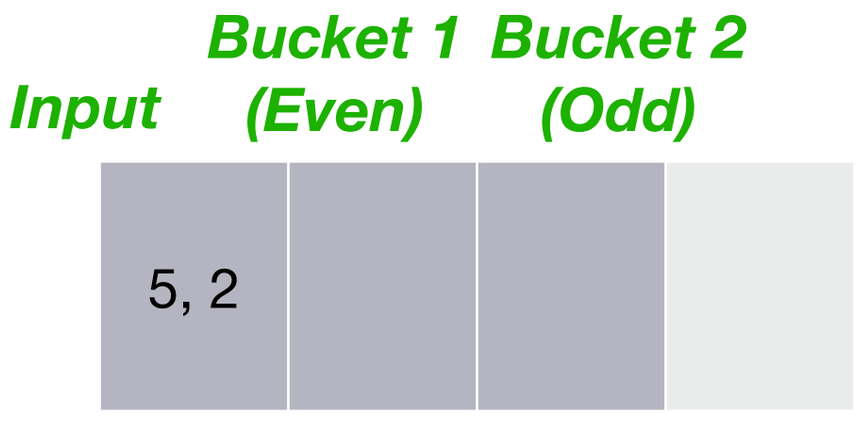
5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

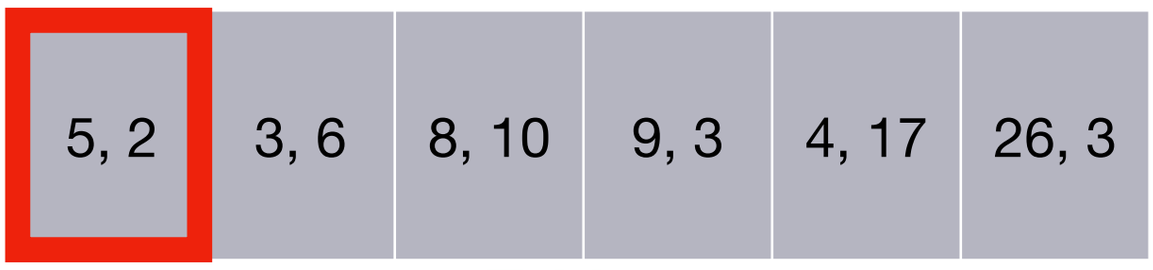
Table 1

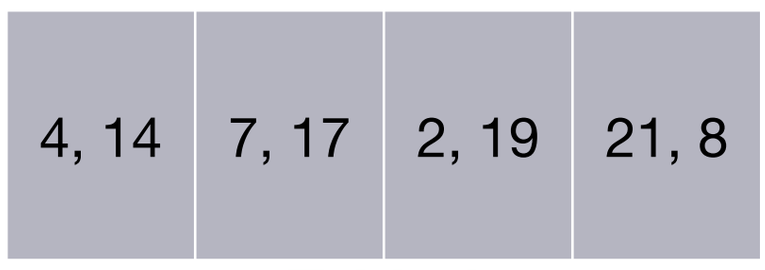
4, 14	7, 17	2, 19	21, 8

Table 2

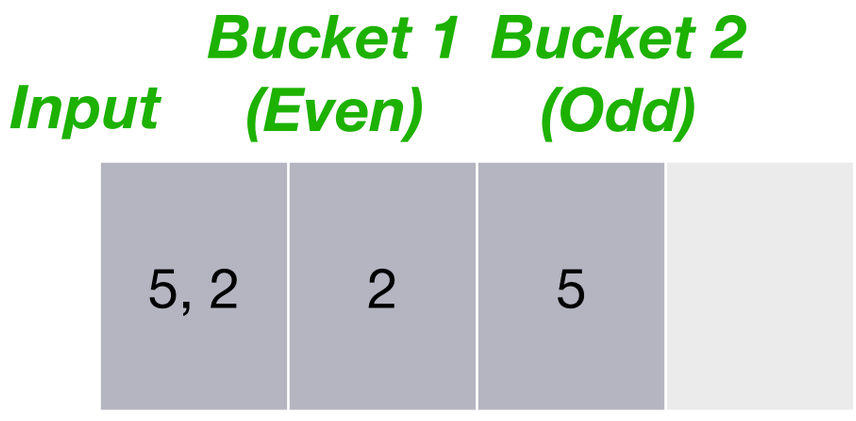
Hash Join - Phase 1







Hash Join - Phase 1



Buffer Pool

Hard Disk

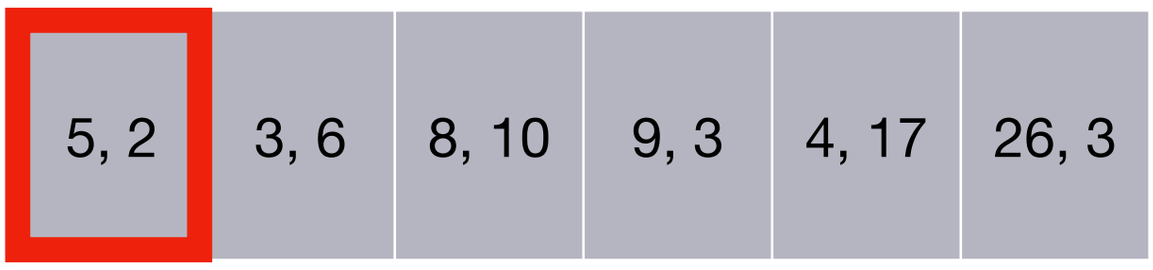


Table 1

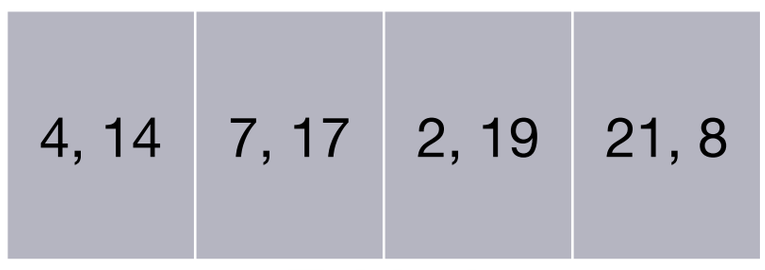
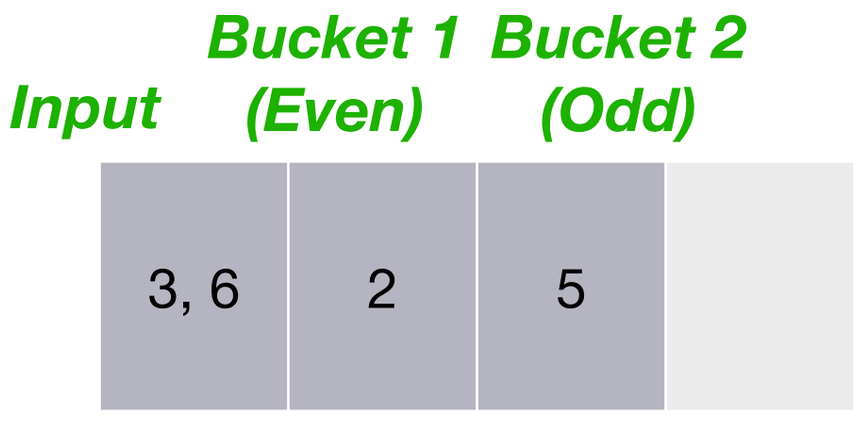


Table 2

Hash Join - Phase 1



Buffer Pool

Hard Disk

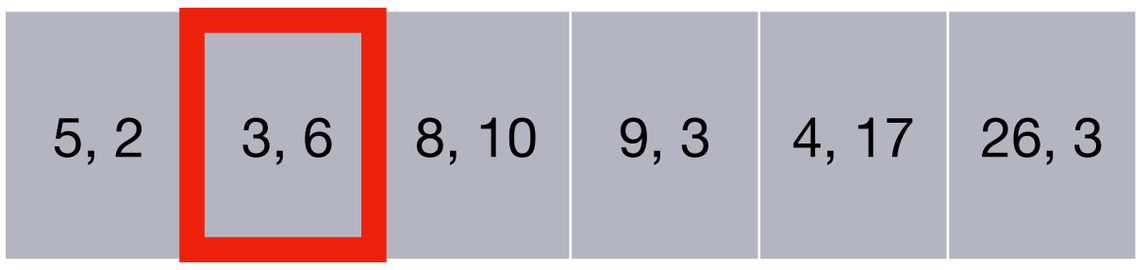


Table 1

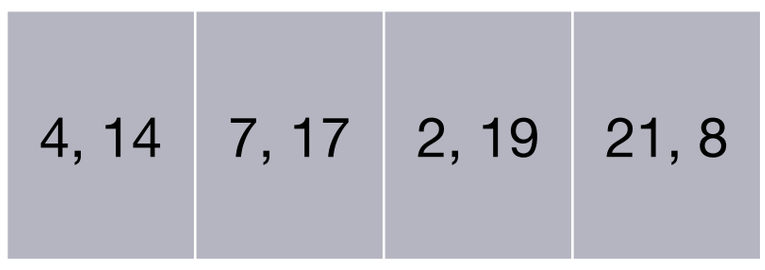
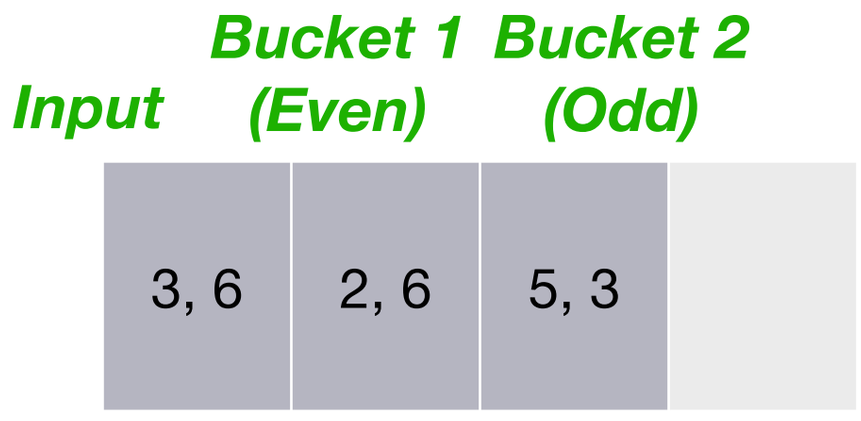
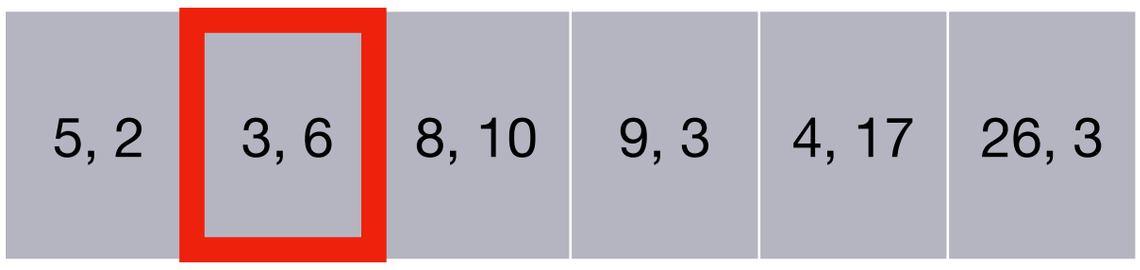
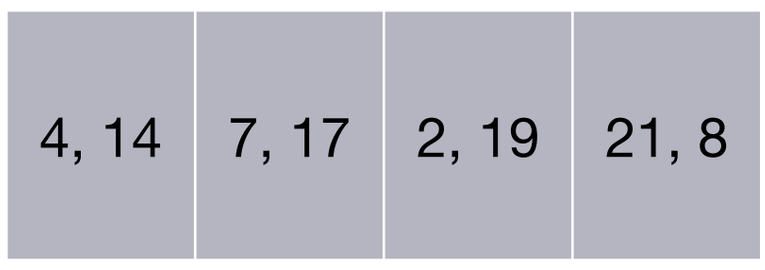


Table 2

Hash Join - Phase 1







Hash Join - Phase 1

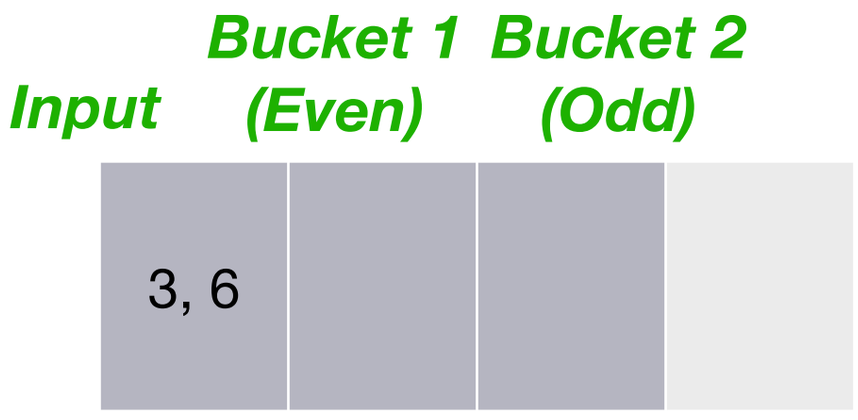




Table 1: Bucket 1



Table 1: Bucket 2

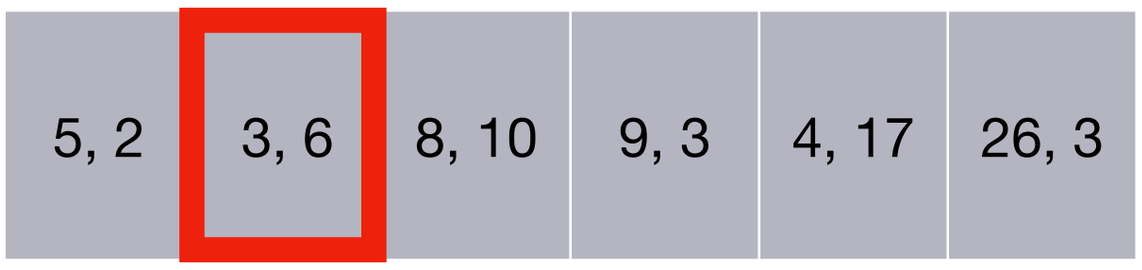


Table 1

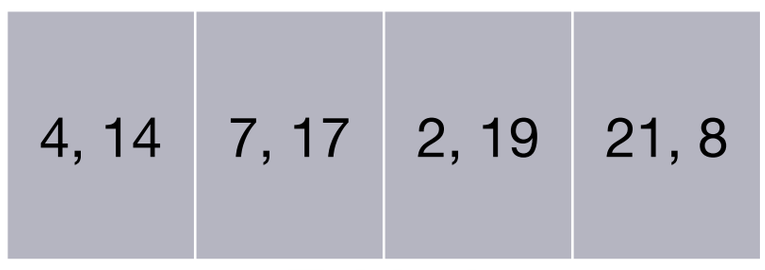


Table 2

Hash Join - Phase 1

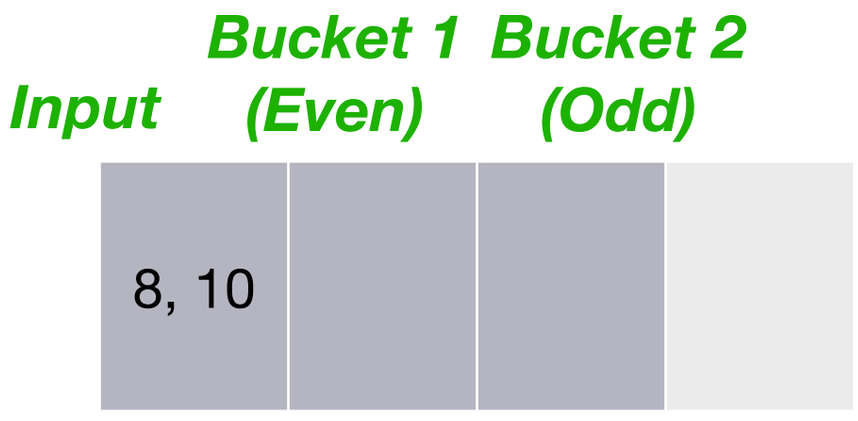




Table 1: Bucket 1



Table 1: Bucket 2

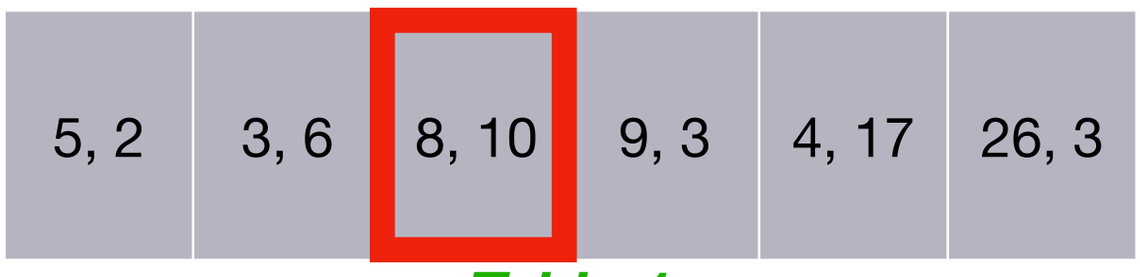


Table 1

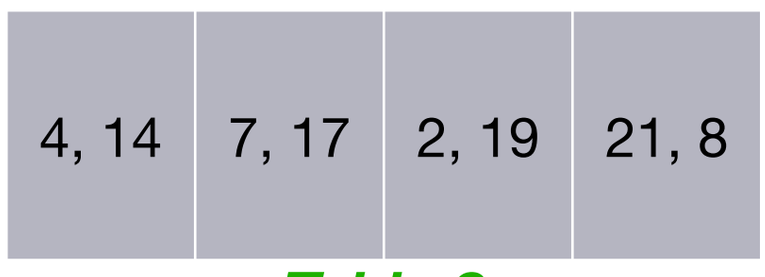


Table 2

Hash Join - Phase 1

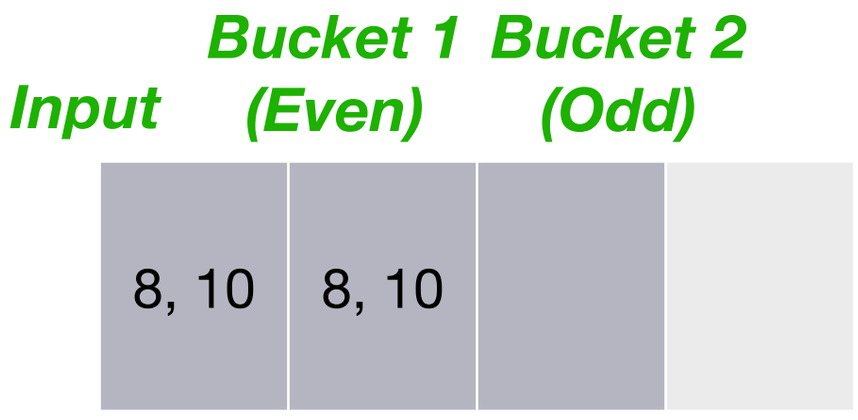




Table 1: Bucket 1



Table 1: Bucket 2

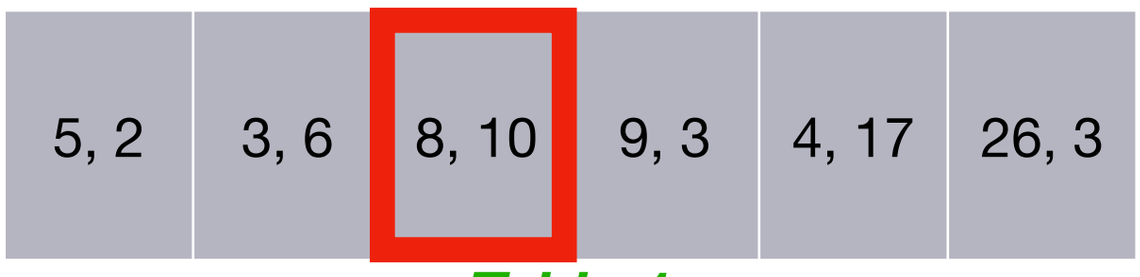


Table 1

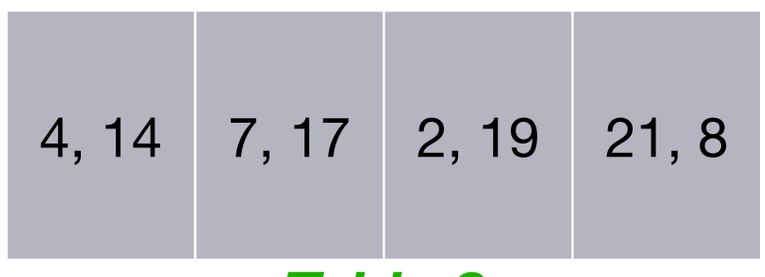
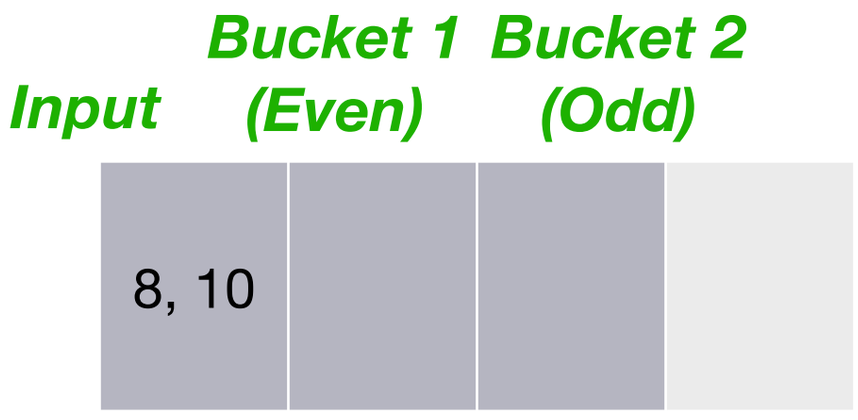


Table 2

Hash Join - Phase 1



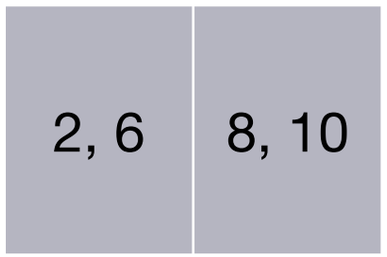


Table 1: Bucket 1



Table 1: Bucket 2

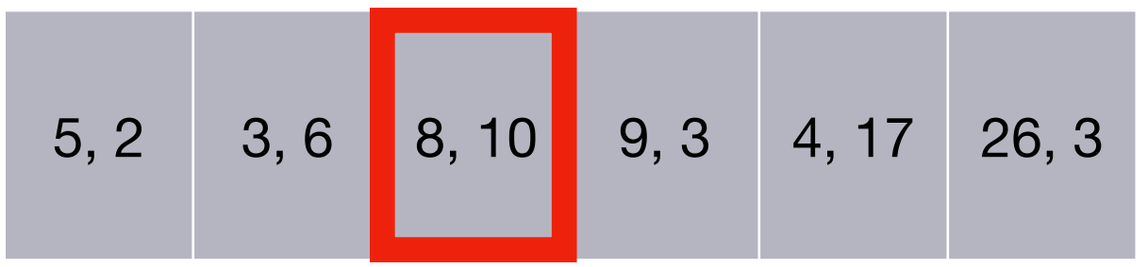


Table 1

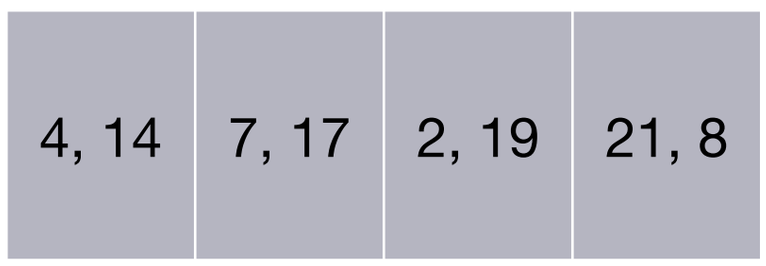
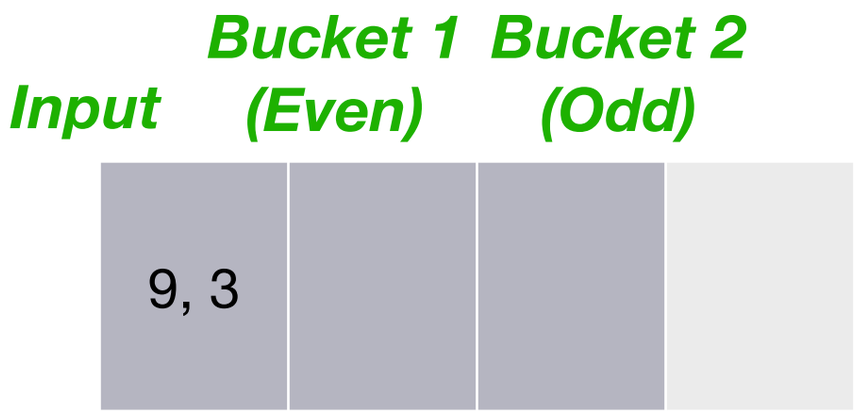


Table 2

Hash Join - Phase 1



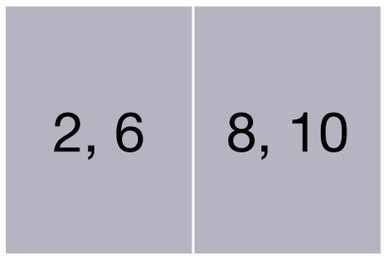


Table 1: Bucket 1



Table 1: Bucket 2

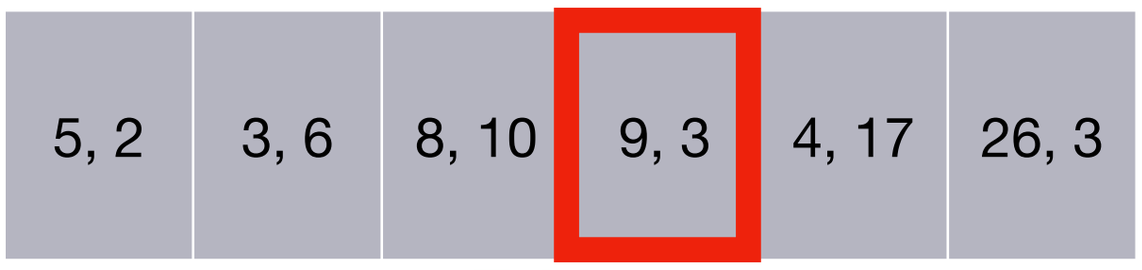


Table 1

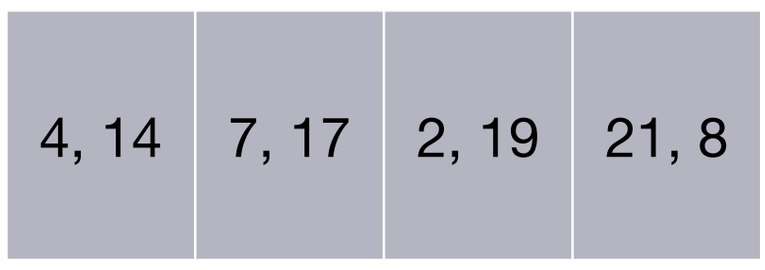
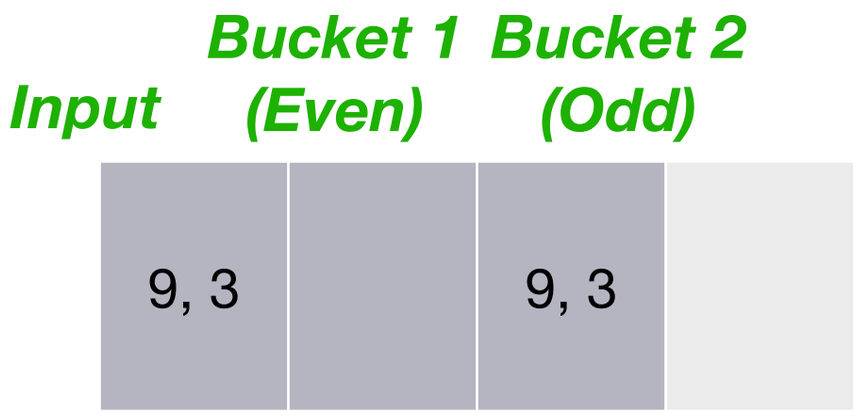


Table 2

Hash Join - Phase 1



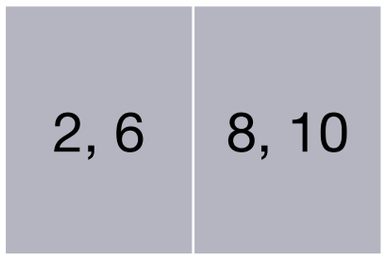


Table 1: Bucket 1



Table 1: Bucket 2

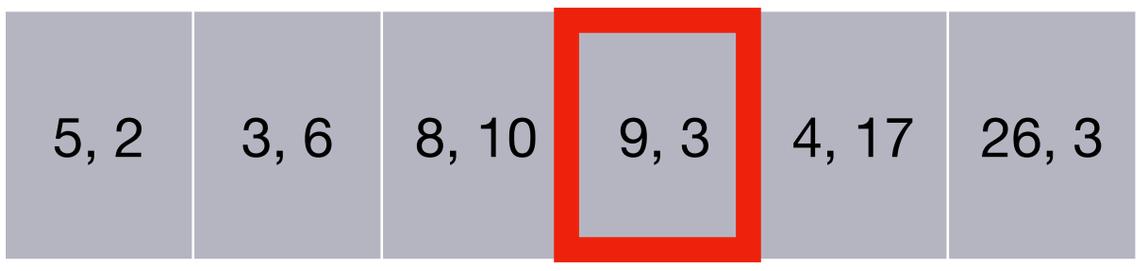


Table 1

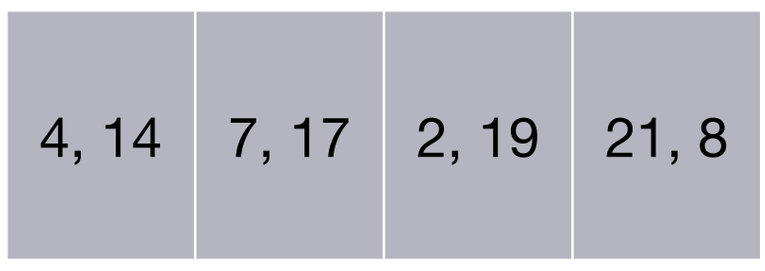
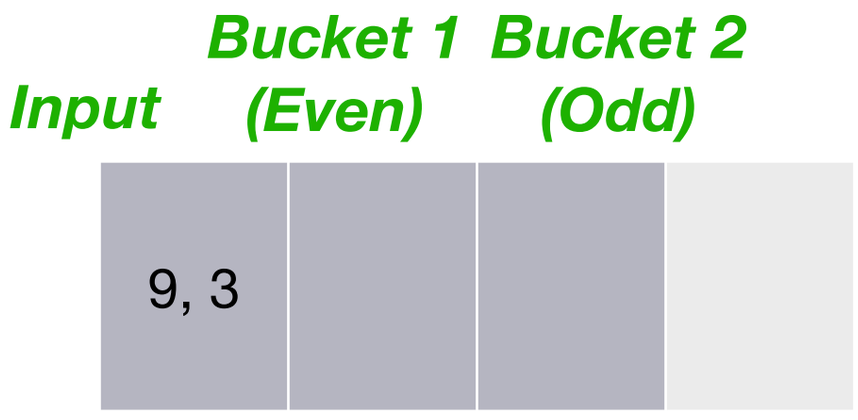


Table 2

Hash Join - Phase 1



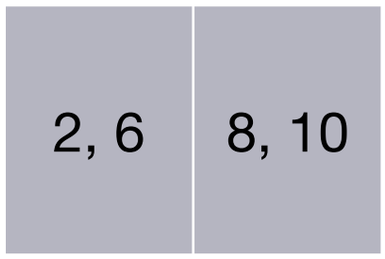


Table 1: Bucket 1

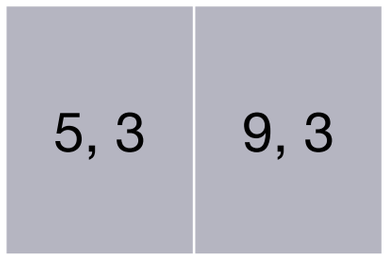


Table 1: Bucket 2

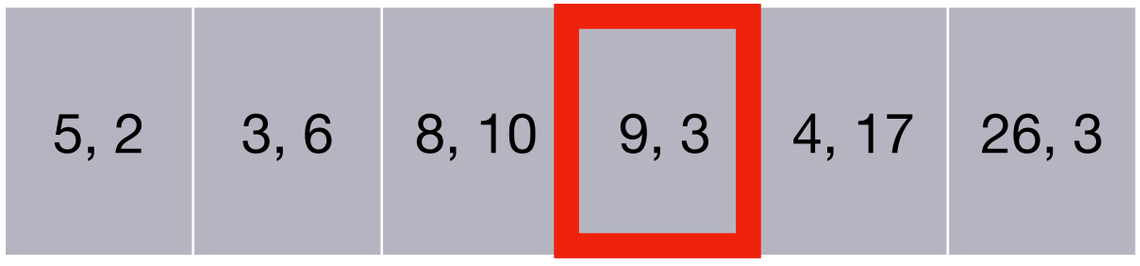


Table 1

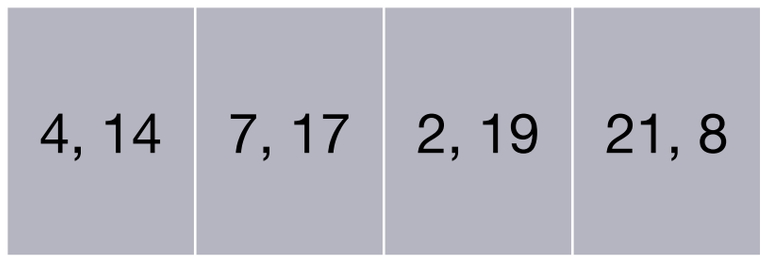
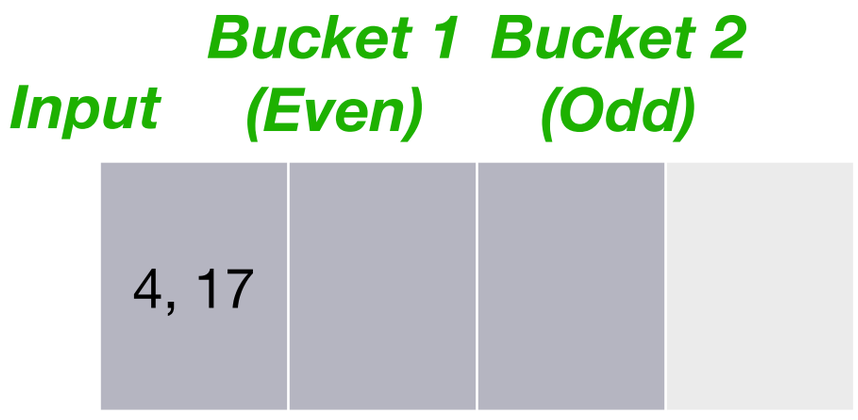


Table 2

Hash Join - Phase 1



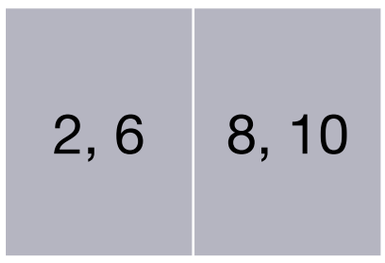


Table 1: Bucket 1

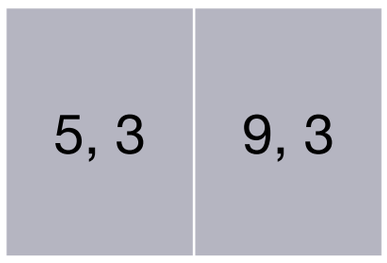


Table 1: Bucket 2

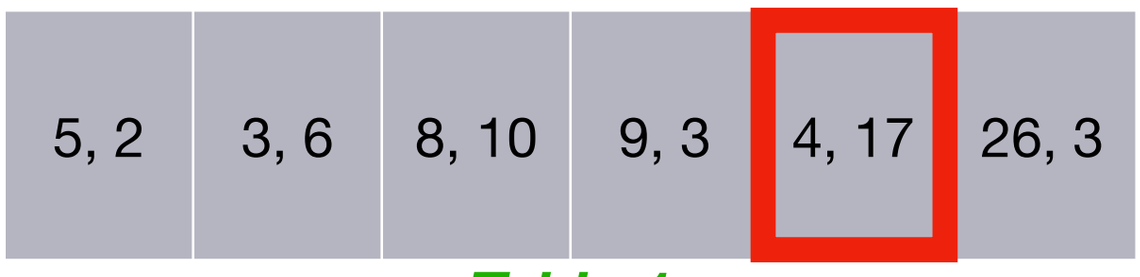


Table 1

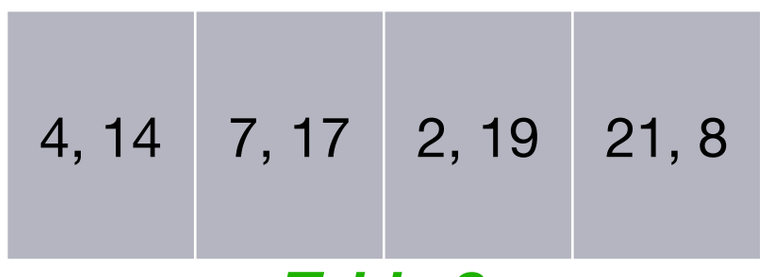
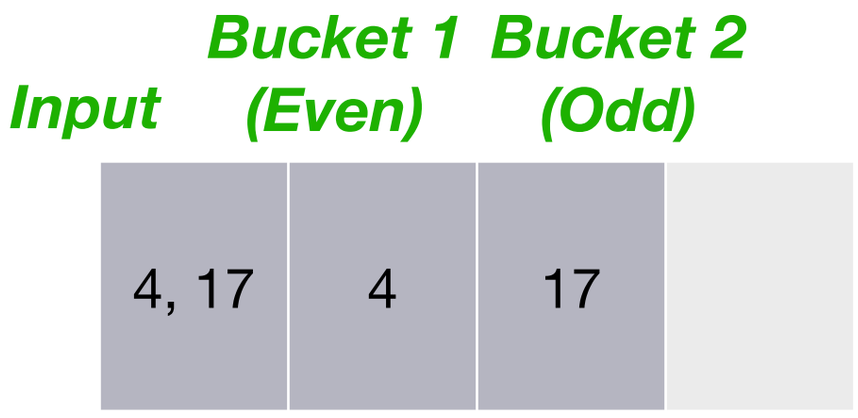


Table 2

Hash Join - Phase 1



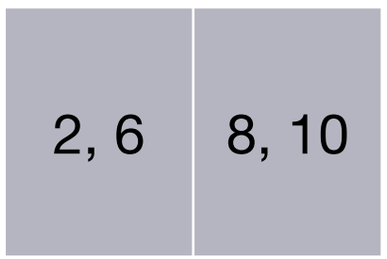


Table 1: Bucket 1

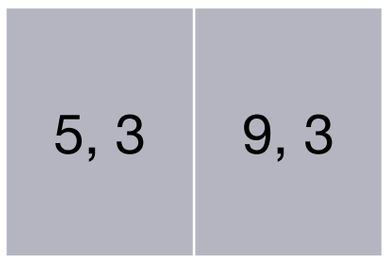


Table 1: Bucket 2

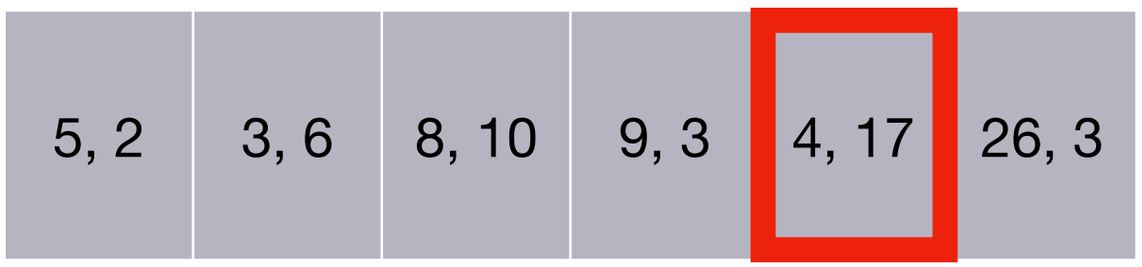


Table 1

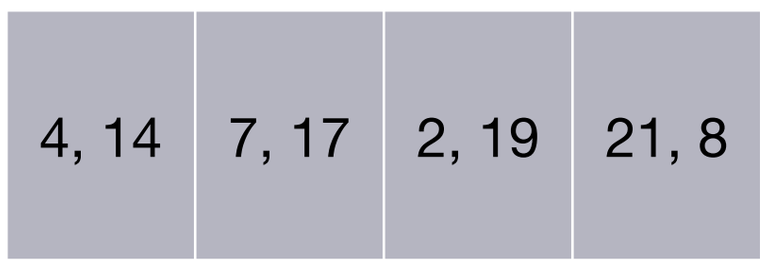
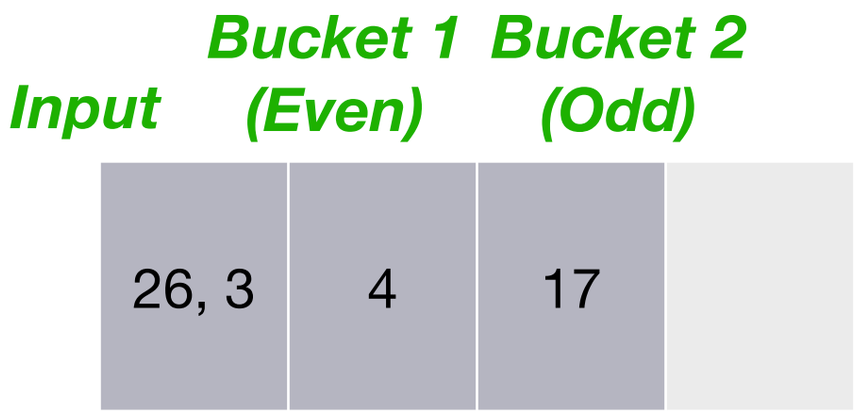


Table 2

Hash Join - Phase 1



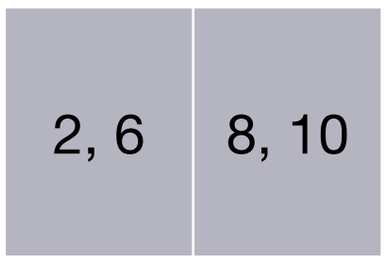


Table 1: Bucket 1

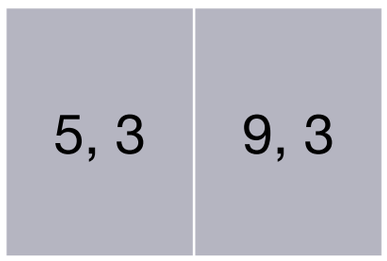


Table 1: Bucket 2

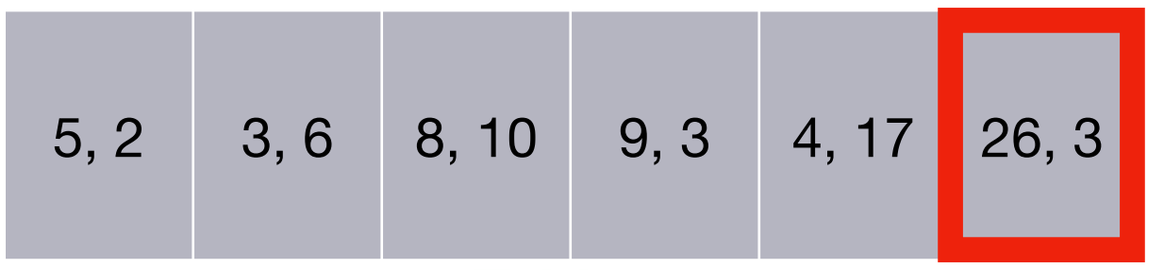


Table 1

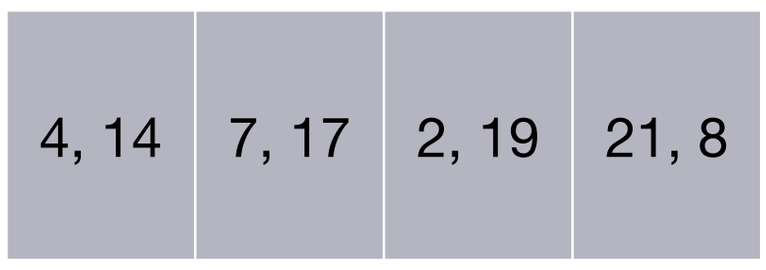
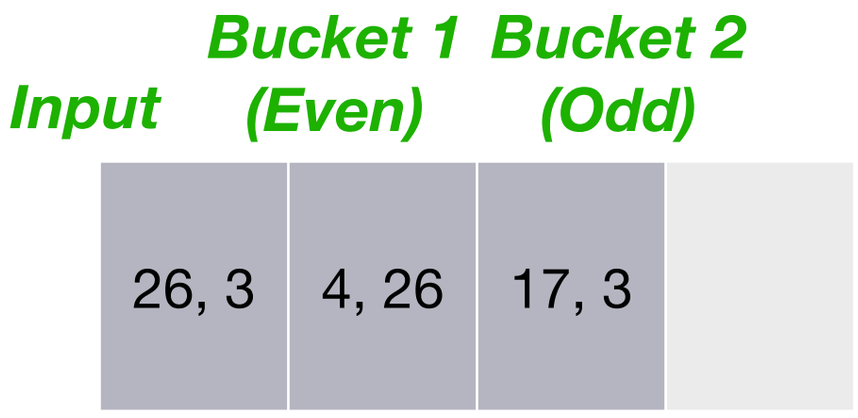


Table 2

Hash Join - Phase 1



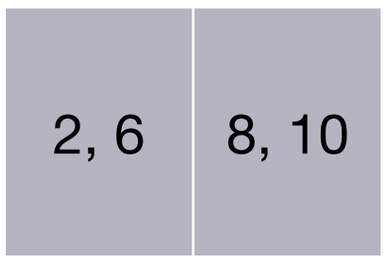


Table 1: Bucket 1

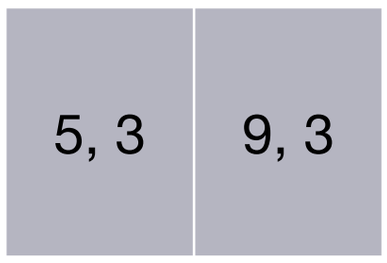


Table 1: Bucket 2

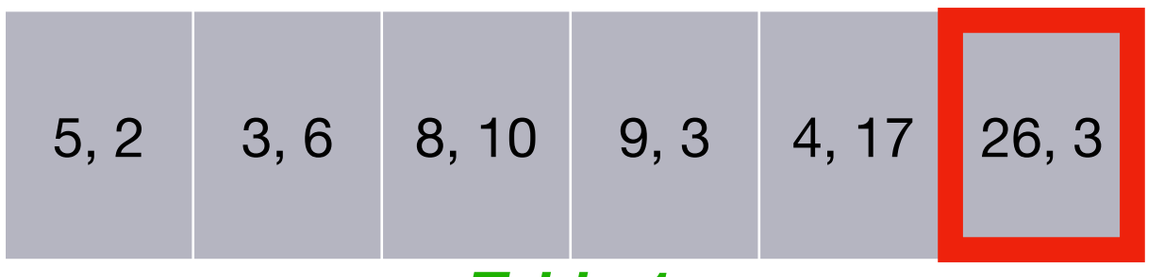


Table 1

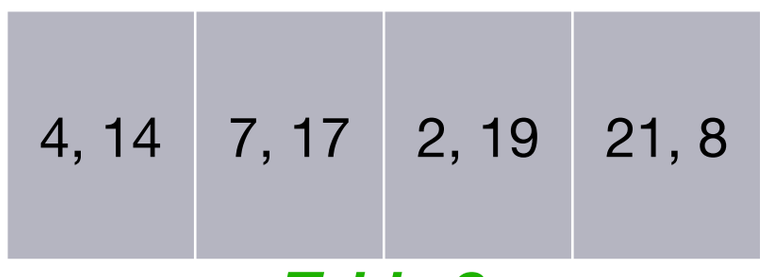
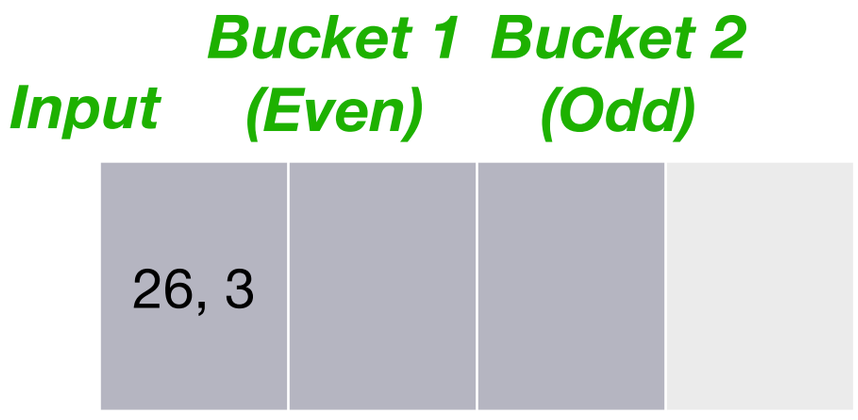
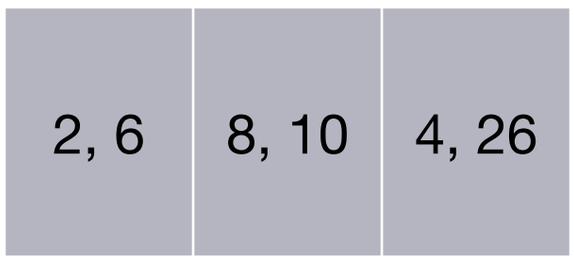
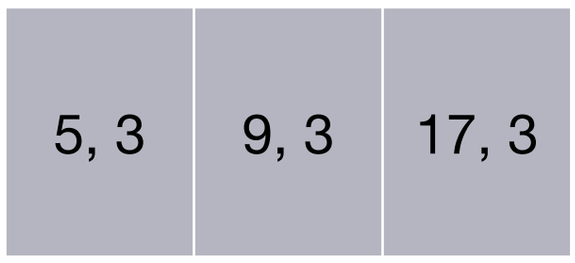


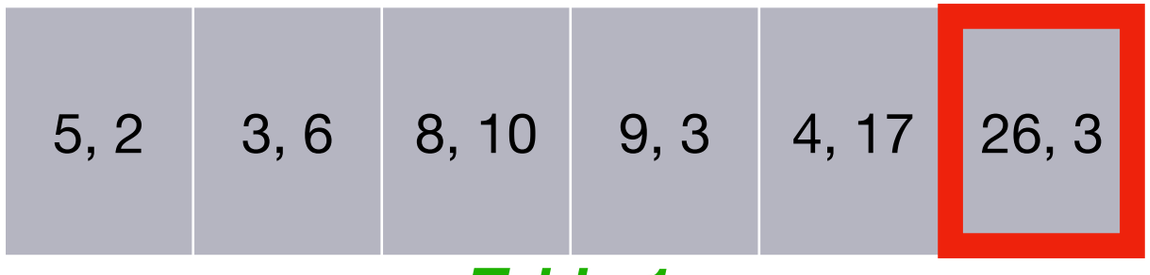
Table 2

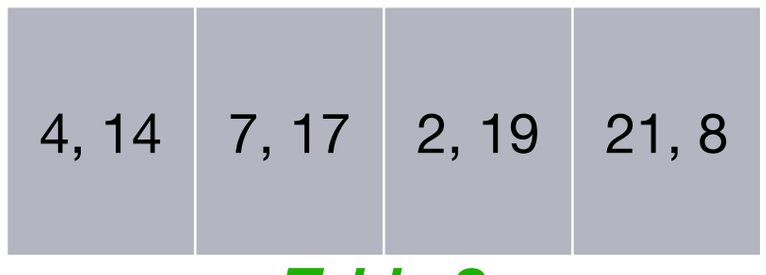
Hash Join - Phase 1



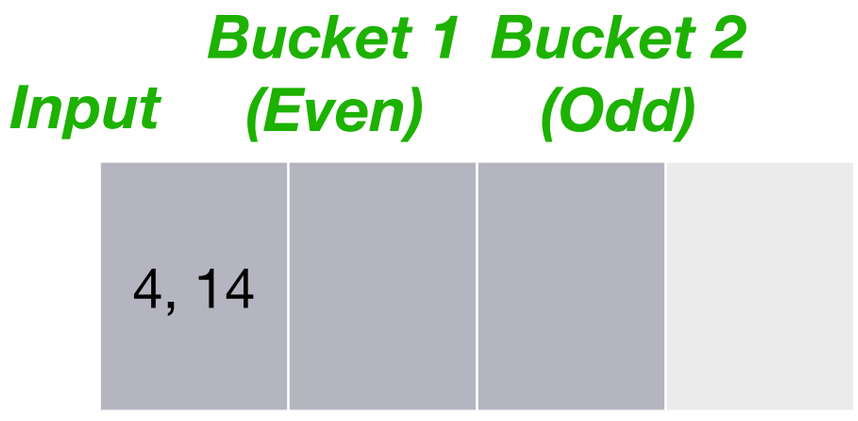








Hash Join - Phase 1



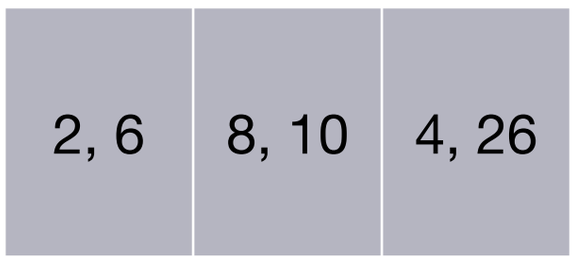


Table 1: Bucket 1

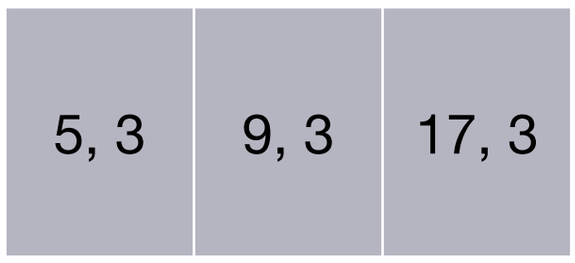


Table 1: Bucket 2

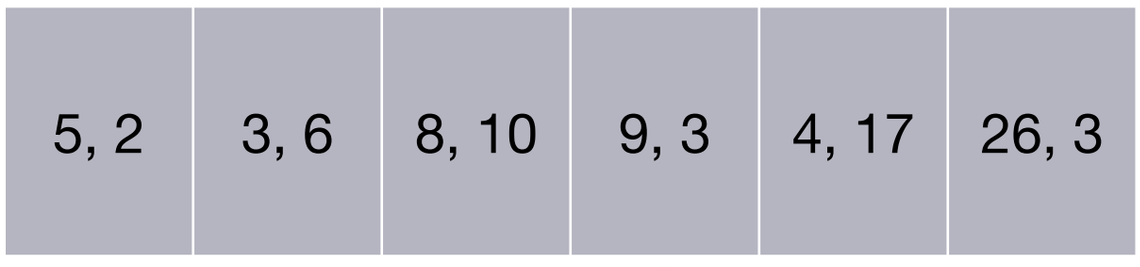


Table 1

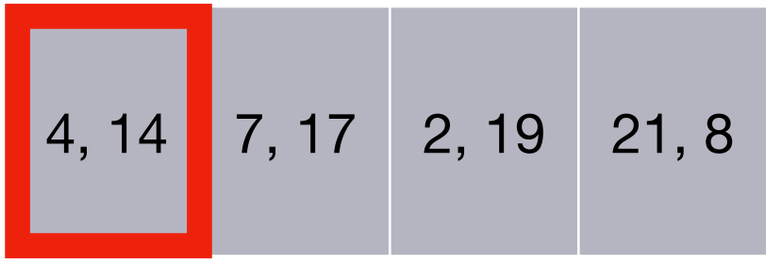
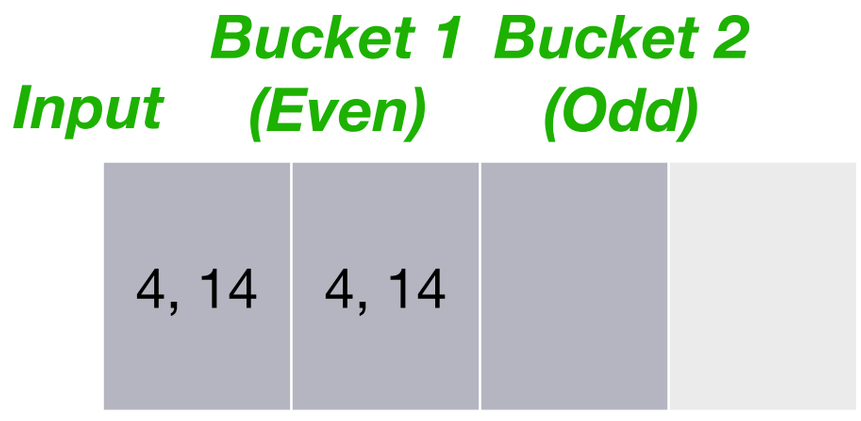


Table 2

Hash Join - Phase 1



2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

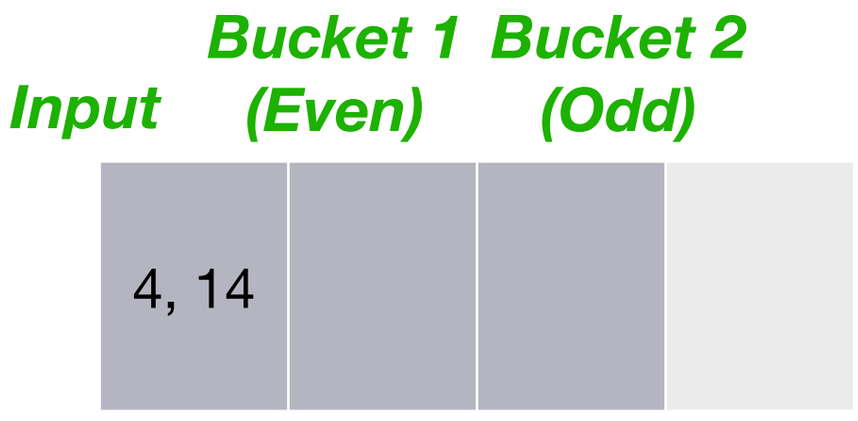
5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 1



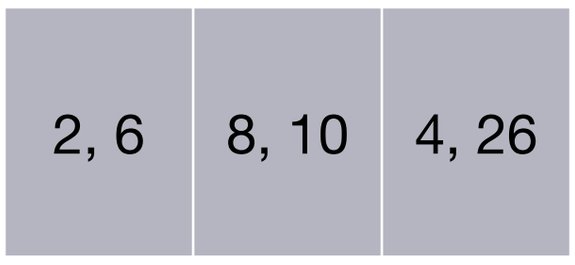


Table 1: Bucket 1

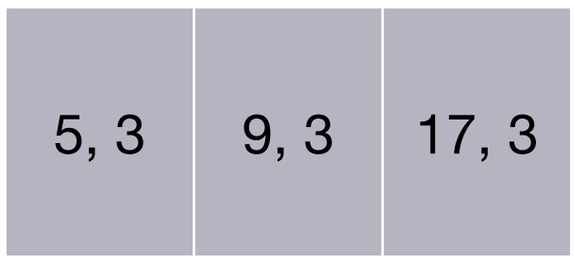


Table 1: Bucket 2



Table 2: Bucket 1

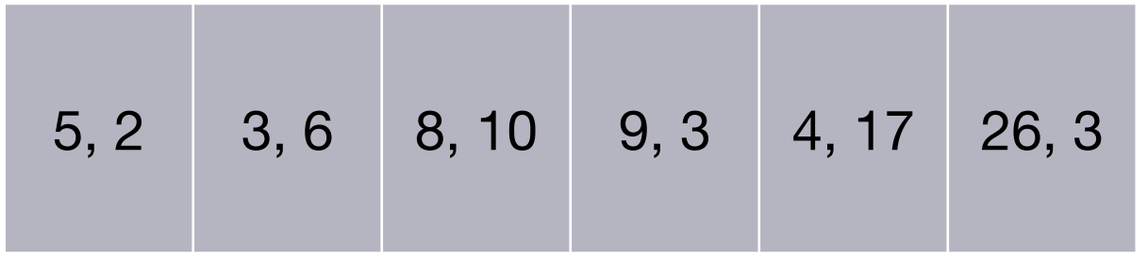


Table 1

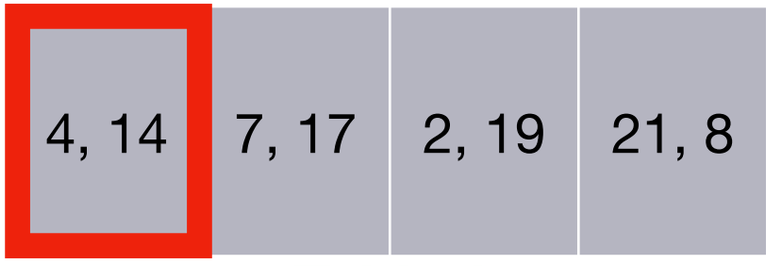
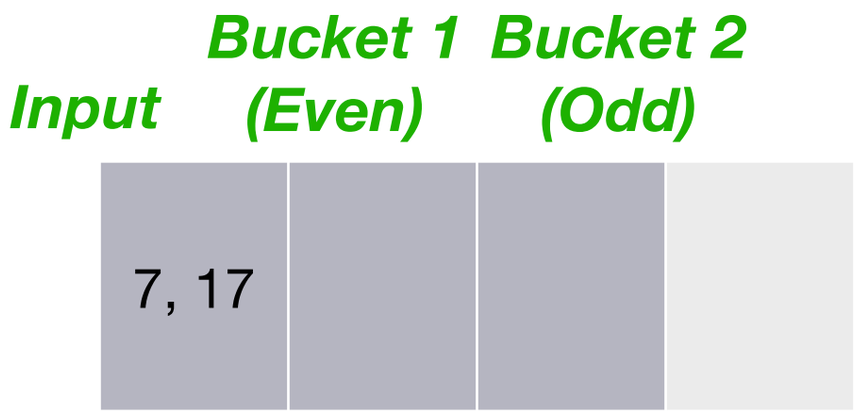


Table 2

Hash Join - Phase 1



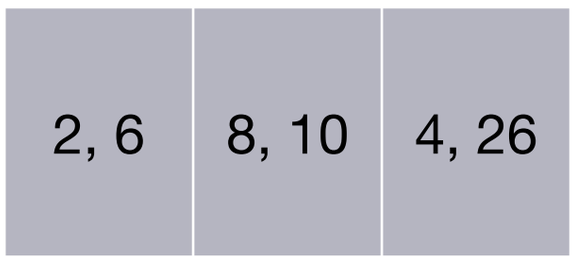


Table 1: Bucket 1

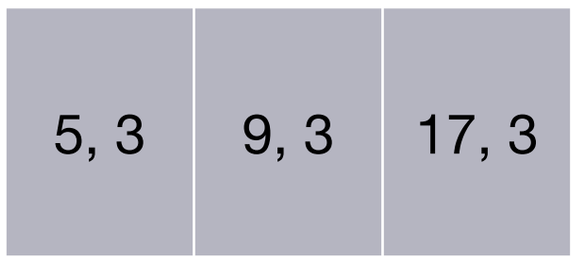


Table 1: Bucket 2



Table 2: Bucket 1

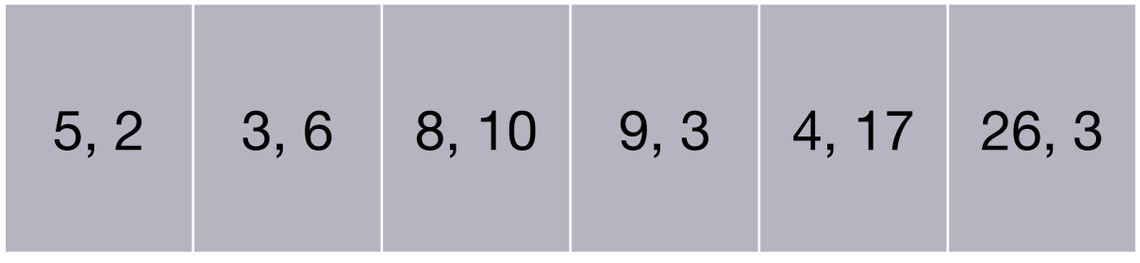


Table 1

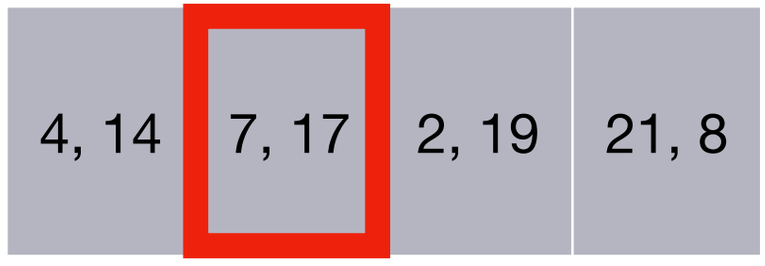
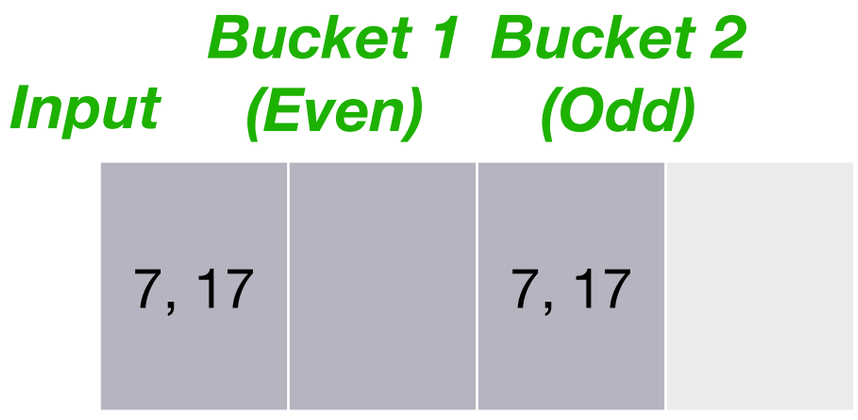


Table 2

Hash Join - Phase 1



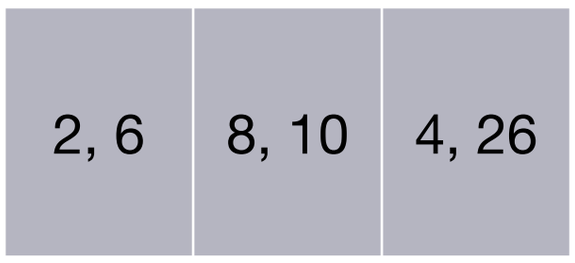


Table 1: Bucket 1

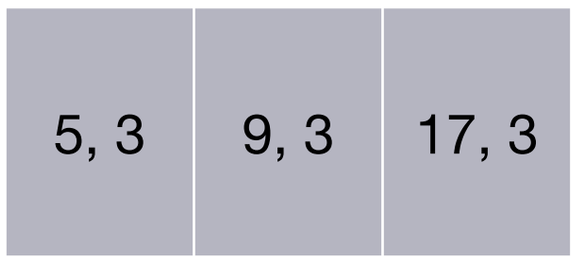


Table 1: Bucket 2



Table 2: Bucket 1

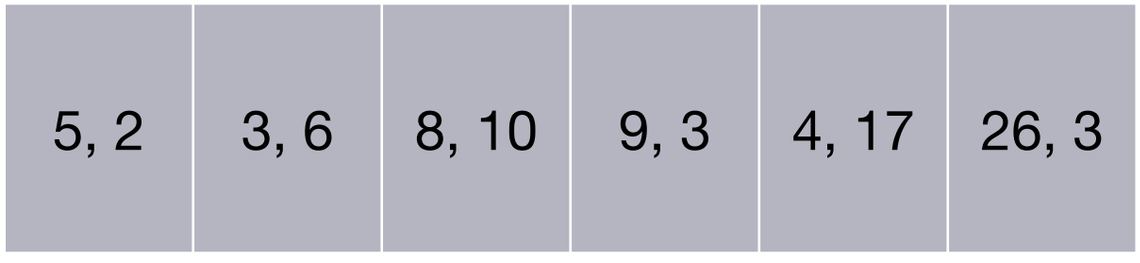


Table 1

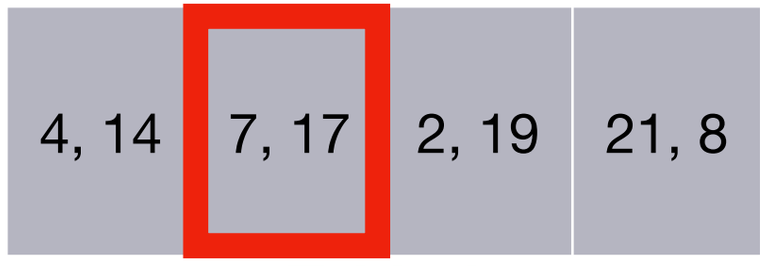
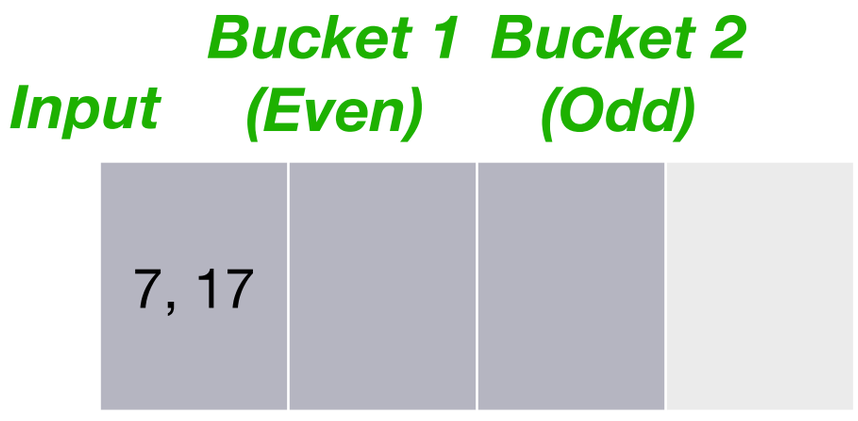
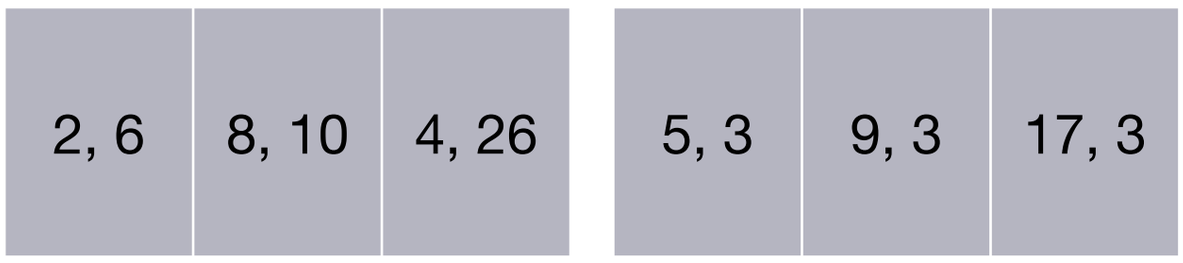


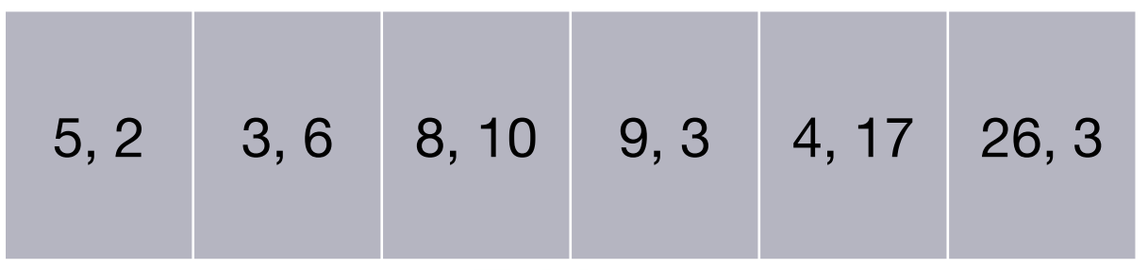
Table 2

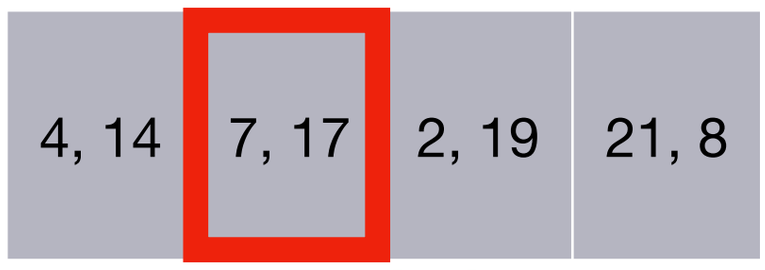
Hash Join - Phase 1



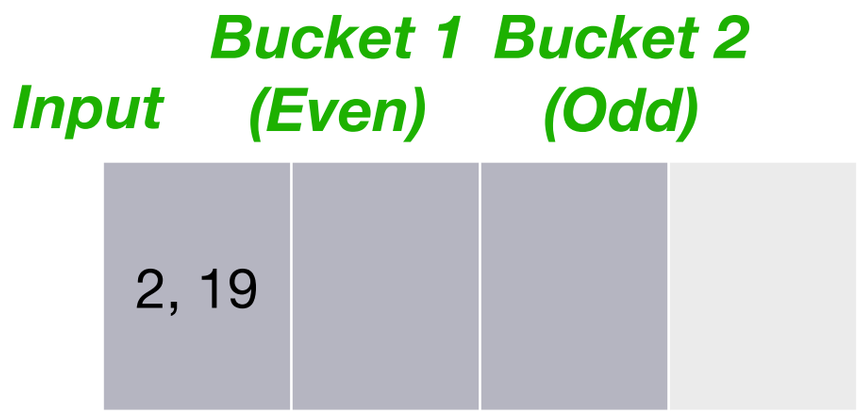


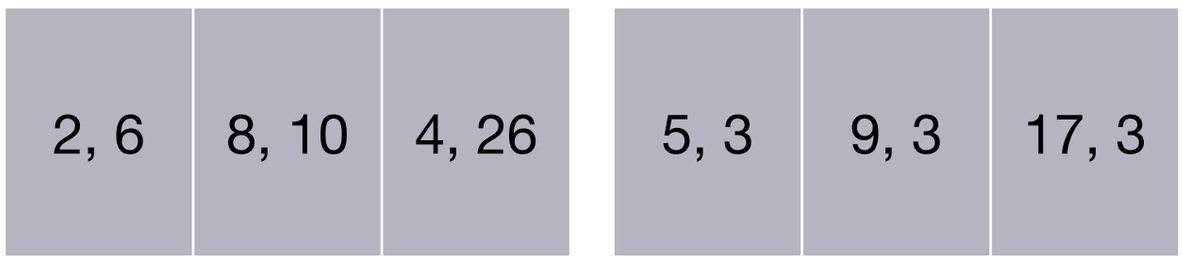




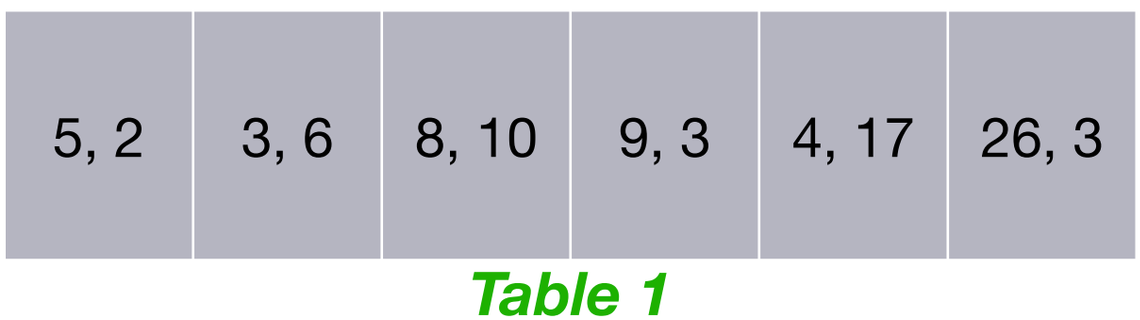


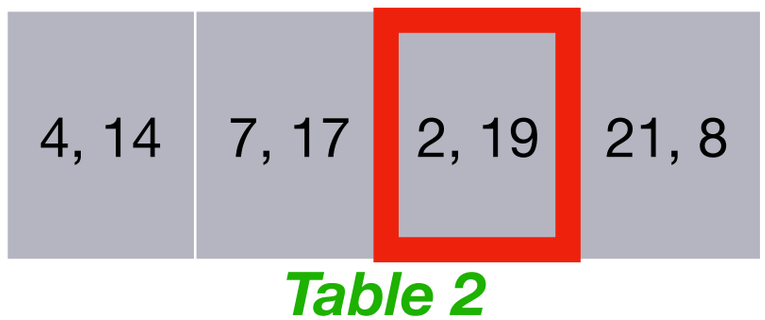
Hash Join - Phase 1



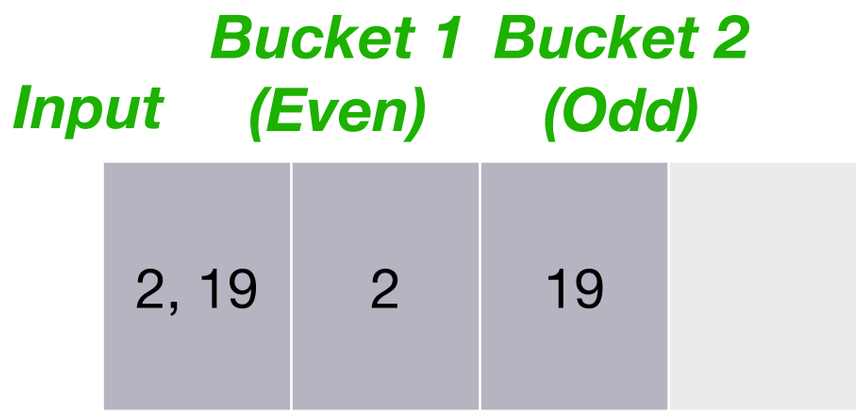


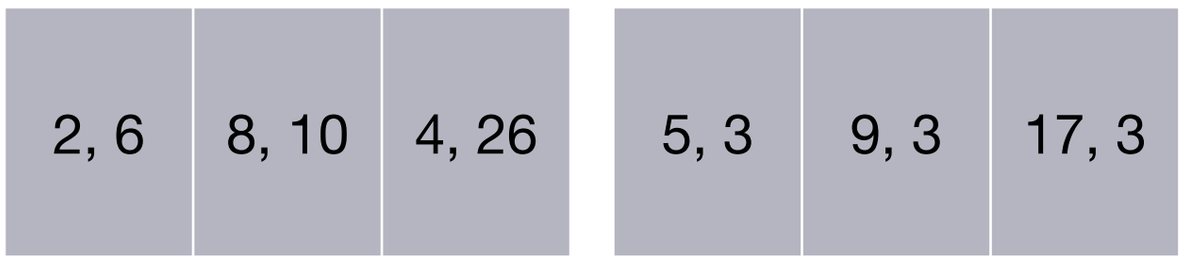




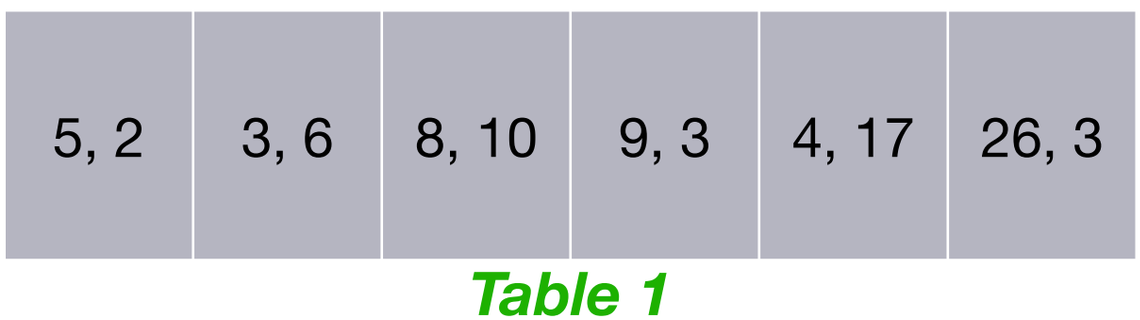


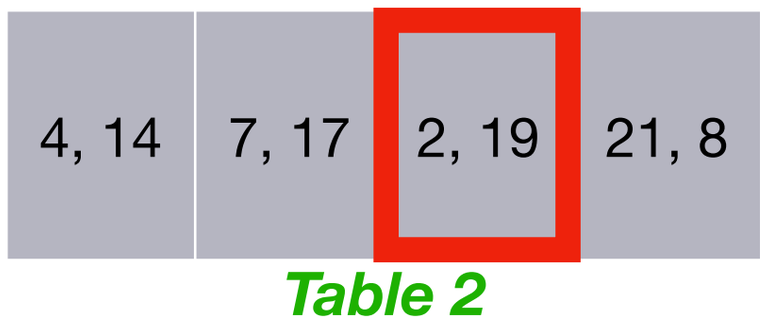
Hash Join - Phase 1



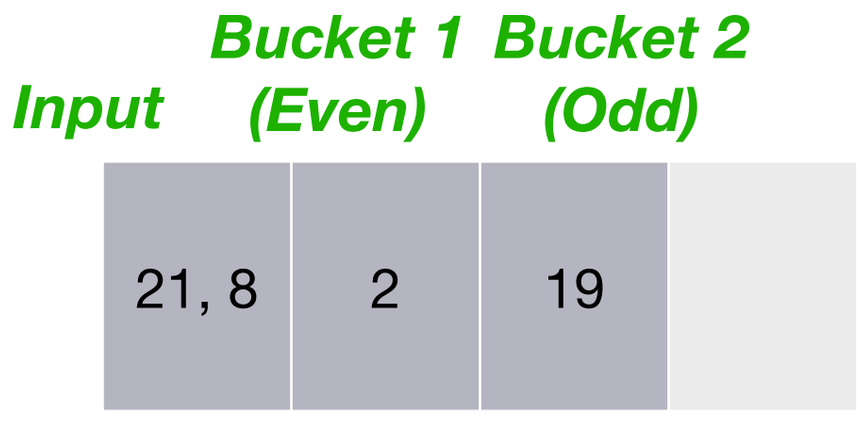


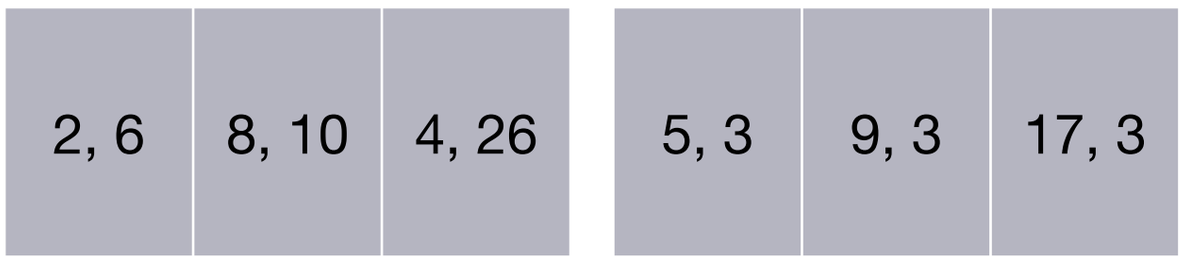




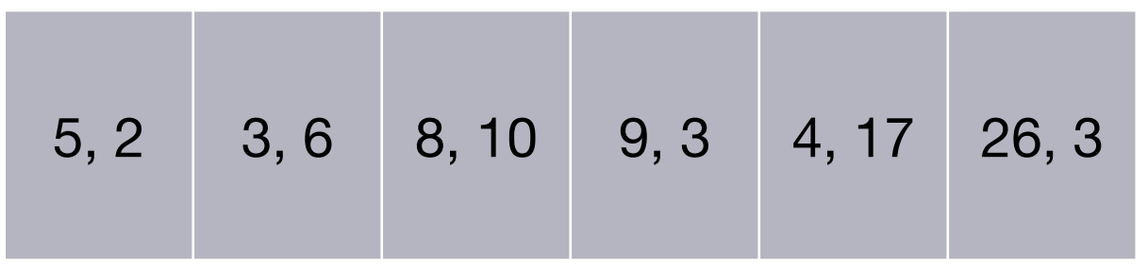


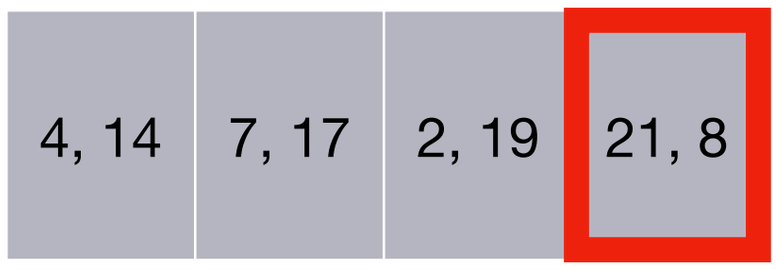
Hash Join - Phase 1



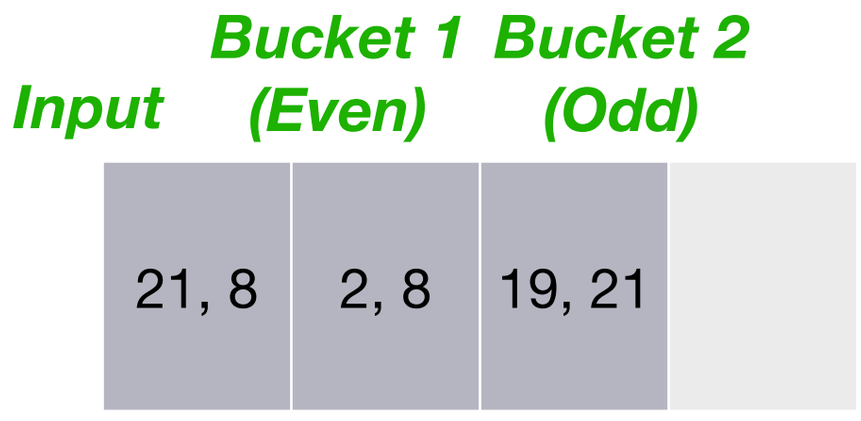


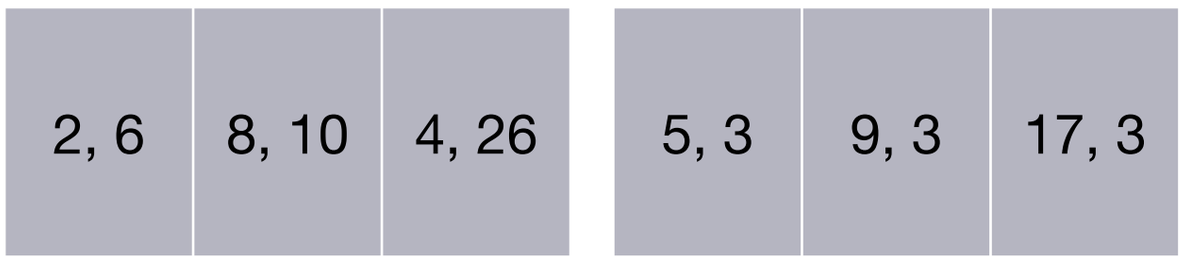




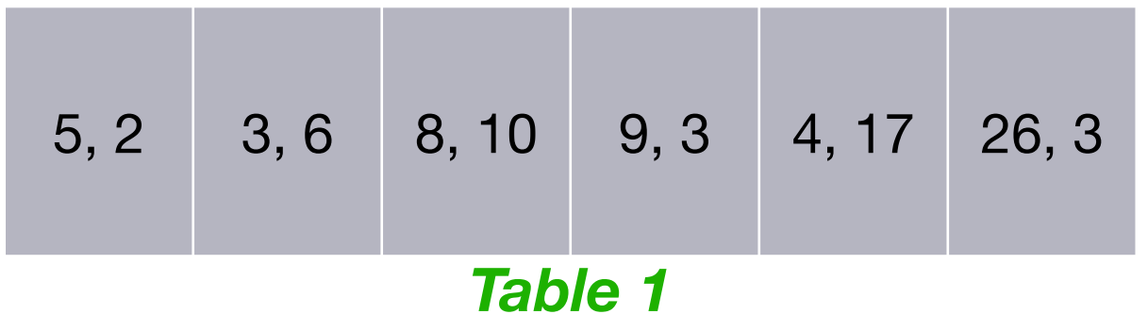


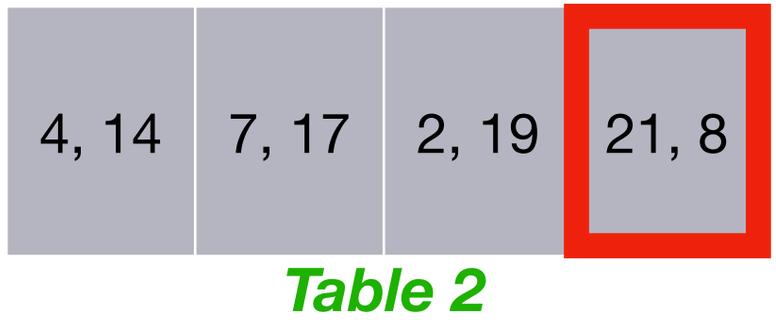
Hash Join - Phase 1



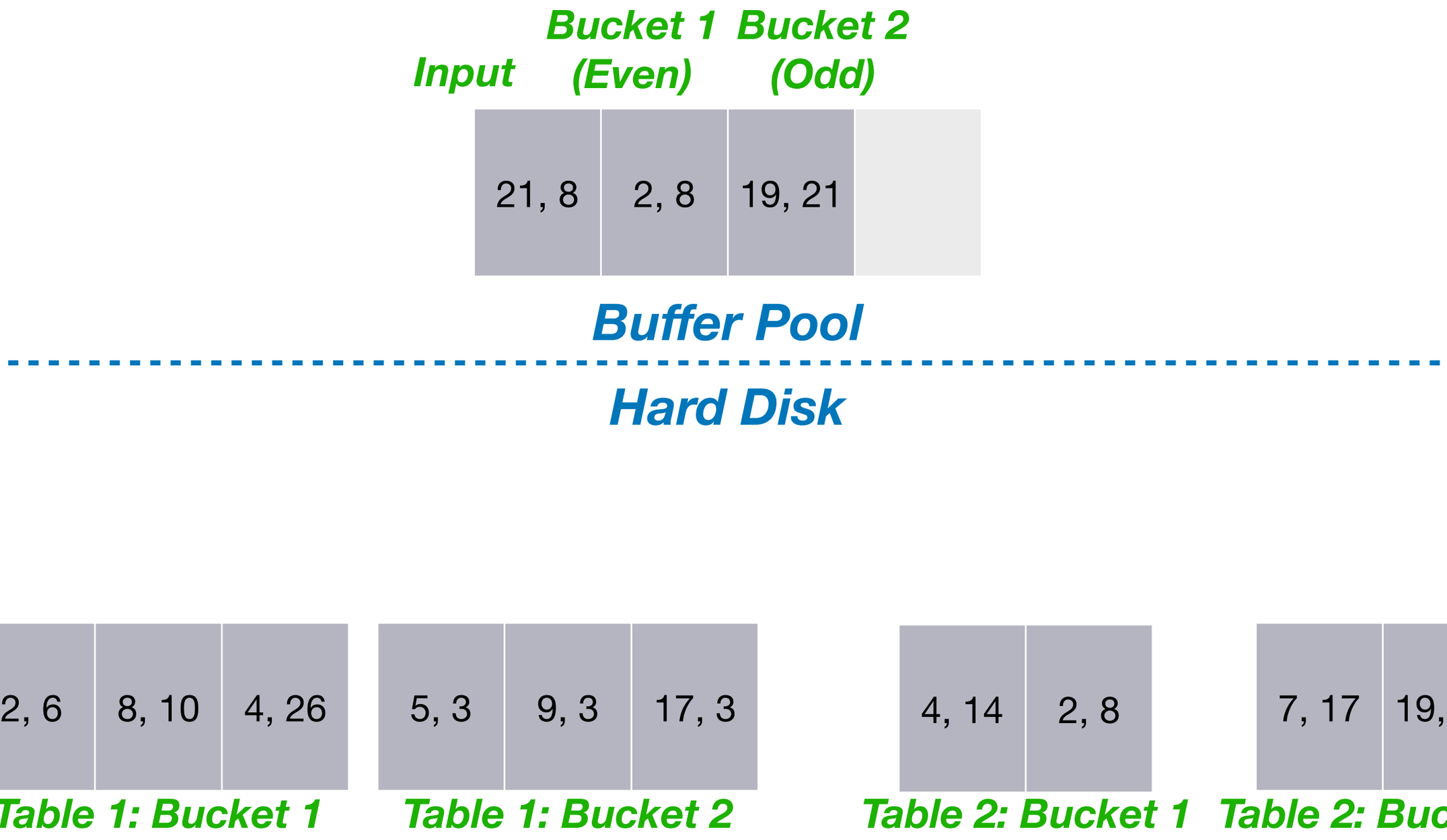




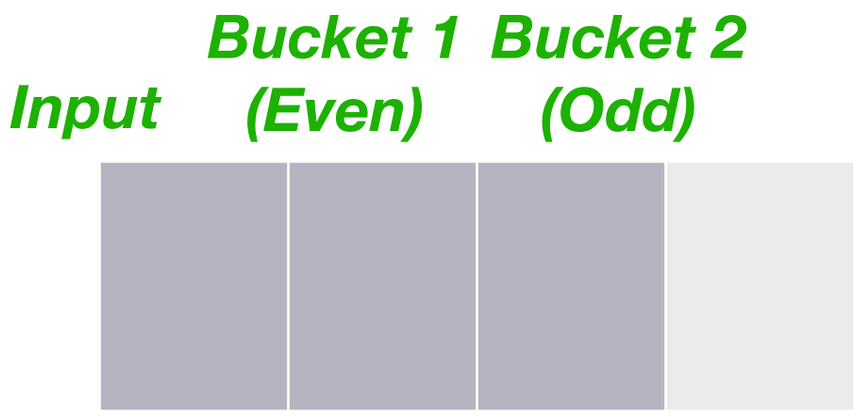




Hash Join - Phase 1



Hash Join - Phase 1



Hard Disk

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

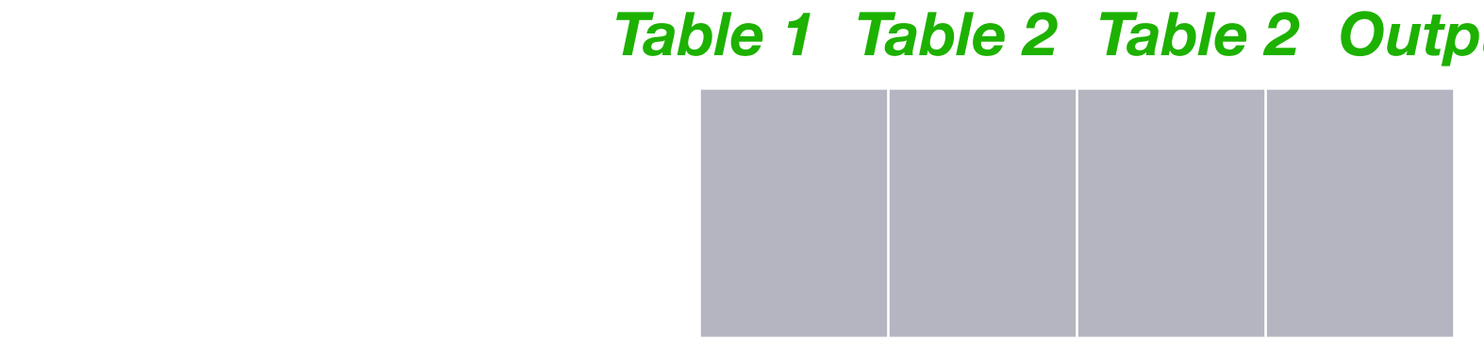
5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2



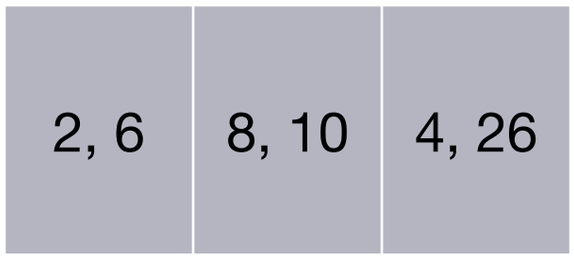


Table 1: Bucket 1

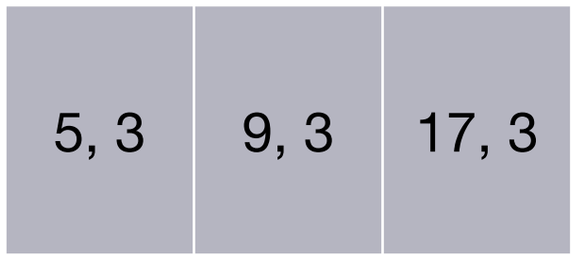


Table 1: Bucket 2

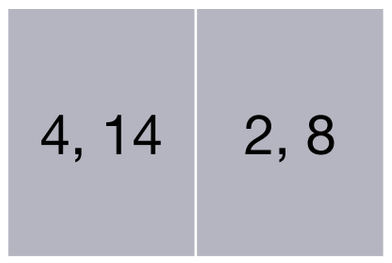


Table 2: Bucket 1

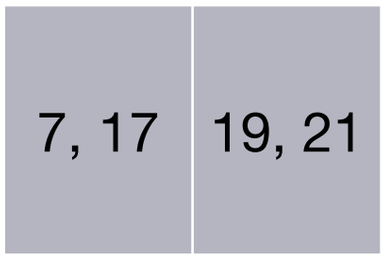


Table 2: Bucket 2

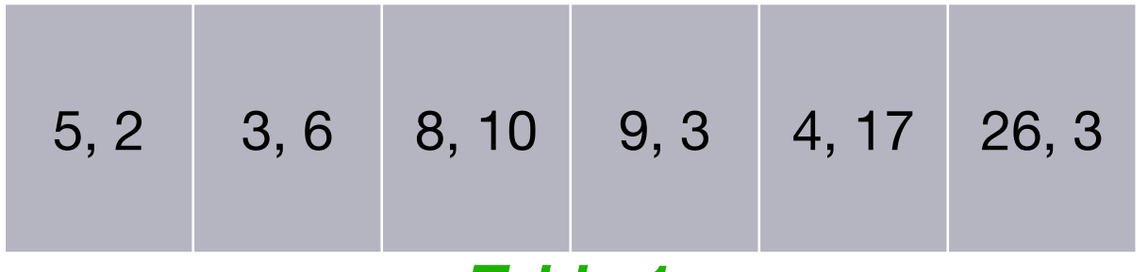


Table 1

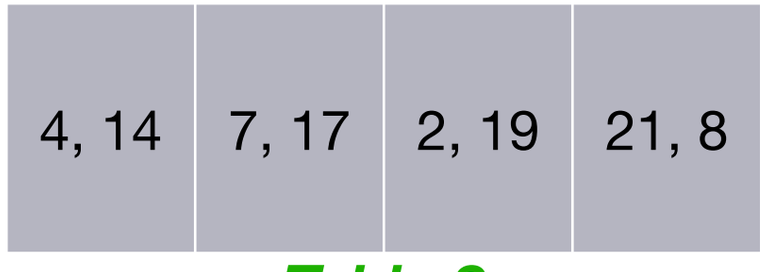
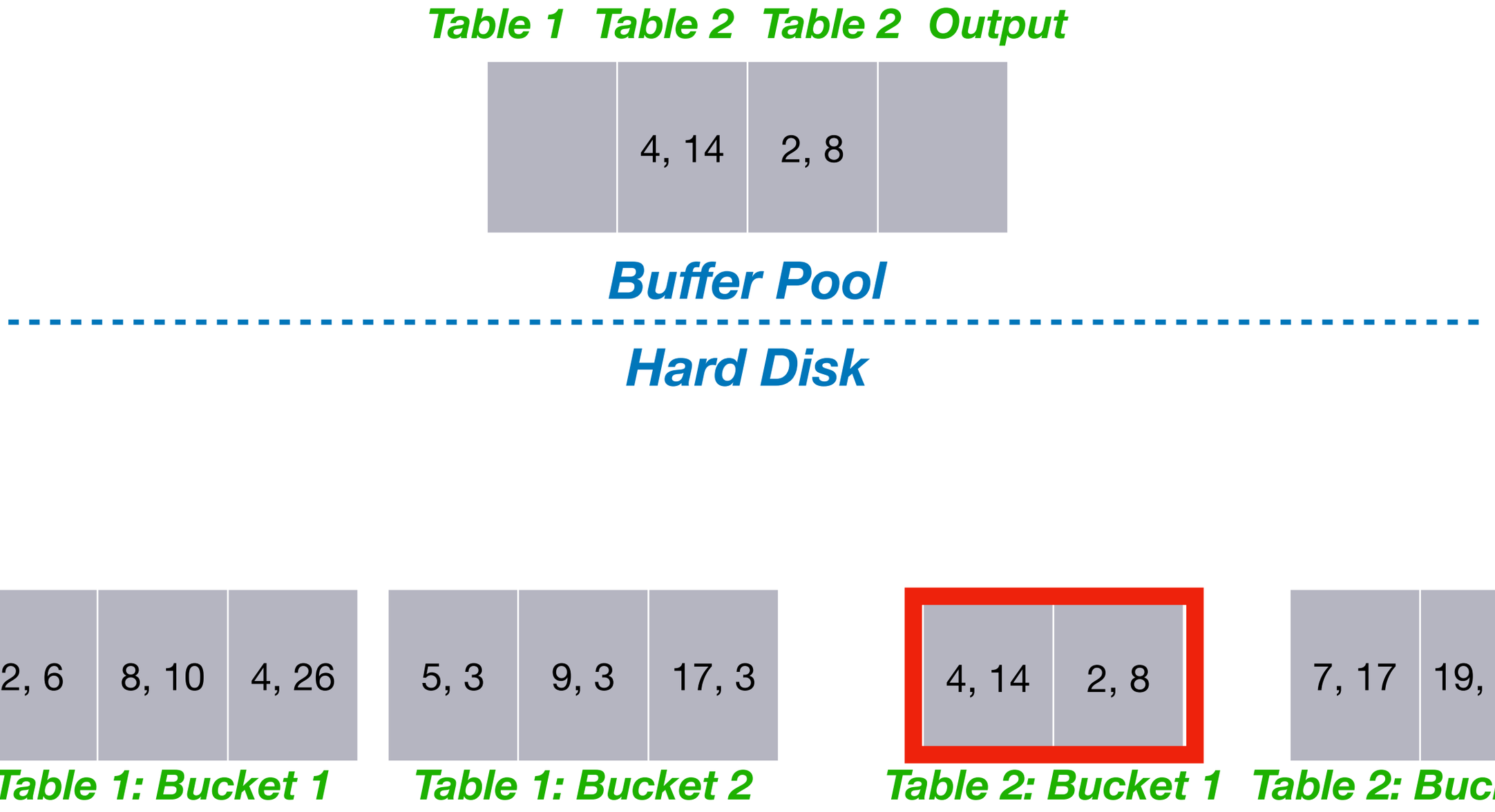


Table 2

Hash Join - Phase 2



Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

2, 6	4, 14	2, 8	

Buffer Pool

Hard Disk

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

2, 6	4, 14	2, 8	2

Buffer Pool

Hard Disk

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

8, 10	4, 14	2, 8	2

Buffer Pool

Hard Disk

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

8, 10	4, 14	2, 8	2, 8

Buffer Pool

Hard Disk

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

8, 10	4, 14	2, 8	

Buffer Pool

Hard Disk

2, 8

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

4, 26	4, 14	2, 8	

Buffer Pool

Hard Disk

2, 8

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

4, 26	4, 14	2, 8	4

Buffer Pool

Hard Disk

2, 8

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

	7, 17	19, 21	4

Buffer Pool

Hard Disk

2, 8

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

5, 3	7, 17	19, 21	4

Buffer Pool

Hard Disk

2, 8

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

9, 3	7, 17	19, 21	4

Buffer Pool

Hard Disk

2, 8

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

17, 3	7, 17	19, 21	4

Buffer Pool

Hard Disk

2, 8

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

17, 3	7, 17	19, 21	4, 17

Buffer Pool

Hard Disk

2, 8

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*

17, 3	7, 17	19, 21	

Buffer Pool

Hard Disk

2, 8	4, 17

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

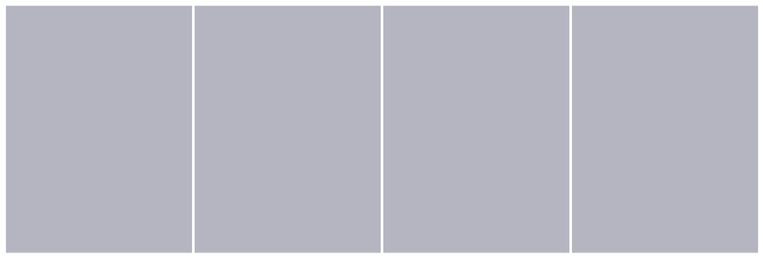
Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Hash Join - Phase 2

Table 1 *Table 2* *Table 2* *Output*



Buffer Pool

Hard Disk

2, 8	4, 17

Result

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Cost Analysis

- Join **Phase 1**:
    - Read data and write partitioned data
- Join **Phase 2**:
    - Read data (and write result)

Cost Analysis

- Join **Phase 1**: ***2 * (#Pages T1 + #Pages T2)***
  - Read data and write partitioned data
- Join **Phase 2**:
  - Read data (and write result)

Cost Analysis

- Join **Phase 1**: ***2 * (#Pages T1 + #Pages T2)***
    - Read data and write partitioned data
- Join **Phase 2**:
    - Read data (~~and write result~~)

Cost Analysis

- Join **Phase 1**: ***2 * (#Pages T1 + #Pages T2)***
  - Read data and write partitioned data
- Join **Phase 2**: ***1 * (#Pages T1 + #Pages T2)***
  - Read data (~~and write result~~)

Cost Analysis

- Join **Phase 1**: ***2 * (#Pages T1 + #Pages T2)***
  - Read data and write partitioned data
- Join **Phase 2**: ***1 * (#Pages T1 + #Pages T2)***
  - Read data (~~and write result~~)

= 3 * (#Pages T1 + #Pages T2)

Example

#Enrollment Pages	1,000
#Student Pages	100
Hash Join Cost	?

Example

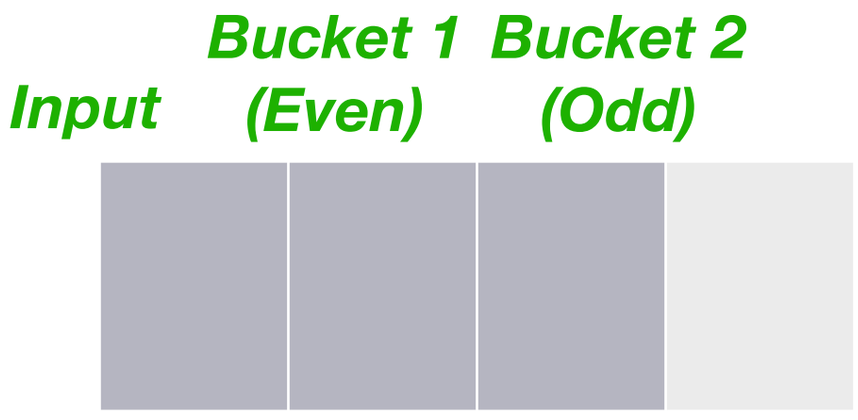
#Enrollment Pages	1,000
#Student Pages	100
Hash Join Cost	**3,300**

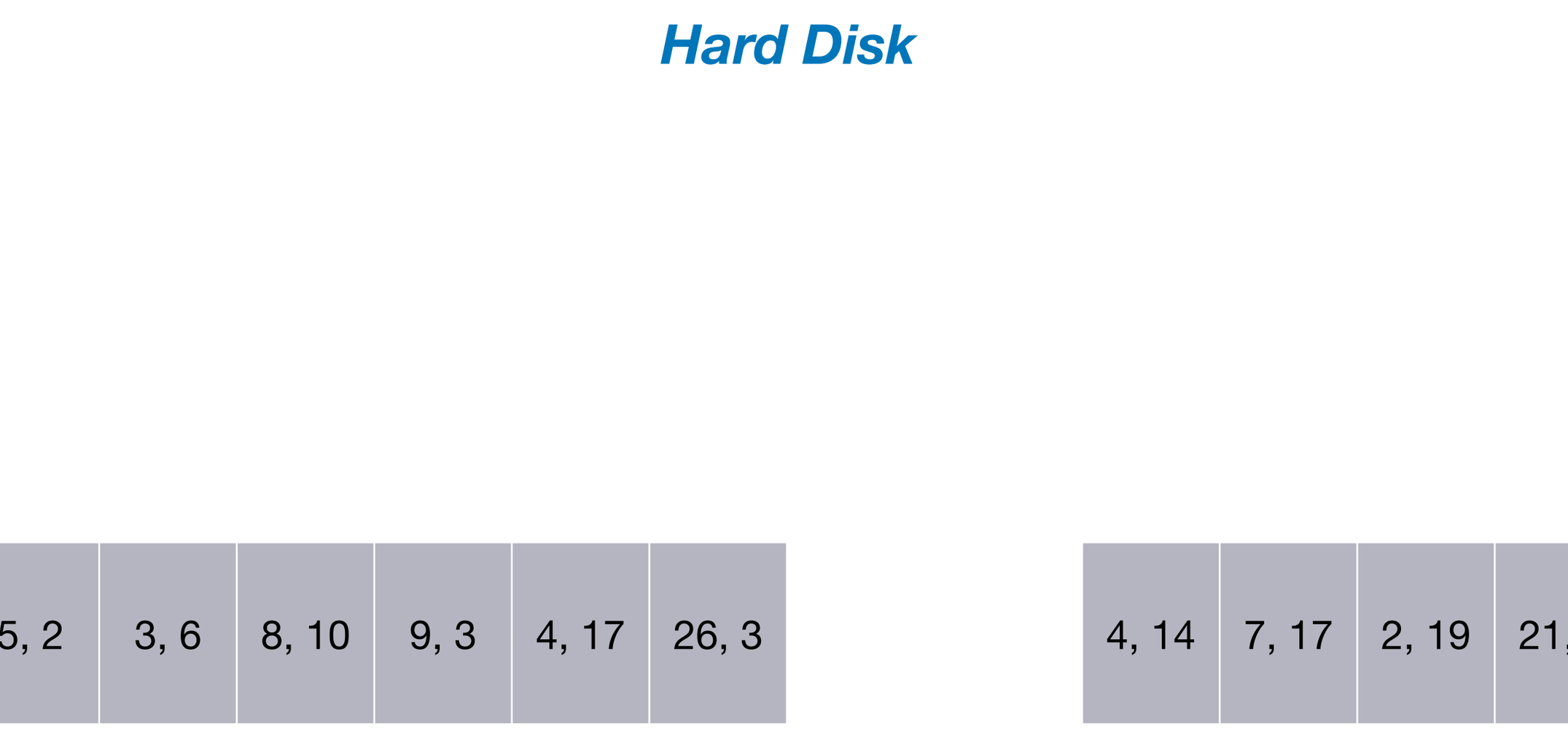
3*(100+1,000)

Cost Comparison

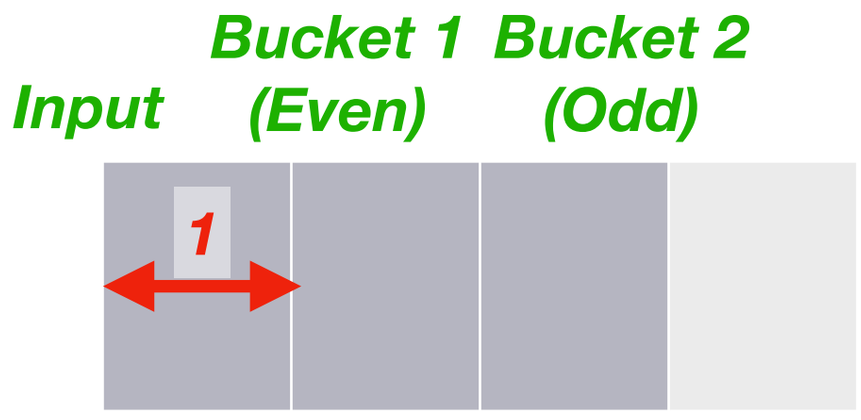
Operator	Cost
Page Nested Loop Join	101,000
Block Nested Loop Join	11,000
Hash Join Cost	3,300

Memory - Phase 1





Memory - Phase 1



Buffer Pool

Hard Disk

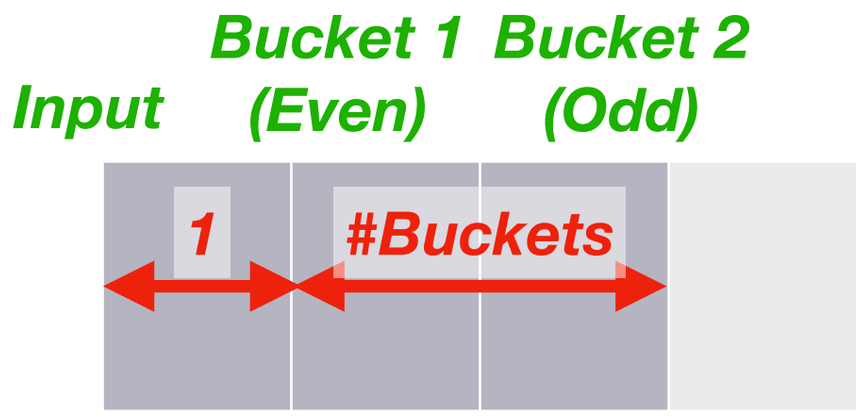
5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Memory - Phase 1



Buffer Pool

Hard Disk

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Memory - Phase 2

Table 1 *Table 2* *Table 2* *Output*



Buffer Pool

Hard Disk

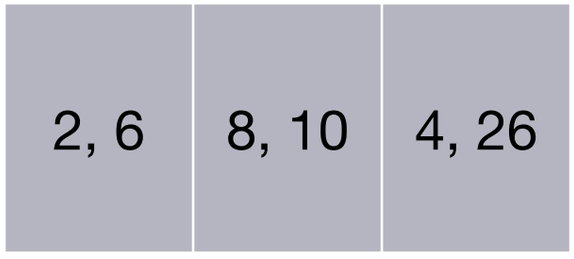


Table 1: Bucket 1

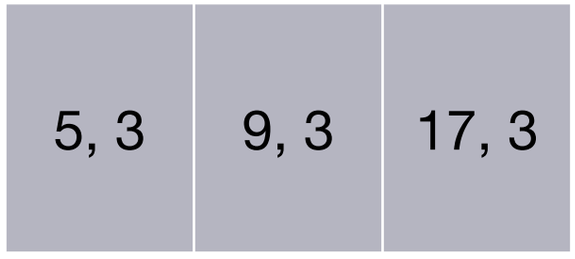


Table 1: Bucket 2

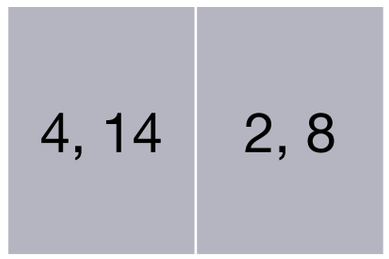


Table 2: Bucket 1

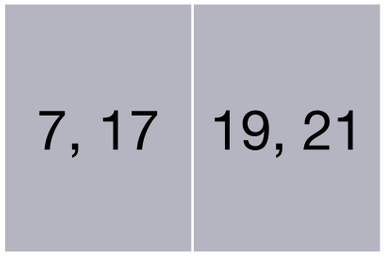


Table 2: Bucket 2

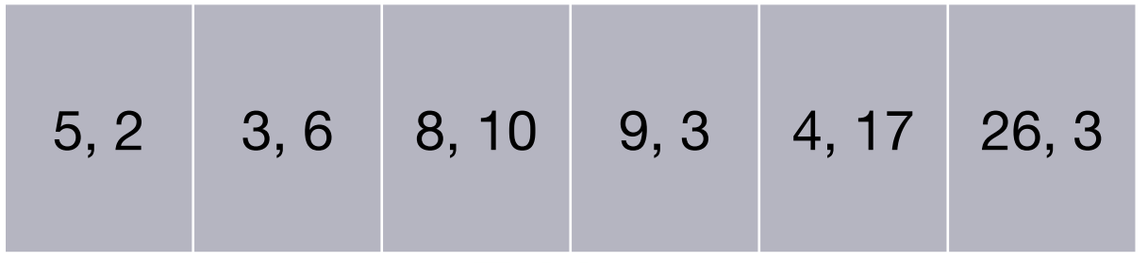


Table 1

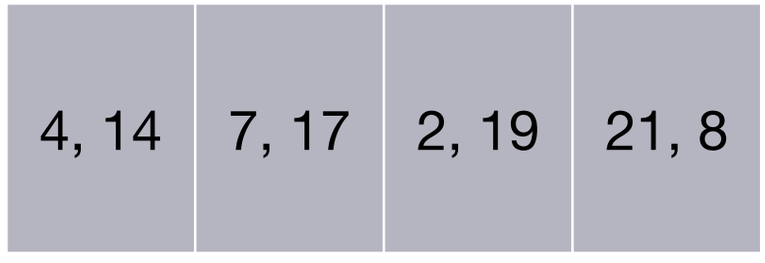
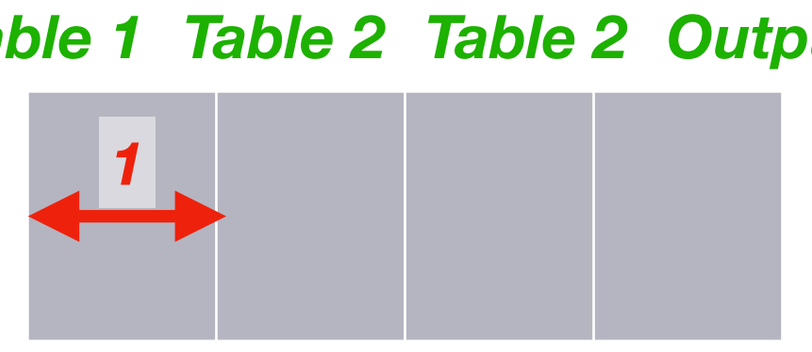


Table 2

Memory - Phase 2



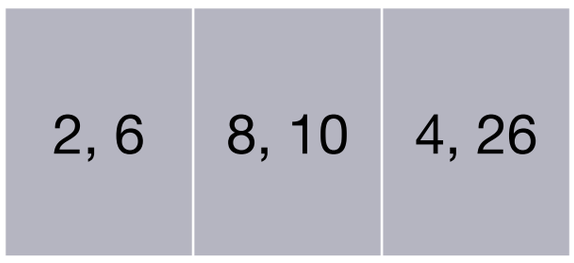


Table 1: Bucket 1

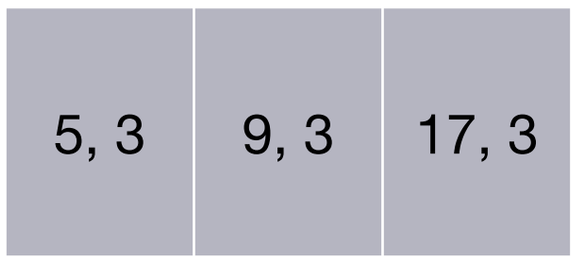


Table 1: Bucket 2

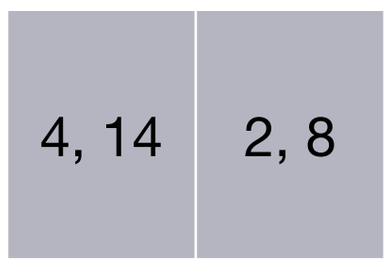


Table 2: Bucket 1

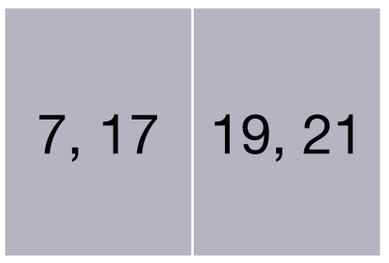


Table 2: Bucket 2

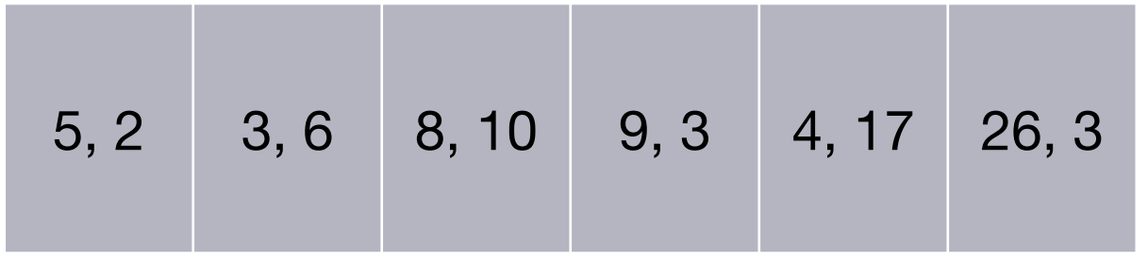


Table 1

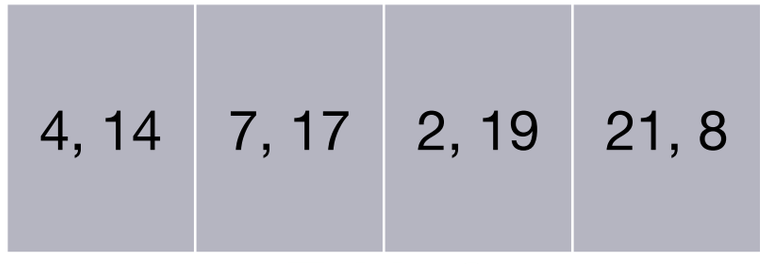
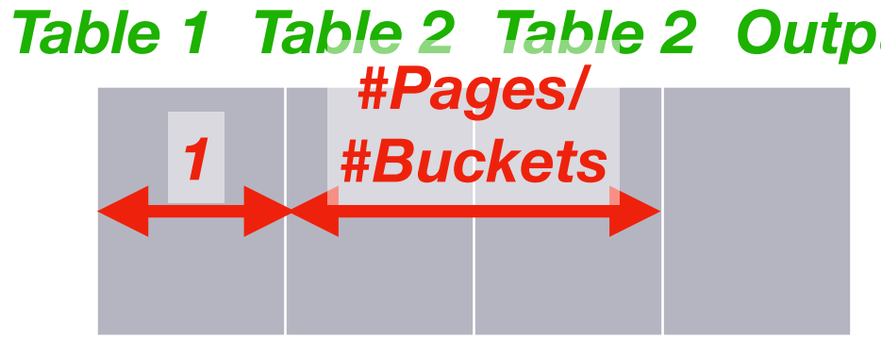


Table 2

Memory - Phase 2



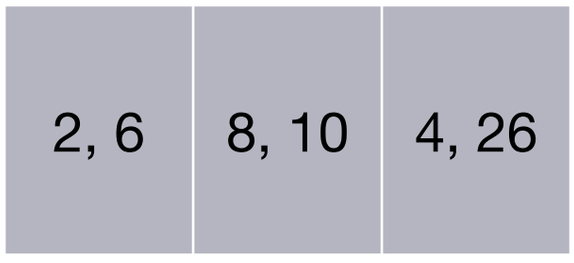


Table 1: Bucket 1

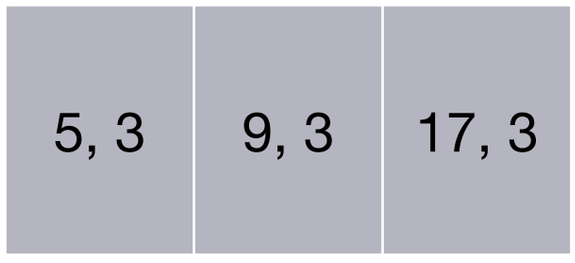


Table 1: Bucket 2

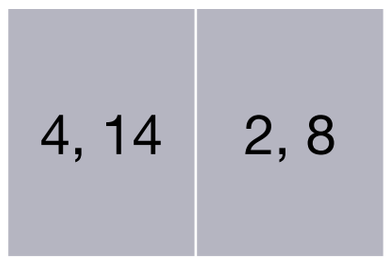


Table 2: Bucket 1

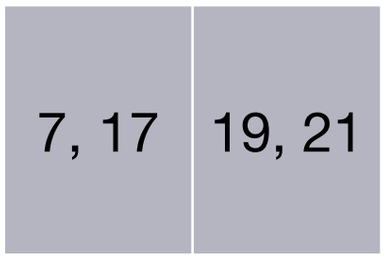


Table 2: Bucket 2

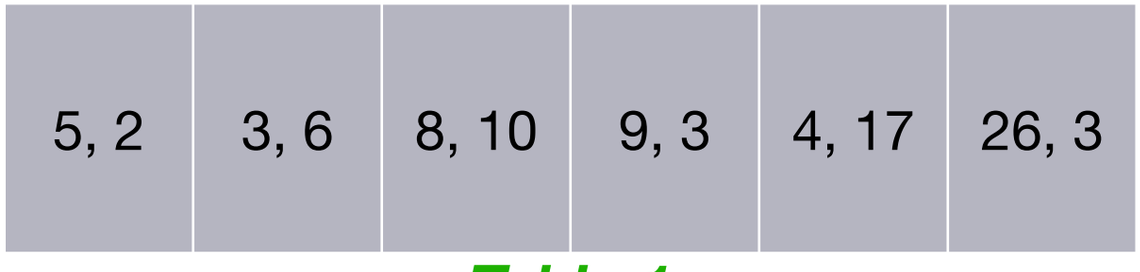


Table 1

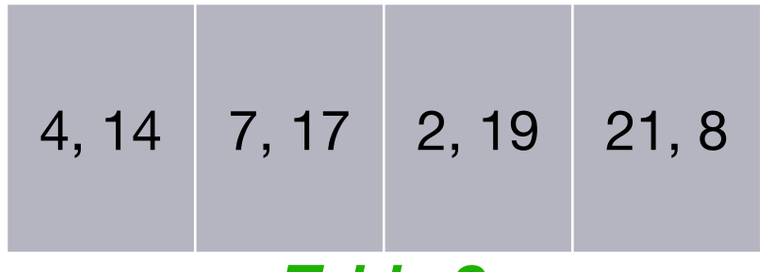
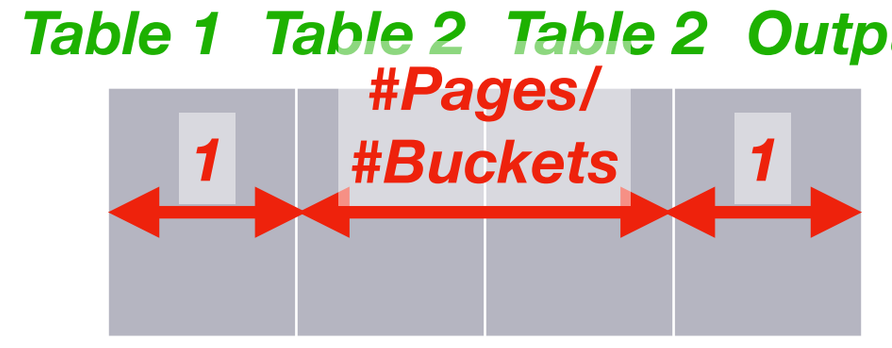


Table 2

Memory - Phase 2



Buffer Pool

Hard Disk

2, 6	8, 10	4, 26

Table 1: Bucket 1

5, 3	9, 3	17, 3

Table 1: Bucket 2

4, 14	2, 8

Table 2: Bucket 1

7, 17	19, 21

Table 2: Bucket 2

5, 2	3, 6	8, 10	9, 3	4, 17	26, 3

Table 1

4, 14	7, 17	2, 19	21, 8

Table 2

Memory Requirements

- **Phase 1**
  - 1 + #Buckets
- **Phase 2**
  - 2 + #Pages (smaller table) / #Buckets

Memory Requirements

- **Phase 1**
  - 1 + #Buckets
- **Phase 2**
  - 2 + #Pages (smaller table) / #Buckets

How to Choose #Buckets?

Memory Requirements

- **Phase 1**
  - 1 + #Buckets
- **Phase 2**
  - 2 + #Pages (smaller table) / #Buckets

#Buckets = √(#Pages (smaller table))

What If We Lack Memory?

- Number of buffer pages limits number of **output buckets**
- Not enough buckets means **too much data** per bucket
- Prevents us from **loading one bucket** entirely in Phase 2
- Hence, perform **multiple passes** over data in phase 1
  - In each pass, buckets are partitioned into **sub-buckets**
  - Iterate until data per bucket **fits** into main memory

?

Sort-Merge Join: Idea

- Also specific to **equality** join conditions
- **Phase 1 (Sort)**
  - **Sort** joined tables on the join column
- **Phase 2 (Merge)**
  - Efficiently **merge** sorted tables together

Join Phase 1: Overview

- Lots of **sorting algorithms** proposed in the literature
- However, typically assume that we access **single** entries
- But random data access can be very **inefficient**
- Hence, want to access **pages** of entries instead
- Need specialized ("**external**") sort algorithms

Algorithm Sketch

- **Step 1**: load chunk of data and **sort**, write back to disk
- **Step 2 .. n**: **merge** sorted runs to produce larger runs
- Each merging step **reduces** number of runs (but longer)
- Finally, have only **one sorted run** left - we're done!

Details on Step 1

- Assume we have **B buffer pages** available
- **Load chunks** of B pages into the buffer
- For each chunk, **sort** by standard sort algorithm
- Then, **write** sorted data to hard disk
- A sorted sequence of data is called a "**run**"

Step 1 Example



Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	12, 29	9, 10	15, 3	26, 4	14, 17	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

1, 8	12, 29	9, 10

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	12, 29	9, 10	15, 3	26, 4	14, 17	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

1, 8	9, 10	12, 29

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	12, 29	9, 10	15, 3	26, 4	14, 17	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

1, 8	9, 10	12, 29

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	15, 3	26, 4	14, 17	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

1, 8	9, 10	12, 29

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	15, 3	26, 4	14, 17	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

15, 3	26, 4	14, 17

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	15, 3	26, 4	14, 17	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

3, 4	14, 15	17, 26

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	15, 3	26, 4	14, 17	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

3, 4	14, 15	17, 26

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

19, 54	8, 90	6, 12

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

6, 8	12, 19	54, 90

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	19, 54	8, 90	6, 12	5, 73	2, 42	3, 9

Step 1 Example

6, 8	12, 19	54, 90

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	5, 73	2, 42	3, 9

Step 1 Example

6, 8	12, 19	54, 90

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	5, 73	2, 42	3, 9

Step 1 Example

5, 73	2, 42	3, 9

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	5, 73	2, 42	3, 9

Step 1 Example

2, 3	5, 9	42, 73

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	5, 73	2, 42	3, 9

Step 1 Example

2, 3	3, 5	9, 73

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	2, 3	5, 9	42, 73

Step 1 Example

2, 3	3, 5	9, 73

Buffer Pool (3 Pages)

Hard Disk (12 Pages)

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	2, 3	5, 9	42, 73
		Run 1			**Run 2**			**Run 3**			**Run 4**

Details on Steps 2 .. n

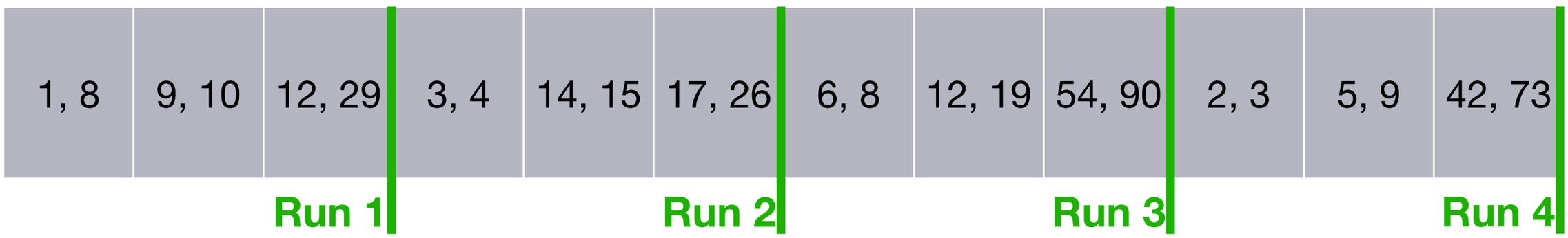
- (Still have B buffer pages available)
- Enables us to **merge B-1 sorted runs** into one in one step
  - **Load first page** of each sorted run into B-1 pages
  - **Copy minimum entry** in input buffers to output buffer
    - If output buffer full, **write to disk and clear**
  - **Erase minimum** entry from input buffer
    - If input buffer becomes empty, **load next page**

Step 2 Example

Input 1 Input 2 Output



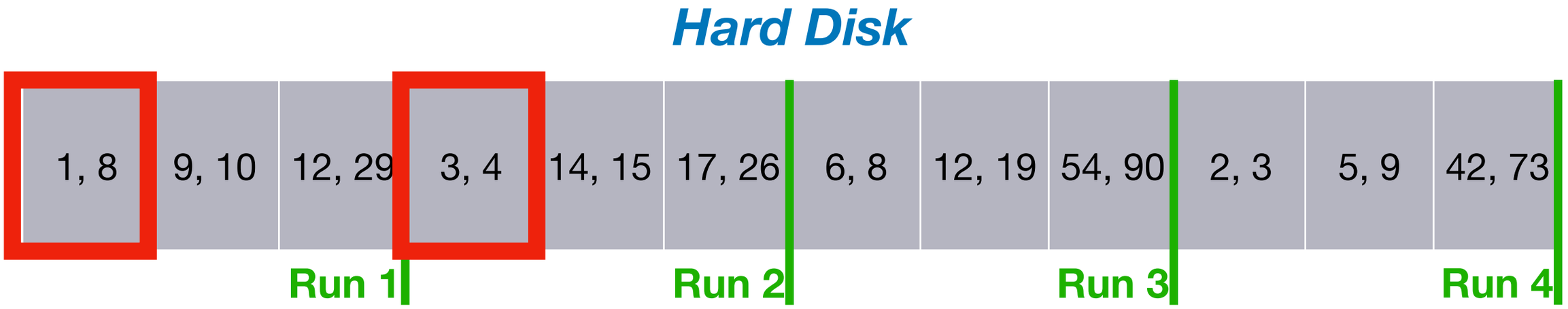
Buffer Pool (3 Pages)



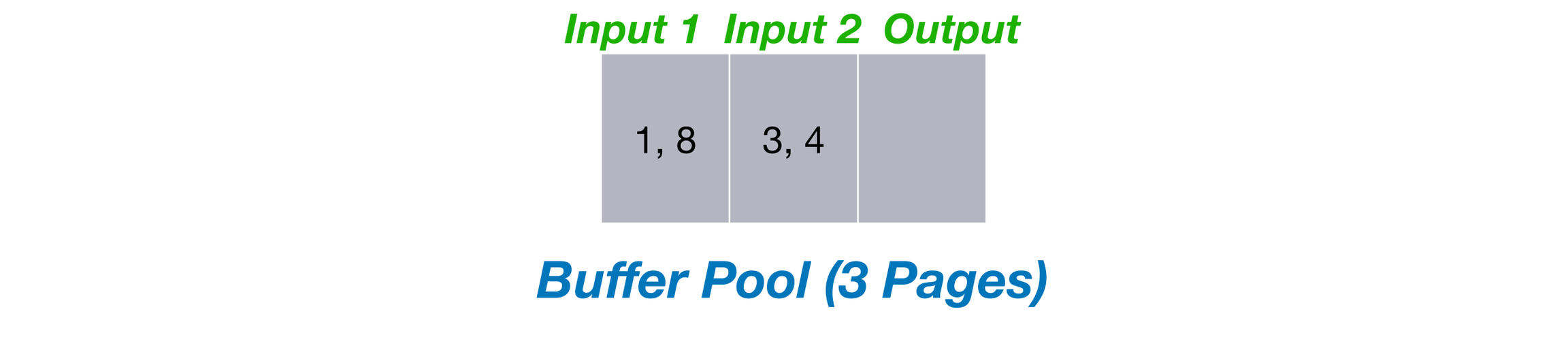
Step 2 Example



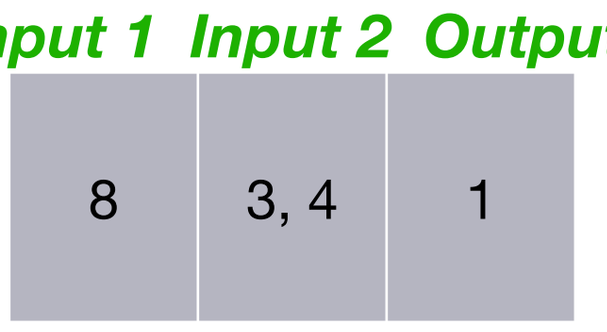
Buffer Pool (3 Pages)

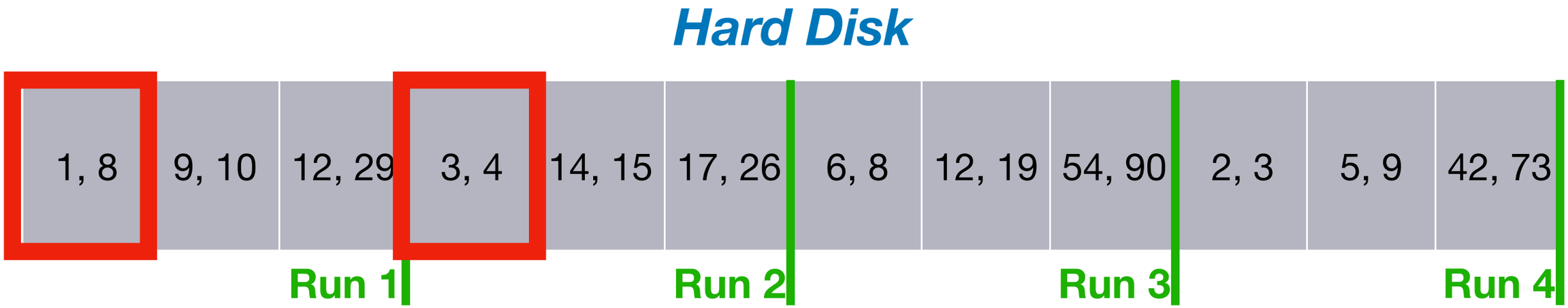


Step 2 Example

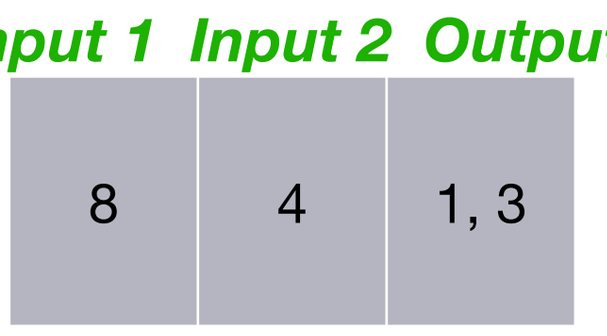


Step 2 Example

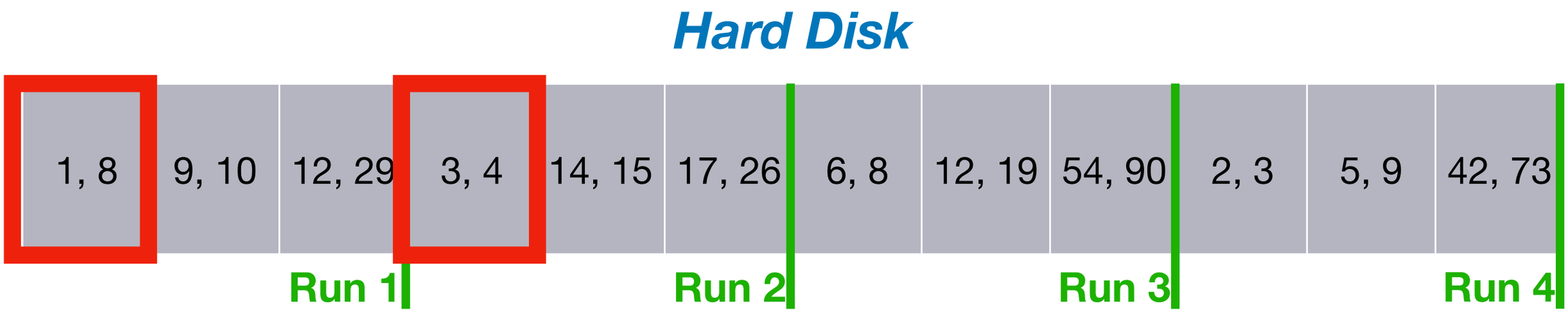




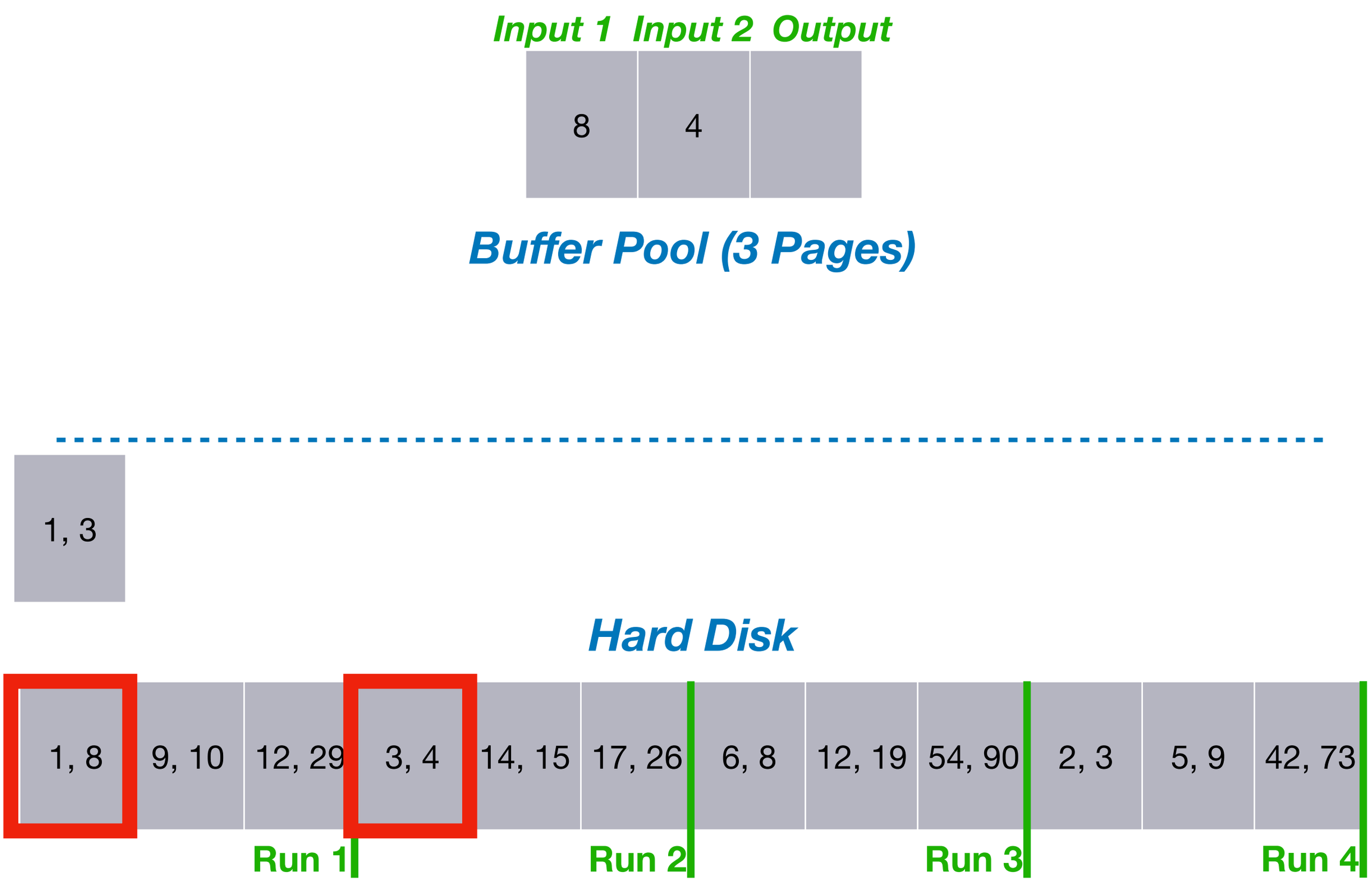
Step 2 Example



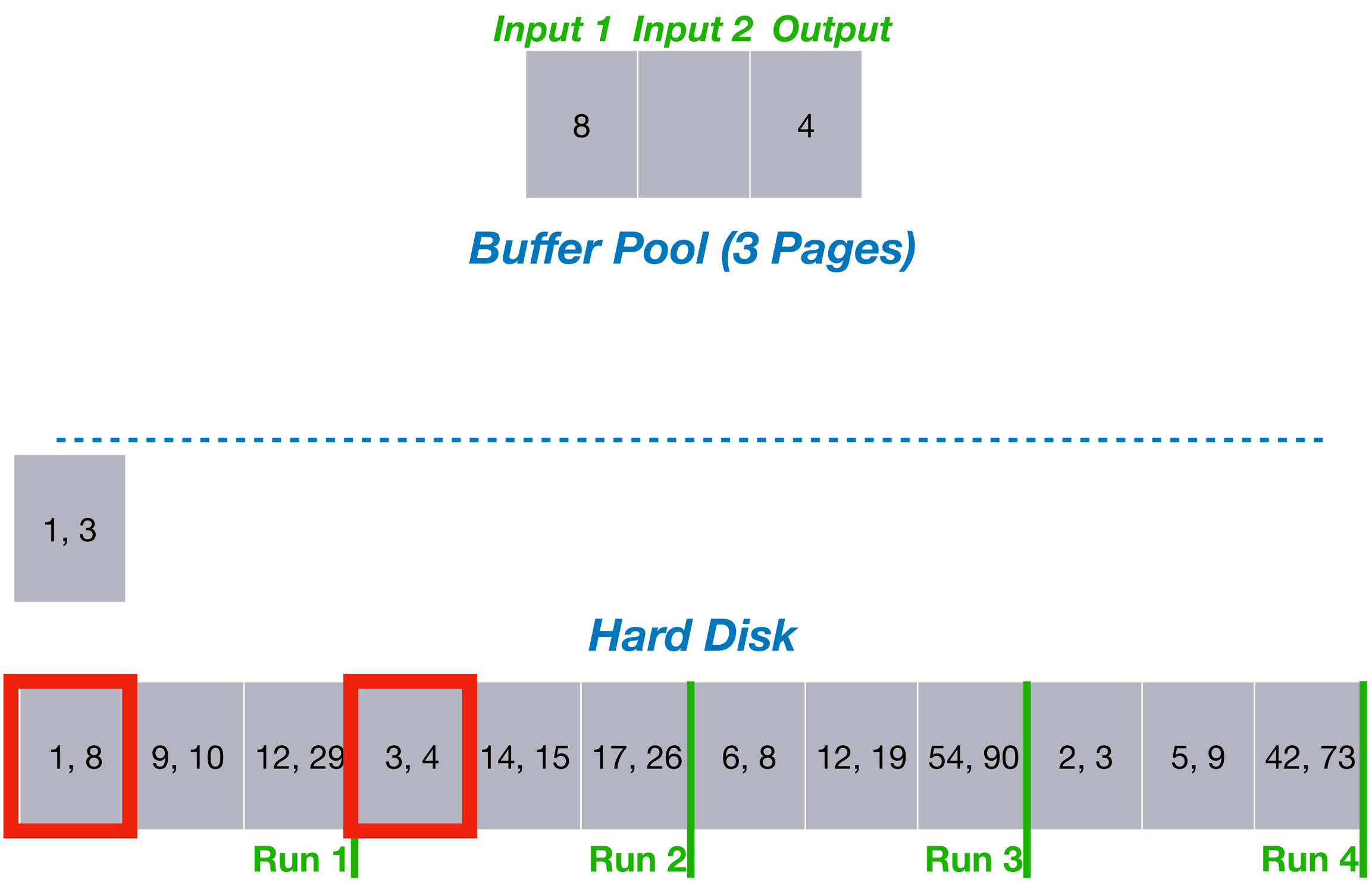
Buffer Pool (3 Pages)



Step 2 Example



Step 2 Example



Step 2 Example

Input 1	*Input 2*	*Output*
8	14, 15	4

Buffer Pool (3 Pages)

1, 3

Hard Disk

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	2, 3	5, 9	42, 73
		Run 1			**Run 2**			**Run 3**			**Run 4**

Step 2 Example

Input 1 Input 2 Output

	14, 15	4, 8

Buffer Pool (3 Pages)

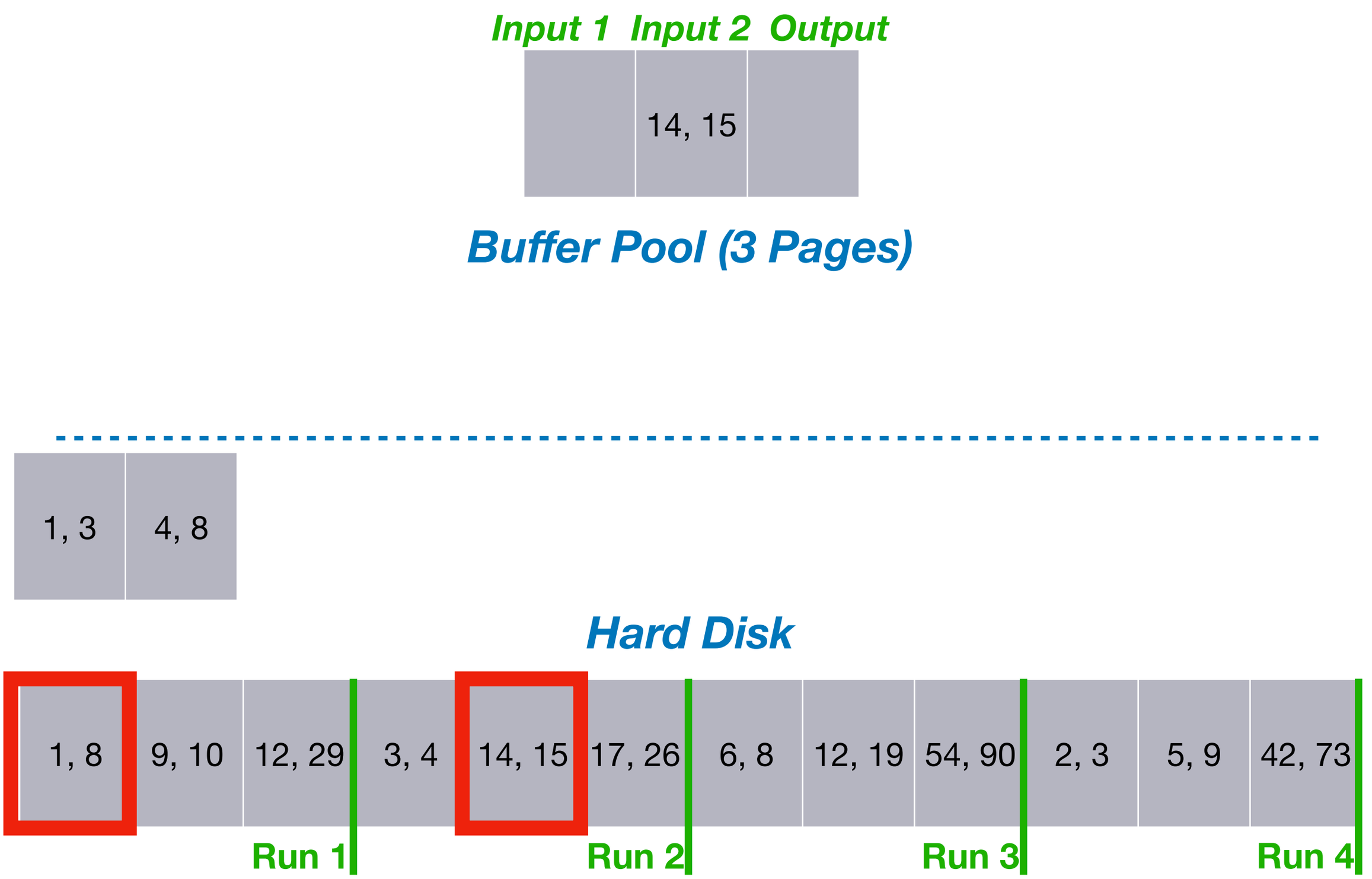
1, 3

Hard Disk

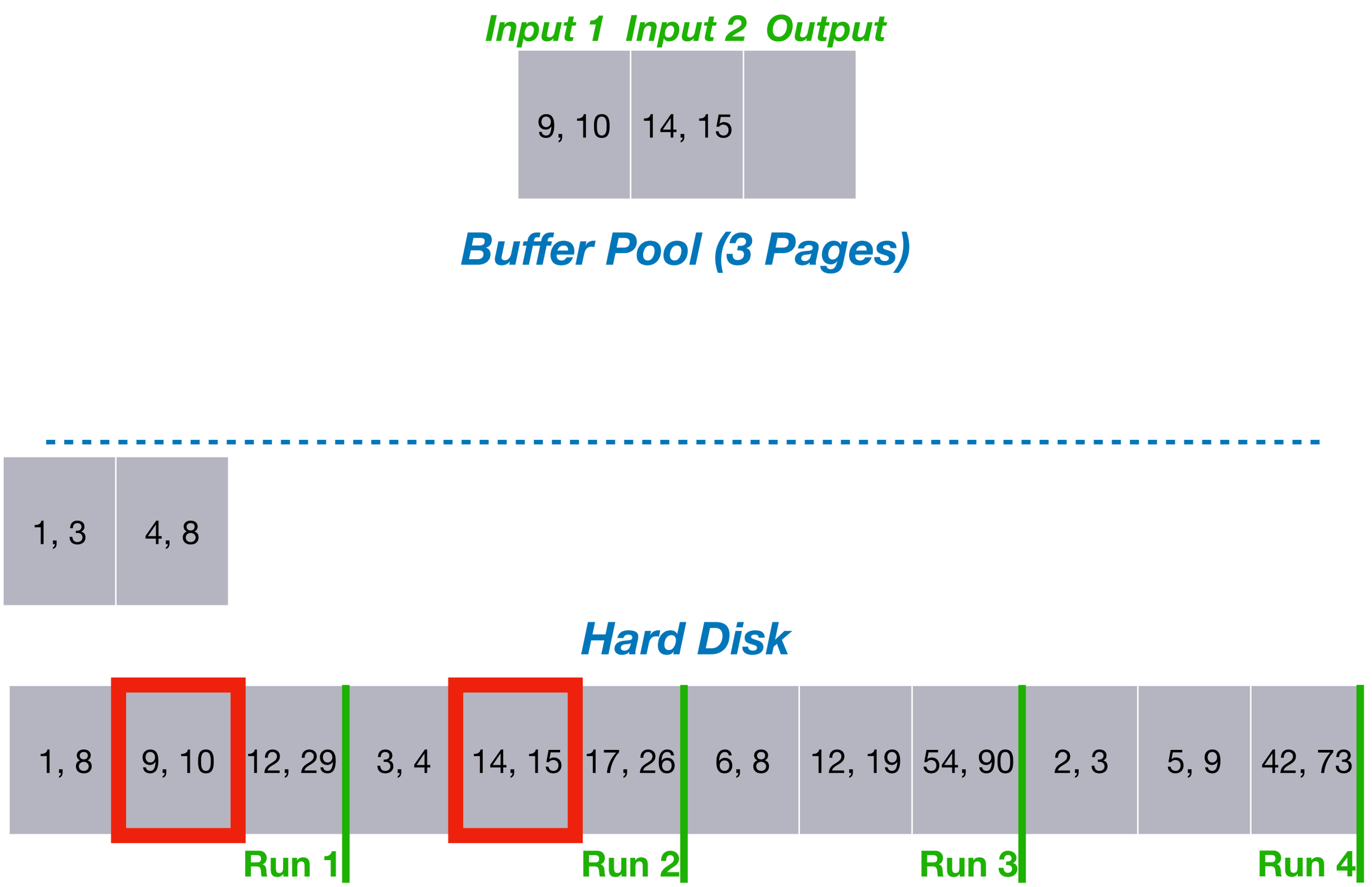
1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	2, 3	5, 9	42, 73

Run 1 **Run 2** **Run 3** **Run 4**

Step 2 Example



Step 2 Example



Step 2 Example

Input 1	*Input 2*	*Output*
10	14, 15	9

Buffer Pool (3 Pages)

1, 3	4, 8

Hard Disk

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	2, 3	5, 9	42, 73
		Run 1			**Run 2**			**Run 3**			**Run 4**

Step 2 Example

Input 1 Input 2 Output

	14, 15	9, 10

Buffer Pool (3 Pages)

1, 3
4, 8

Hard Disk

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	2, 3	5, 9	42, 73	
		Run 1			**Run 2**			**Run 3**			**Run 4**	

Step 2 Example

Input 1 Input 2 Output

12, 29	14, 15	

Buffer Pool (3 Pages)

1, 3	4, 8	9, 10

Hard Disk

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	2, 3	5, 9	42, 73
		Run 1			**Run 2**			**Run 3**			**Run 4**

Step 2 Example

Input 1	*Input 2*	*Output*
29	14, 15	12

Buffer Pool (3 Pages)

1, 3	4, 8	9, 10

Hard Disk

1, 8	9, 10	12, 29	3, 4	14, 15	17, 26	6, 8	12, 19	54, 90	2, 3	5, 9	42, 73

Run 1 **Run 2** **Run 3** **Run 4**

Example Summary

- Have **12 pages** to sort with **3 buffer** pages
- First step: produce **4 sorted runs** of **length 3**
- Can **merge 2 runs** in each merge step
- Second step: produce **2 sorted runs** of **length 6**
- Third step: produce **1 sorted run** of **length 12**

Cost Analysis (Phase 1)

- Multiple **sorting passes**, we read and write data once in each
  - Cost per pass is **2 * N (N is number of pages)**
- **How many steps** must we make with B buffer pages?
  - **First step** produces runs of length B
  - **Second step** produces runs of length (B-1) * B
  - **Third step** produces runs of length (B-1) * (B-1) * B ...
  - Stop once **$(B-1)^{steps-1} * B \geq N$**, after **$1+Ceil(\log_{B-1}(N/B))$** steps

Join Phase 2: Overview

- (Have **sorted** both input tables by their join column)
- **Load first page** of both sorted tables into memory
- **Find matching tuples** and add to join result output
- **Load next page** for table with smallest last entry
- **Keep doing** until no pages left for one table

Join Phase 2 Example



Buffer Pool (3 Pages)

Hard Disk

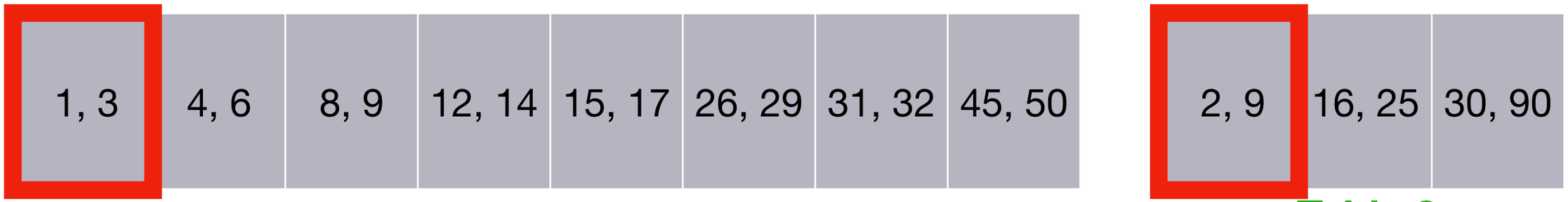
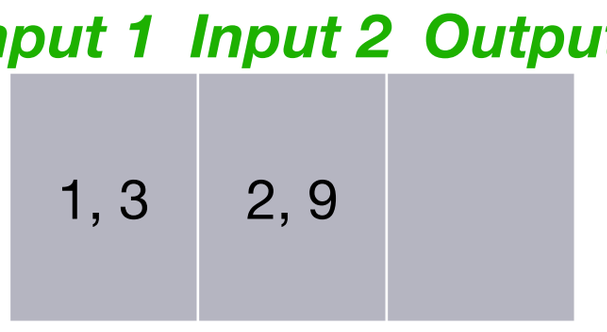
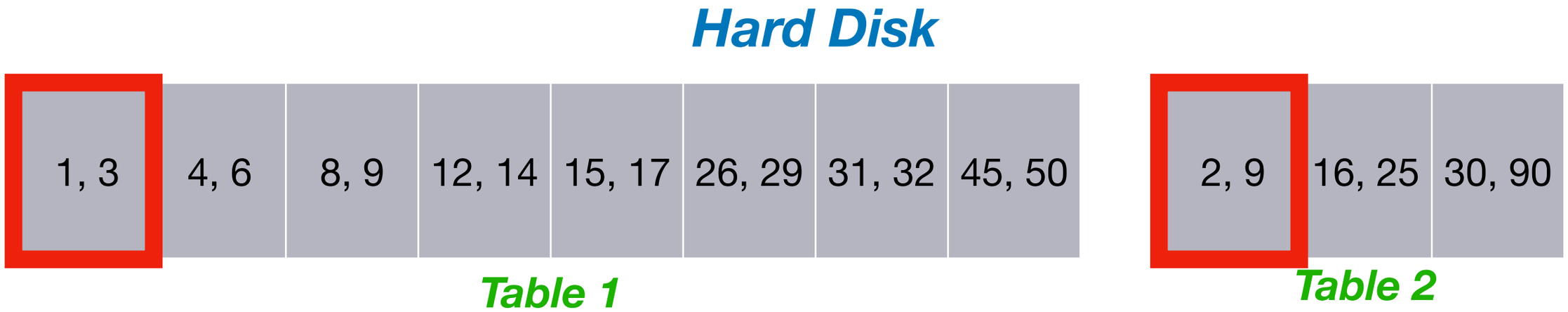


Table 1

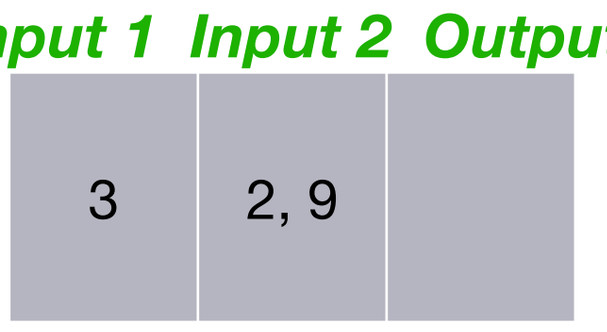
Table 2

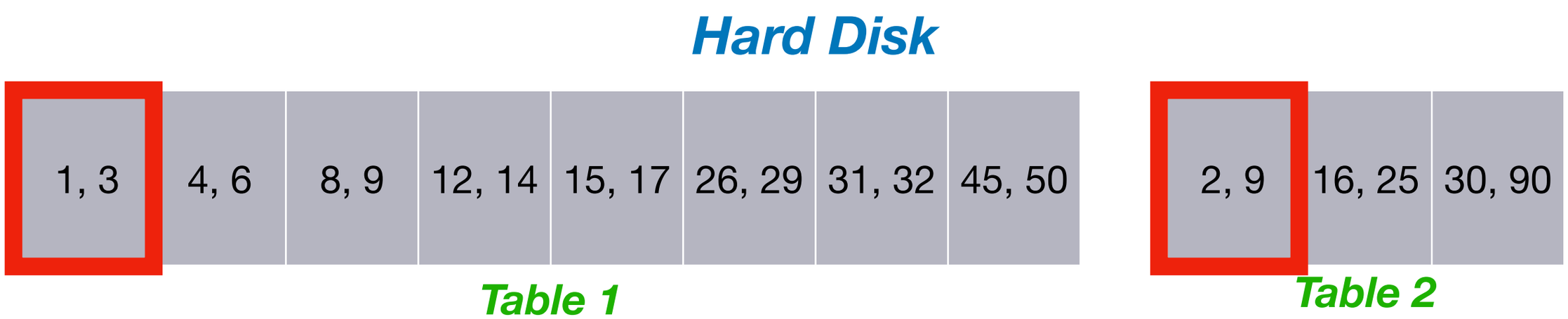
Join Phase 2 Example



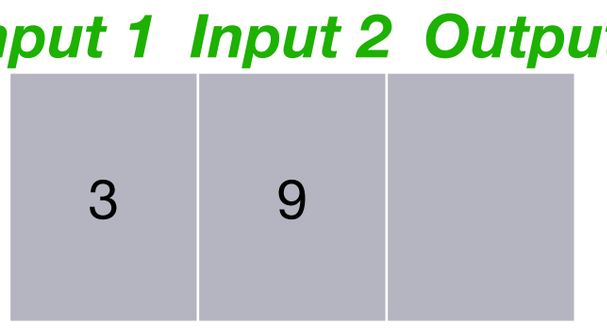


Join Phase 2 Example





Join Phase 2 Example



Buffer Pool (3 Pages)

Hard Disk

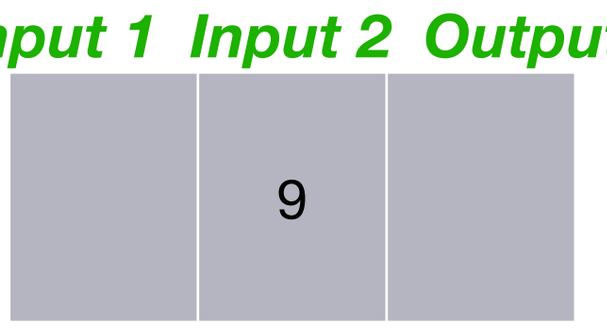
1, 3	4, 6	8, 9	12, 14	15, 17	26, 29	31, 32	45, 50

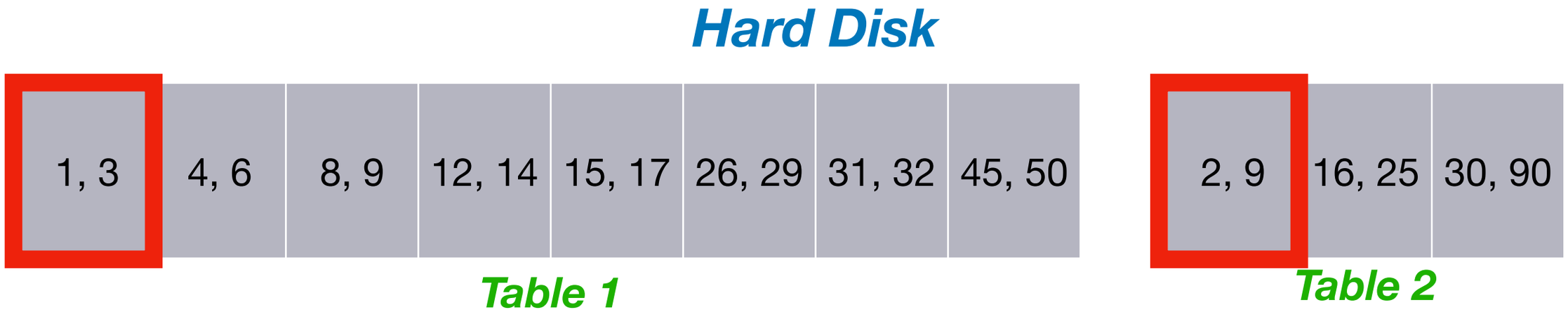
Table 1

2, 9	16, 25	30, 90

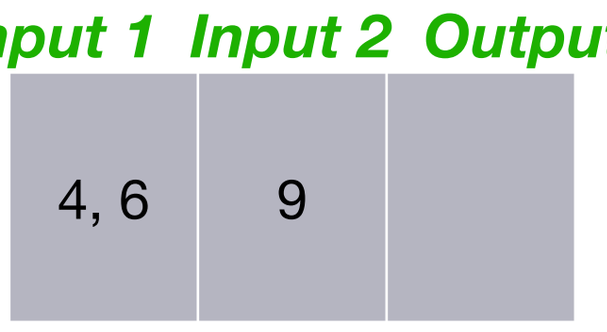
Table 2

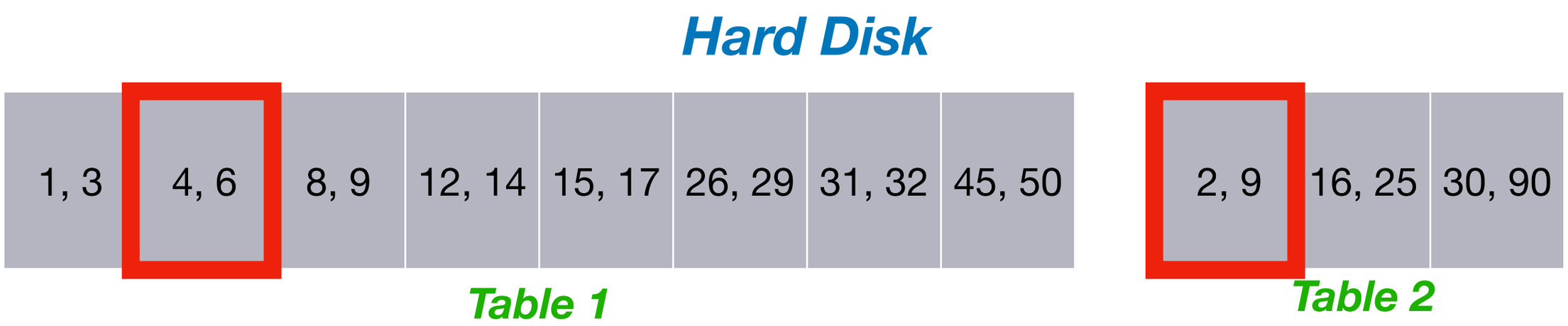
Join Phase 2 Example



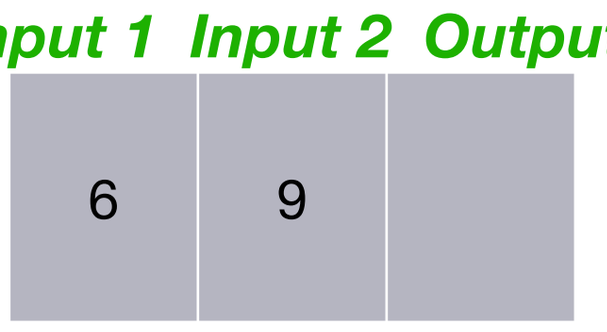


Join Phase 2 Example





Join Phase 2 Example



Buffer Pool (3 Pages)

Hard Disk

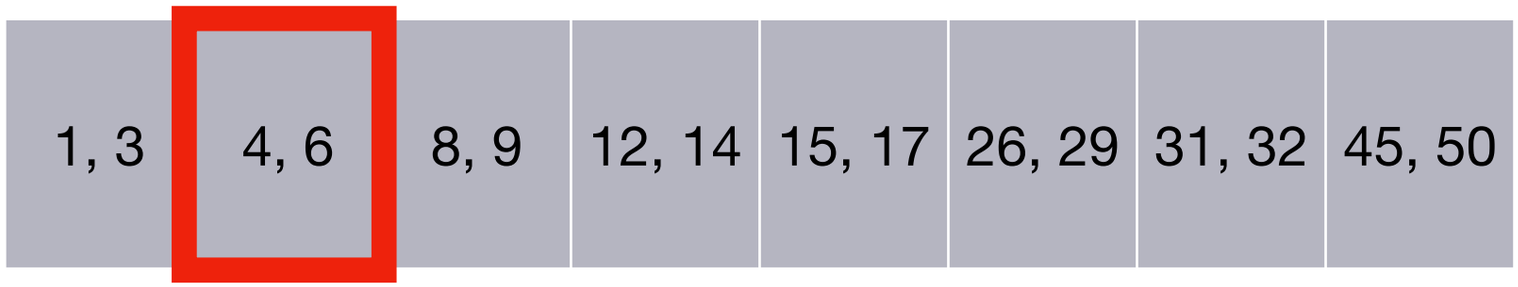


Table 1

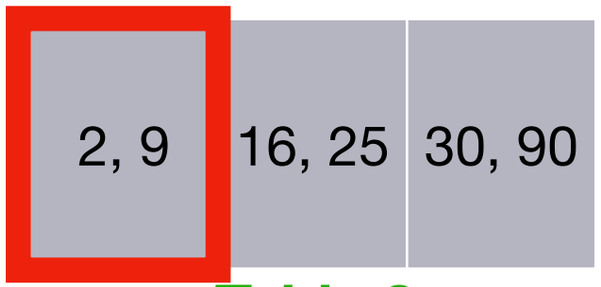
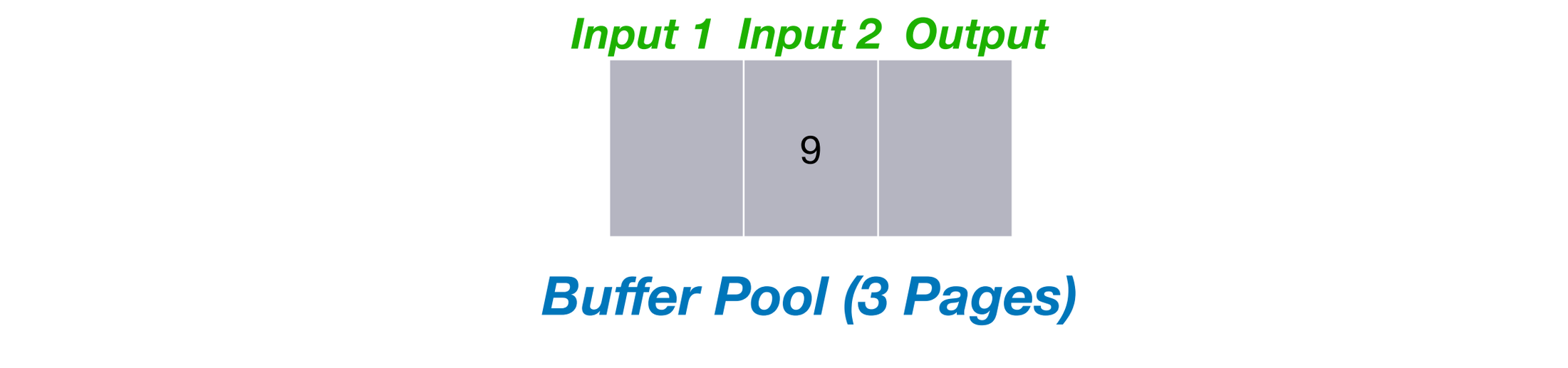
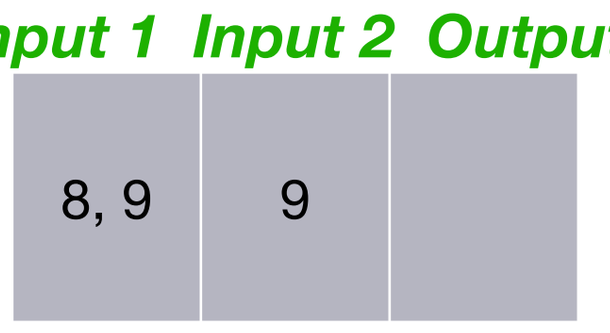


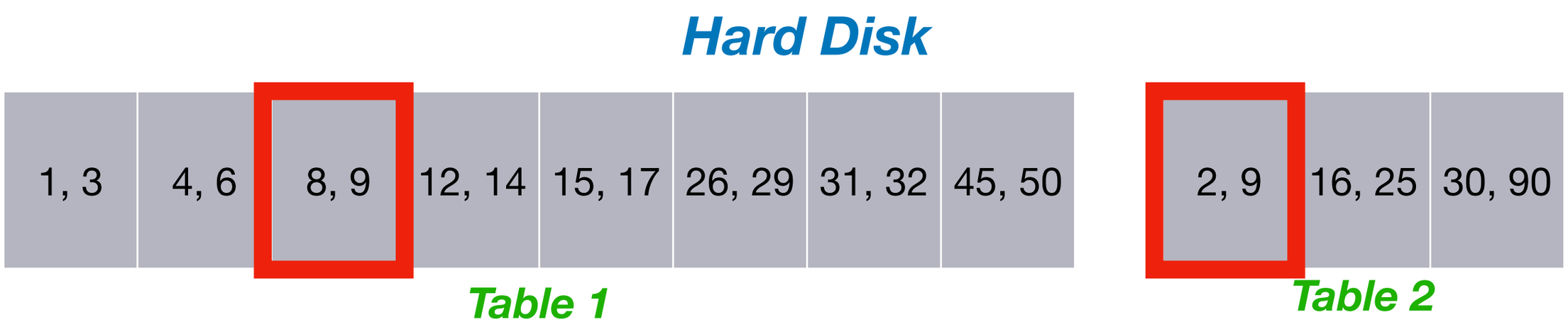
Table 2

Join Phase 2 Example

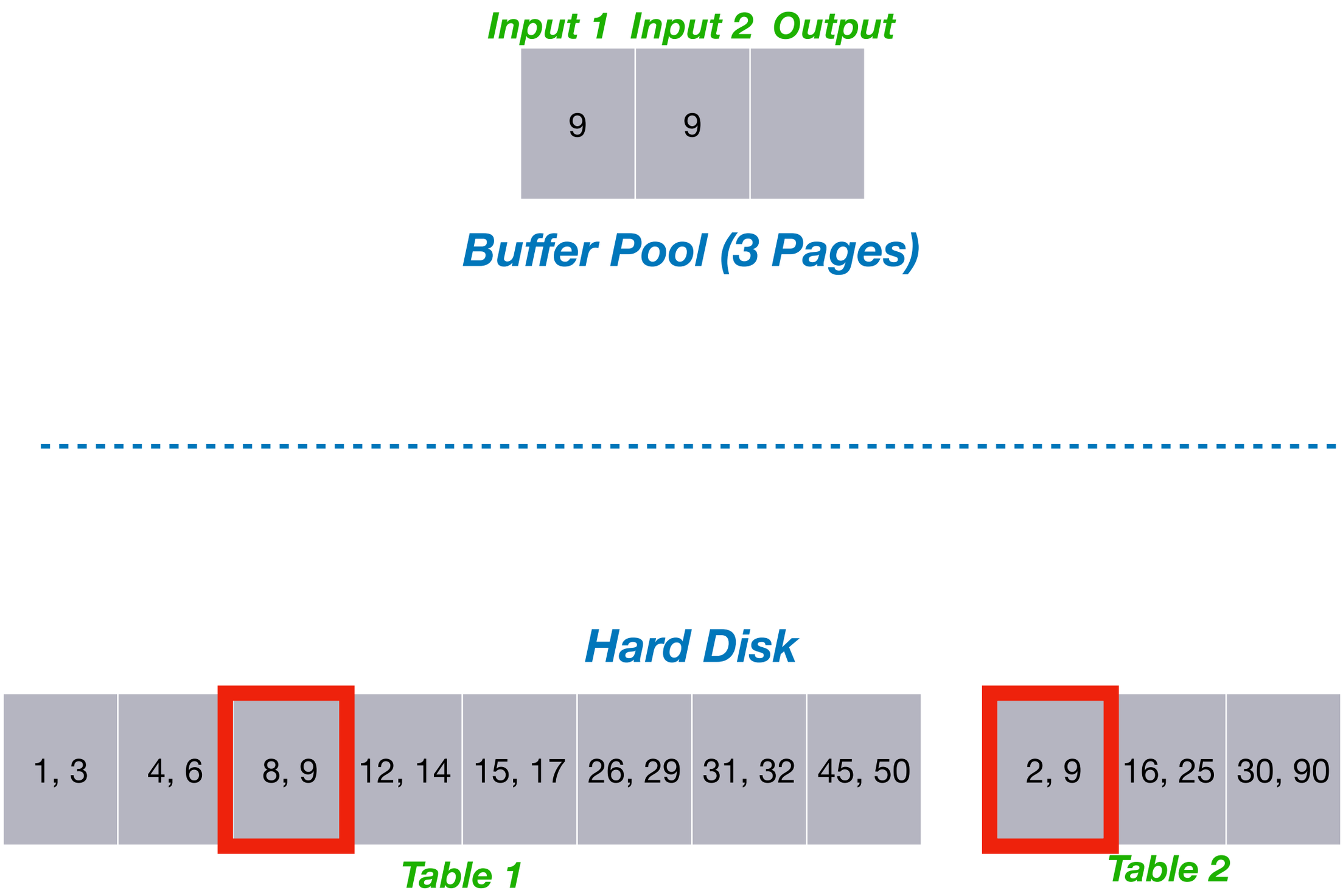


Join Phase 2 Example

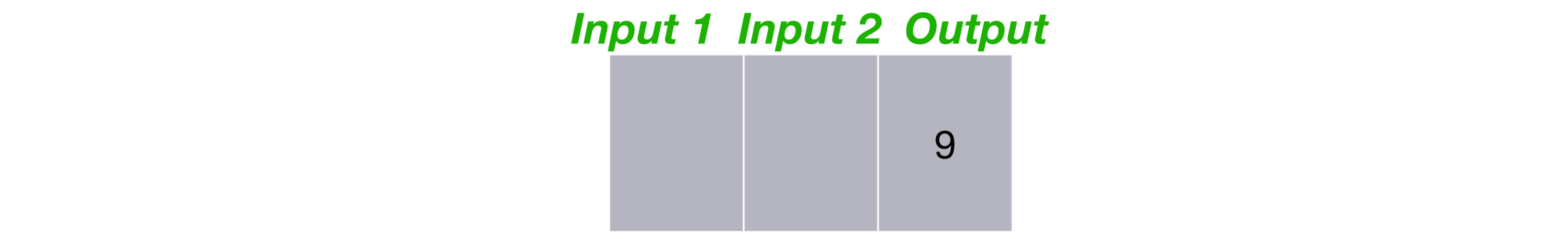




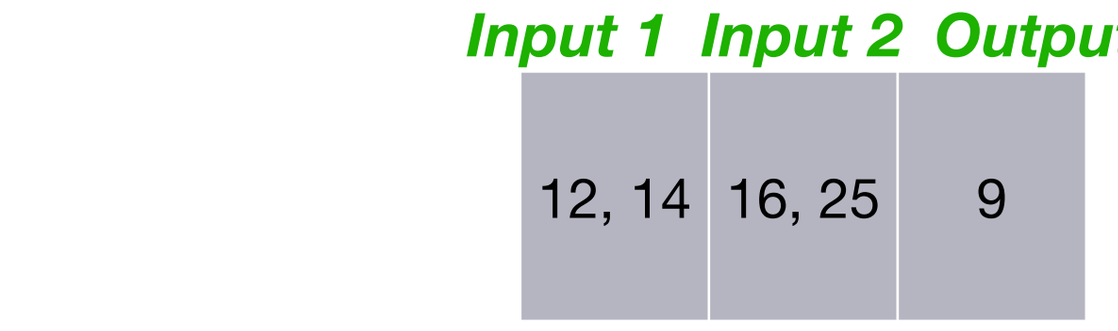
Join Phase 2 Example

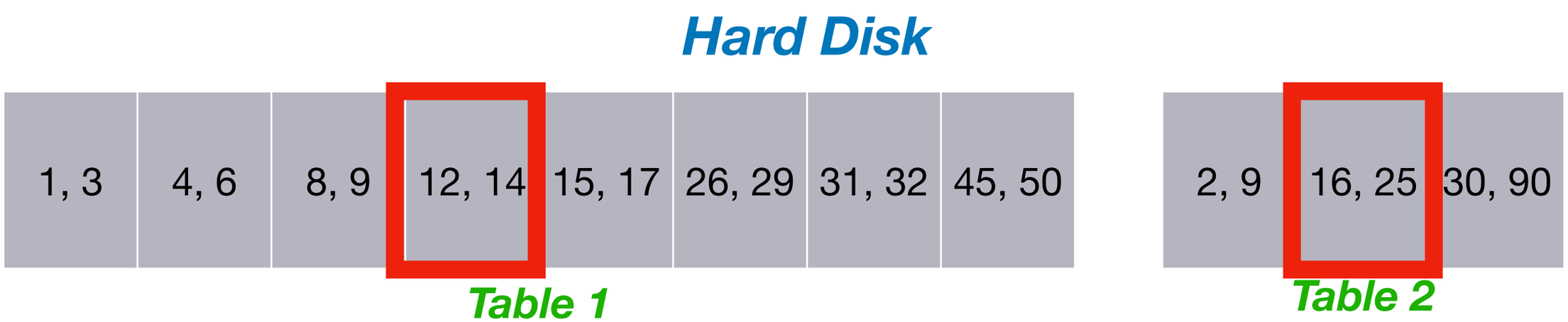


Join Phase 2 Example

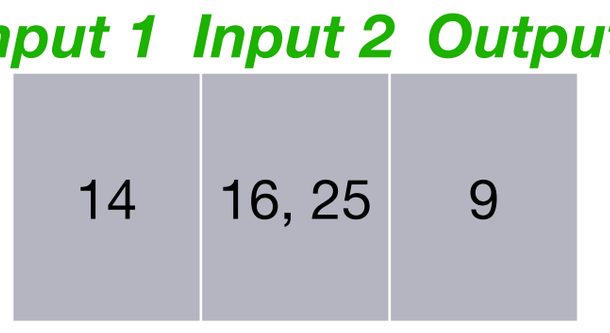


Join Phase 2 Example





Join Phase 2 Example



Buffer Pool (3 Pages)

Hard Disk

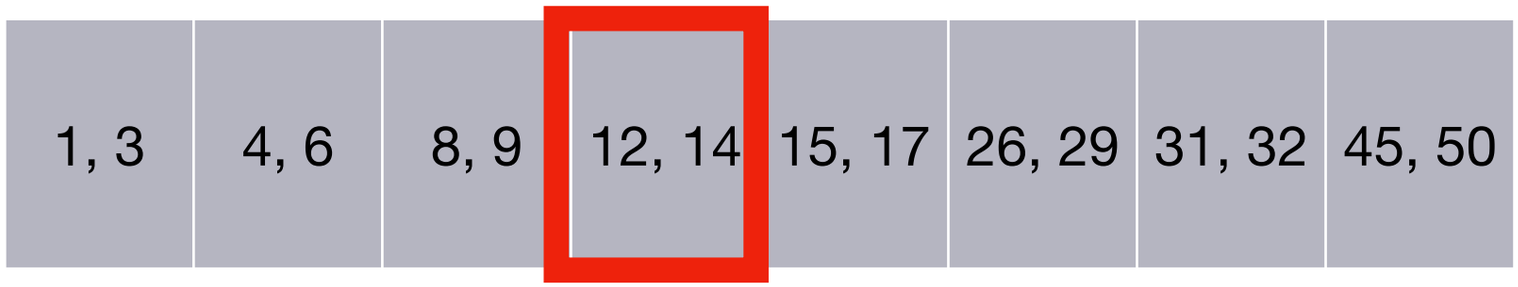


Table 1

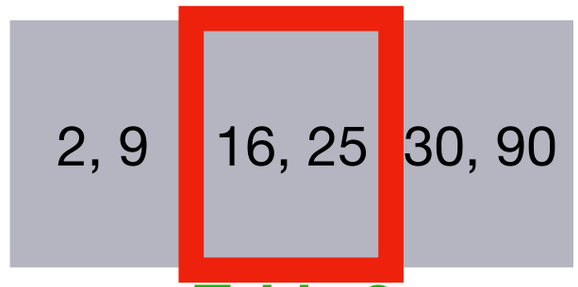
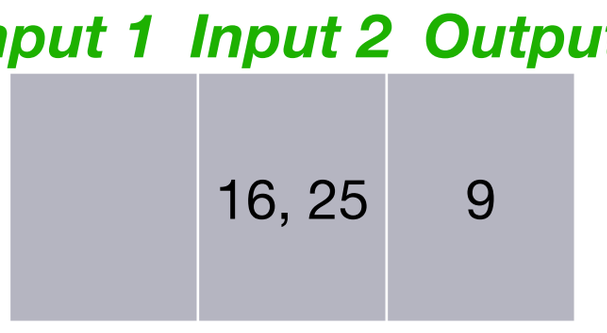
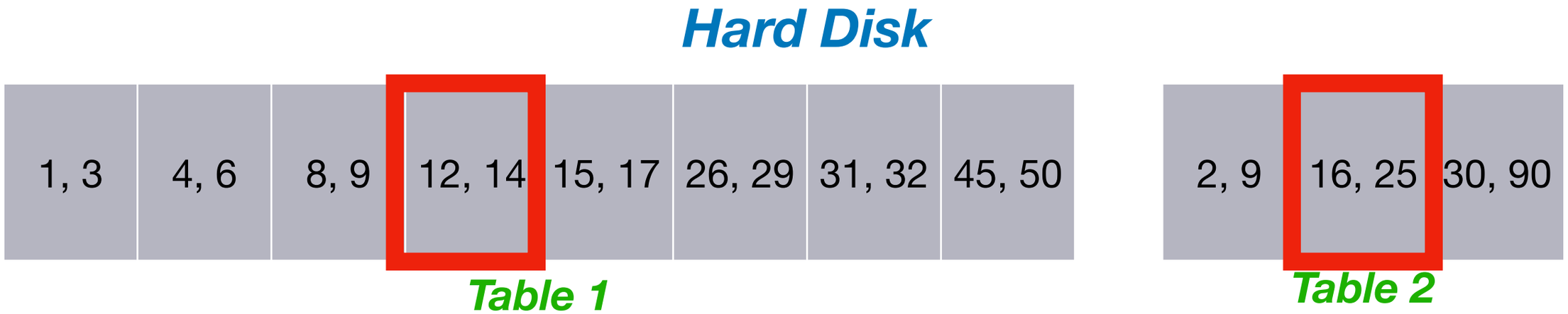


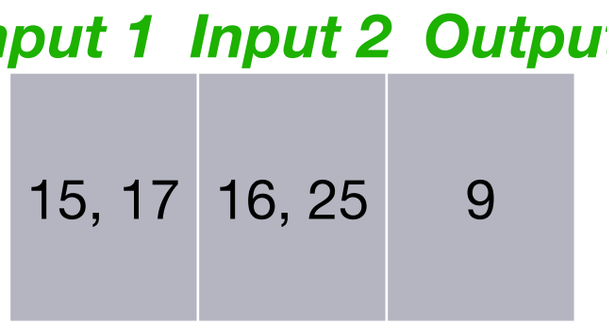
Table 2

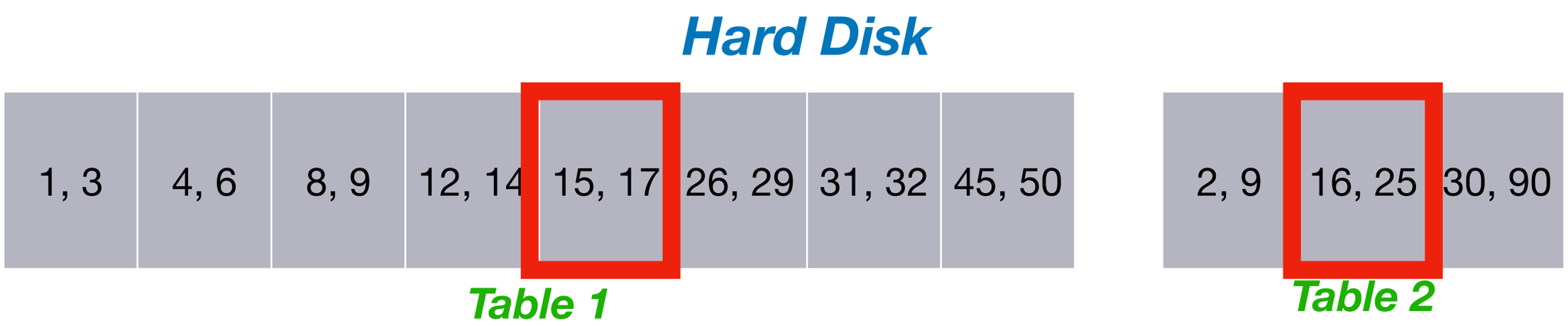
Join Phase 2 Example





Join Phase 2 Example





Join Phase 2 Example

Input 1 *Input 2* *Output*

17	16, 25	9

Buffer Pool (3 Pages)

Hard Disk

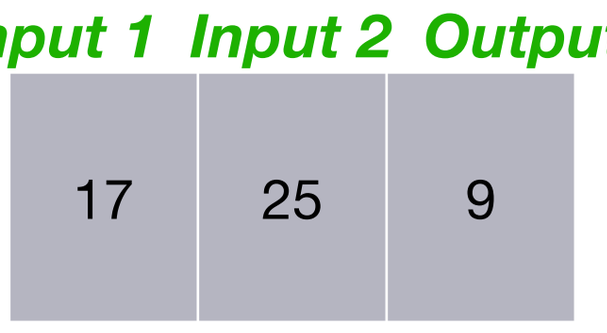
1, 3	4, 6	8, 9	12, 14	15, 17	26, 29	31, 32	45, 50

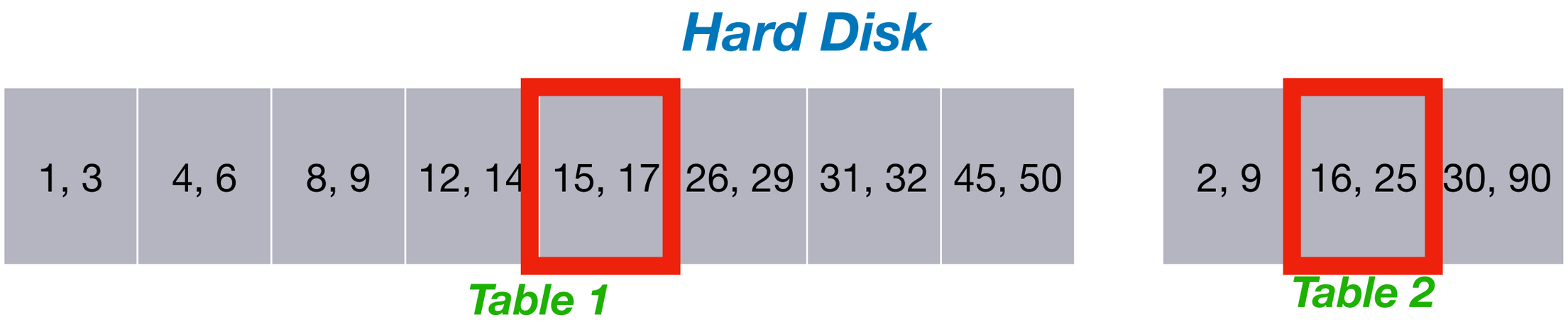
Table 1

2, 9	16, 25	30, 90

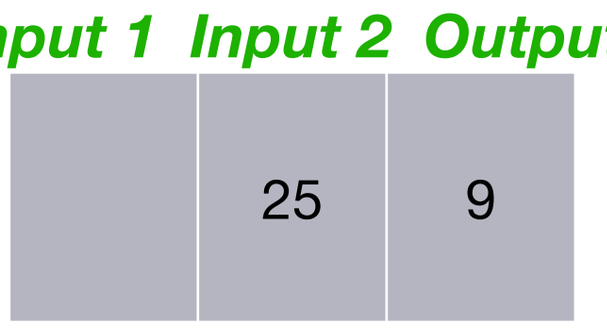
Table 2

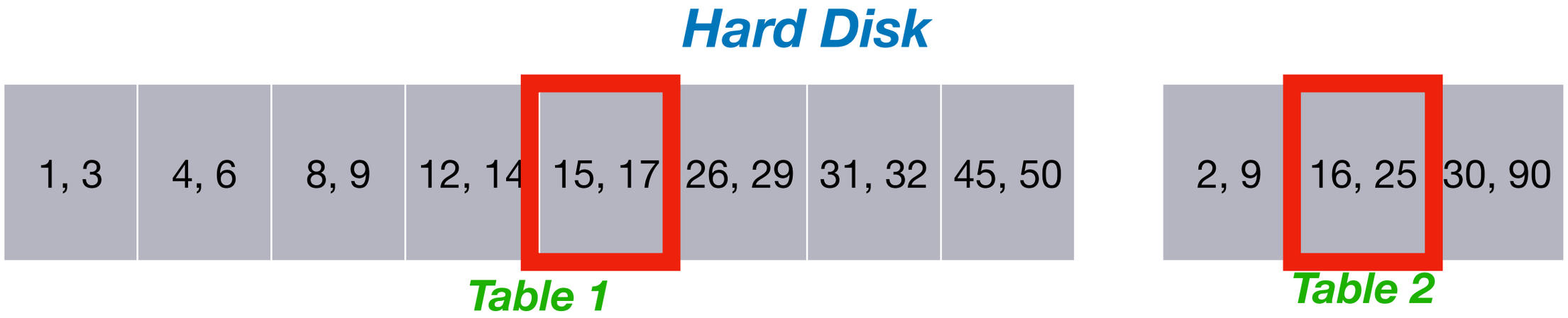
Join Phase 2 Example



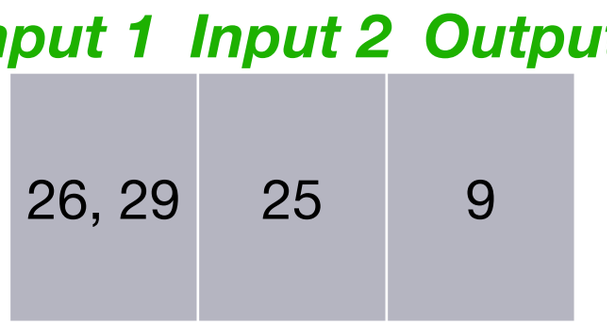


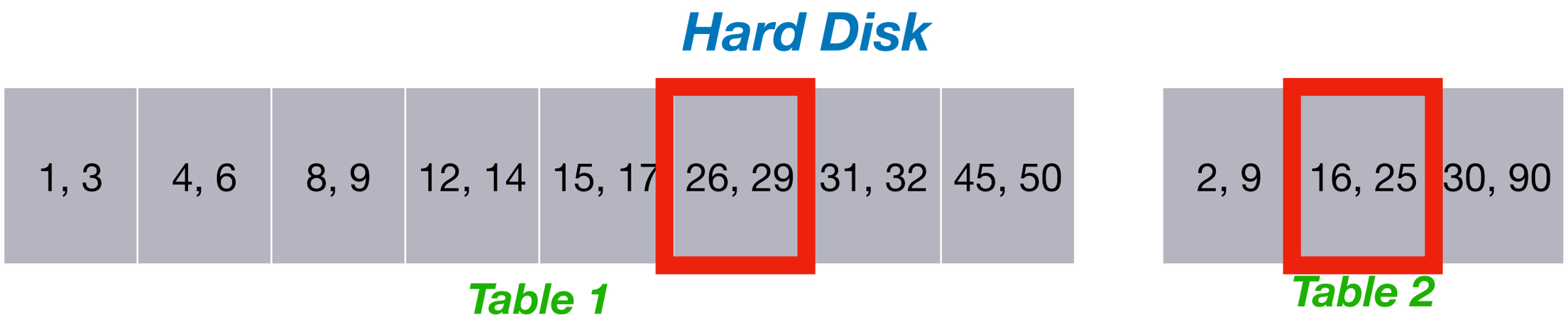
Join Phase 2 Example



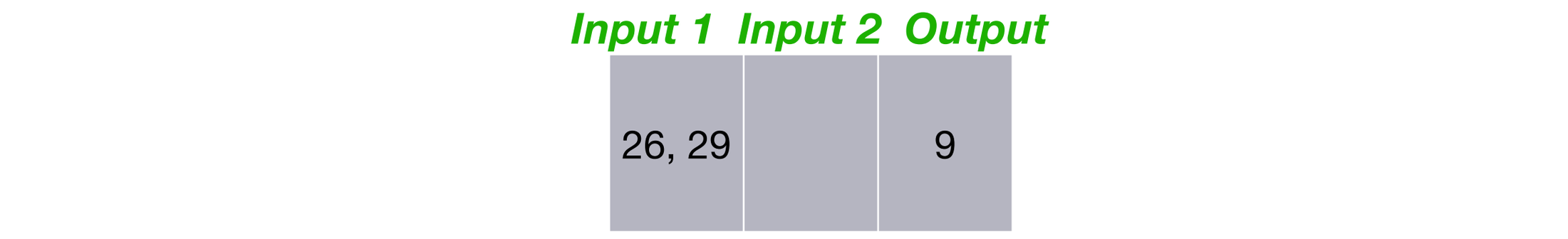


Join Phase 2 Example





Join Phase 2 Example



Join Phase 2 Example

Input 1 Input 2 Output

26, 29	30, 90	9

Buffer Pool (3 Pages)

Hard Disk

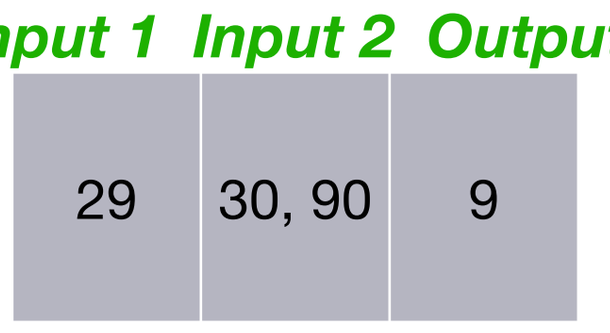
1, 3	4, 6	8, 9	12, 14	15, 17	26, 29	31, 32	45, 50

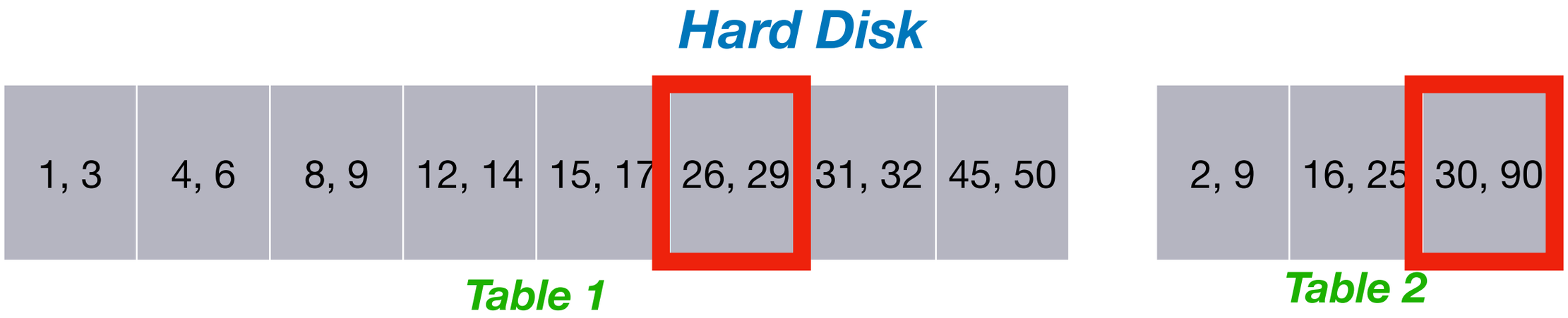
Table 1

2, 9	16, 25	30, 90

Table 2

Join Phase 2 Example





Cost Analysis (Phase 2)

- Assume all **duplicate entries** on same page
  - Duplicate entry: **same value in join column**
- Means that each input page is **only read once**
- Cost is proportional to **number of input pages**
  - I.e., Pages from **both** input tables

Total Join Cost

- Two input tables with M and N pages, B buffer pages
- First phase has cost
  - **$2*M*(1+Ceil(\log_{B-1}(M/B)))$** for sorting table 1
  - **$2*N*(1+Ceil(\log_{B-1}(N/B)))$** for sorting table 2
- Second phase has cost
  - **M+N** (we don't count cost for writing output!)

Increasing Memory: Which Phase Benefits?

How Much Memory?

- First phase: try to exploit **all buffer pages**
  - More buffer means less merging passes!
- Second phase: only exploit **three buffer pages**
  - One for first input, one for second input, one output

How Much Memory?

- First phase: try to exploit **all buffer pages**
  - More buffer means less merging passes!
- Second phase: only exploit **three buffer pages**
  - One for first input, one for second input, one output

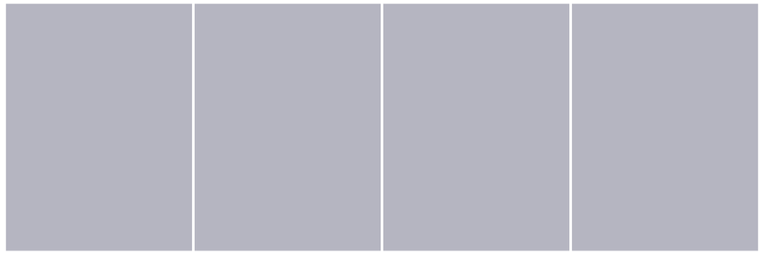
Seems Sub-Optimal!

How to Exploit More Memory in Phase 2 (Merging)?

Refined Sort-Merge Join

- Idea: can merge **more than two** sorted tables in phase 2
- Hence, **do not need to sort** tables completely in phase 1
- Means we can **save steps** (i.e., passes over the data)
- **First phase**: only sort data chunks that fit into memory
- **Second phase**: join all sorted chunks together (one step)

Refined SMJ - Phase 1



Buffer Pool (4 Pages)

Hard Disk

15, 3	45, 6	32, 1	12, 26	14, 31	9, 29	50, 8	4, 17

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

15, 3	45, 6	32, 1	12, 26

Buffer Pool (4 Pages)

Hard Disk

15, 3	45, 6	32, 1	12, 26	14, 31	9, 29	50, 8	4, 17

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

1, 3	6, 12	15, 26	32, 45

Buffer Pool (4 Pages)

Hard Disk

15, 3	45, 6	32, 1	12, 26	14, 31	9, 29	50, 8	4, 17

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

1, 3	6, 12	15, 26	32, 45

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	14, 31	9, 29	50, 8	4, 17

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

14, 31	9, 29	50, 8	4, 17

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	14, 31	9, 29	50, 8	4, 17

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

4, 8	9, 14	17, 29	31, 50

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	14, 31	9, 29	50, 8	4, 17

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

4, 8	9, 14	17, 29	31, 50

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

25, 90	16, 2	30, 9	

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

2, 9	16, 25	30, 90	

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

25, 90	16, 2	30, 9

Table 2

Refined SMJ - Phase 1

2, 9	16, 25	30, 90	

Buffer Pool (4 Pages)

Hard Disk

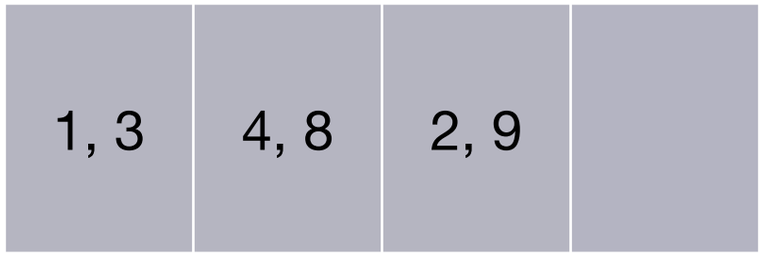
1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

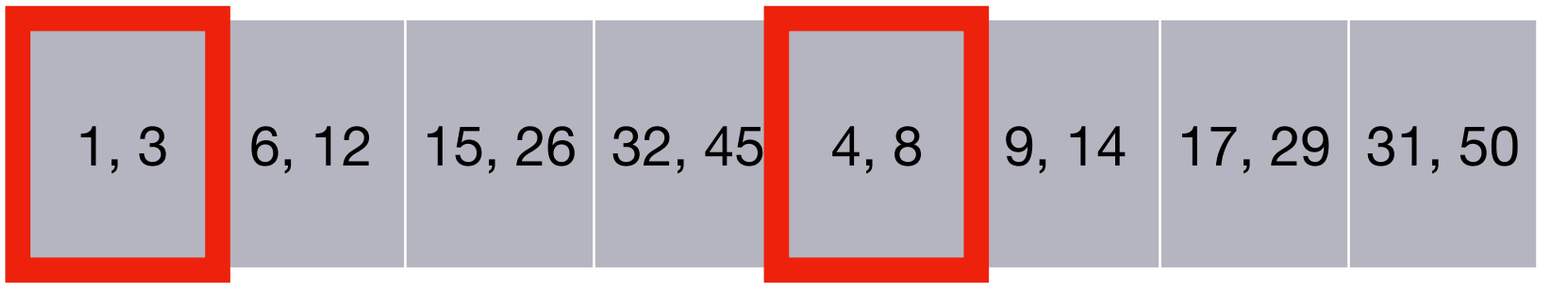
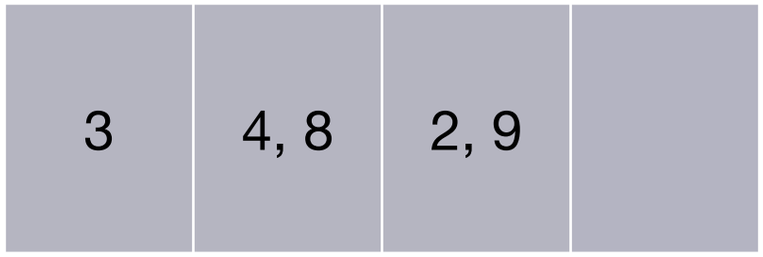


Table 1

Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

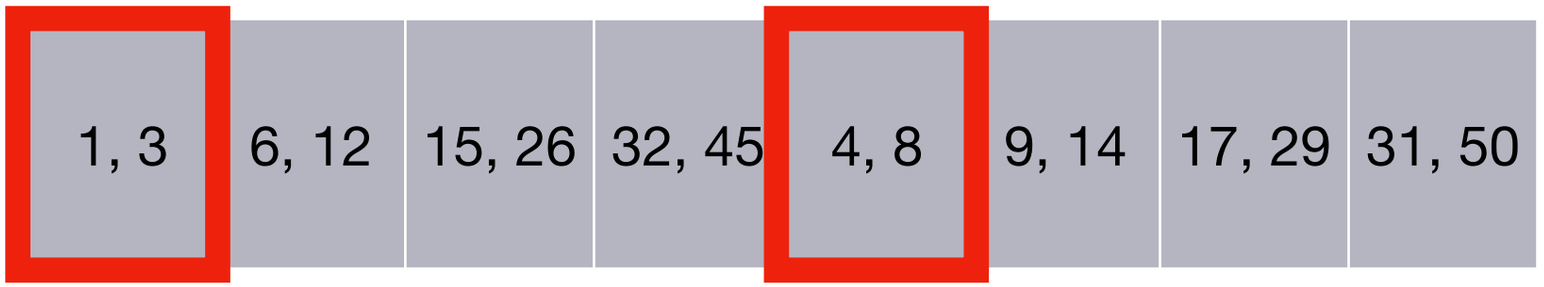


Table 1

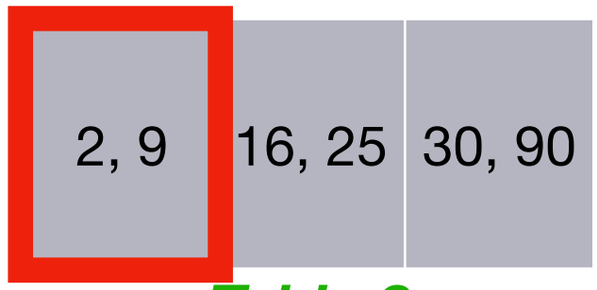


Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

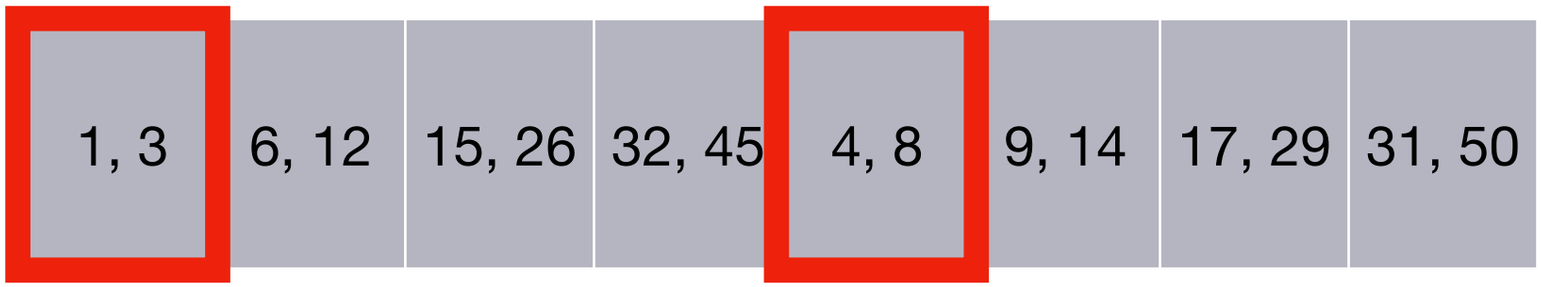


Table 1

Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

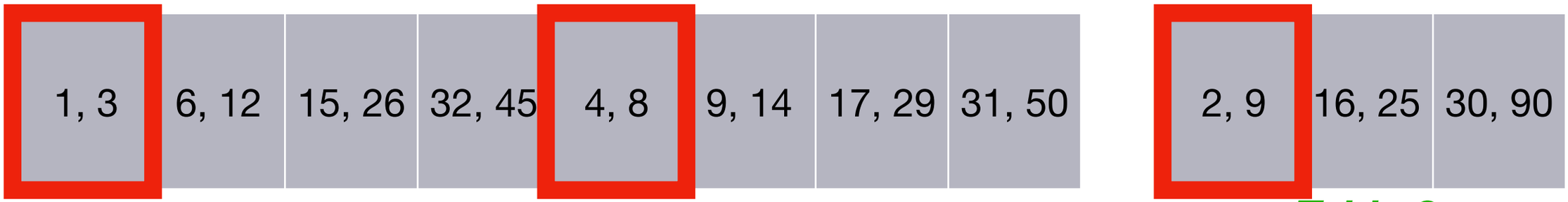


Table 1

Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

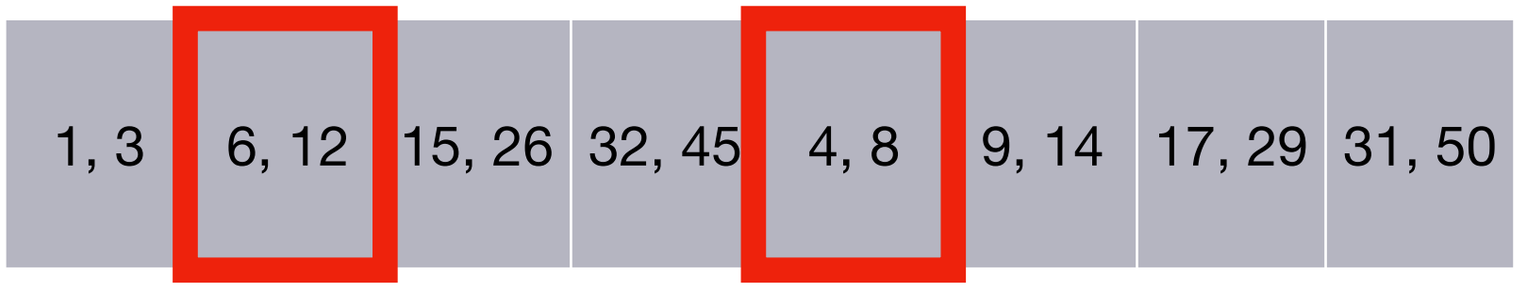


Table 1

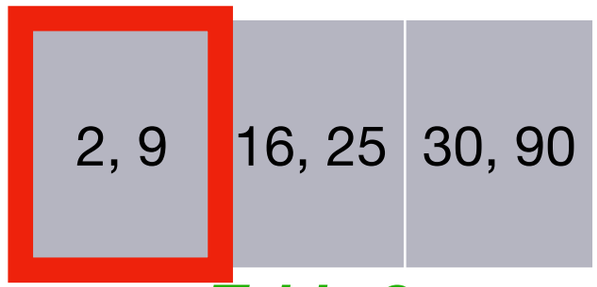
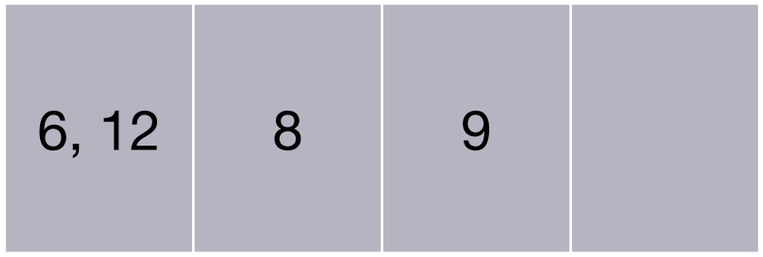


Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

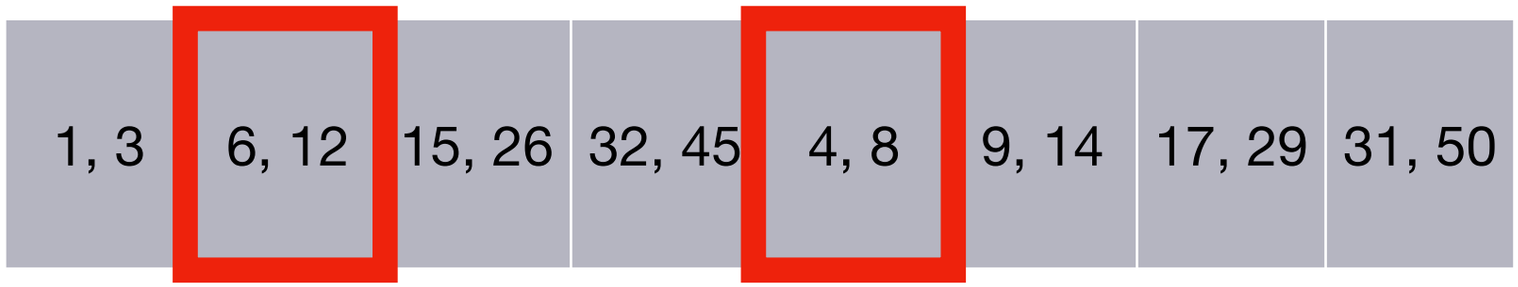


Table 1

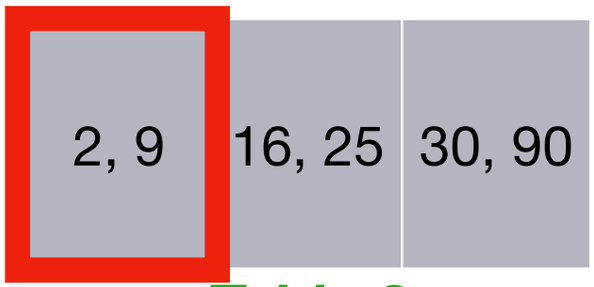
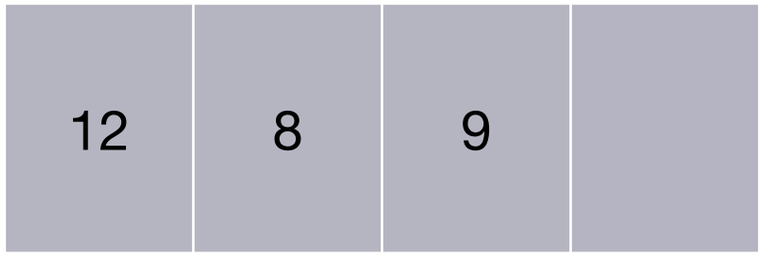
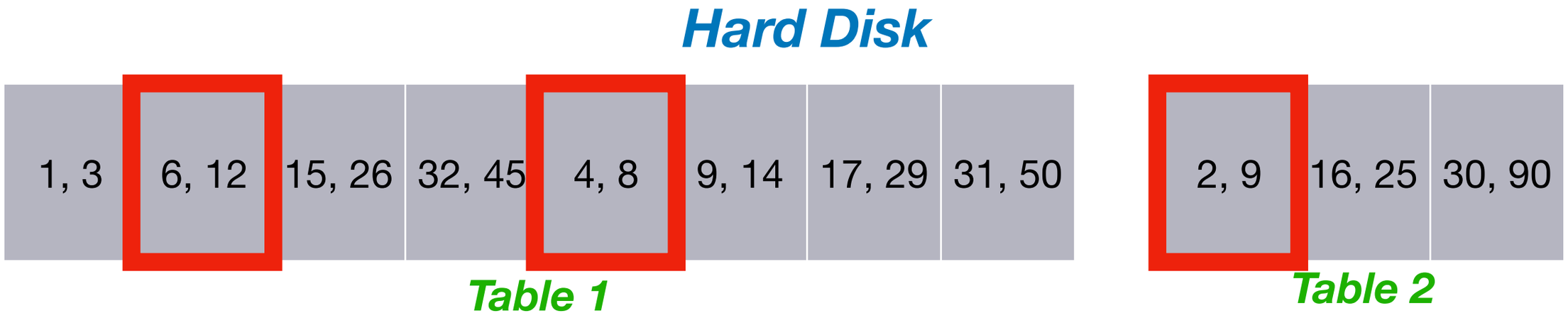


Table 2

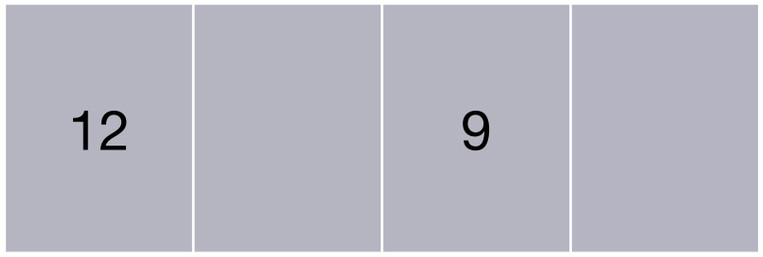
Refined SMJ - Phase 2



Buffer Pool (4 Pages)



Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

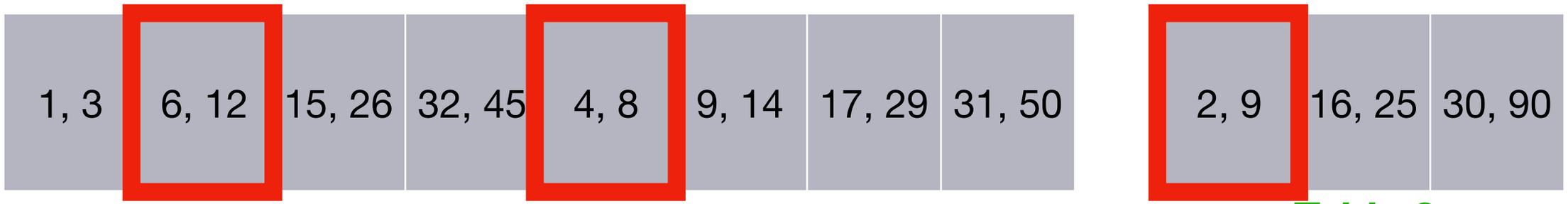
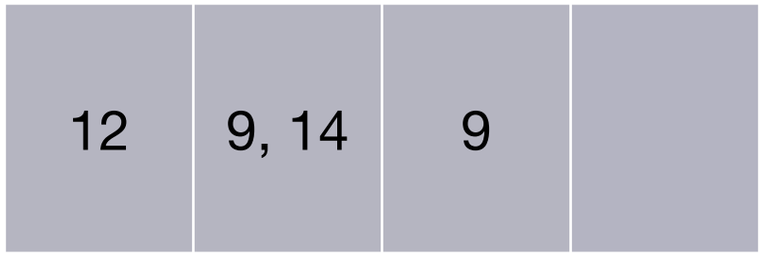


Table 1

Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

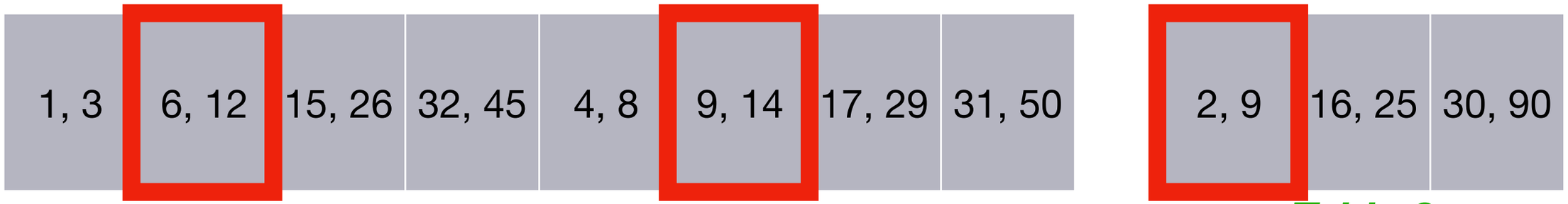


Table 1

Table 2

Refined SMJ - Phase 2

12	9, 14	9	9

Buffer Pool (4 Pages)

Hard Disk

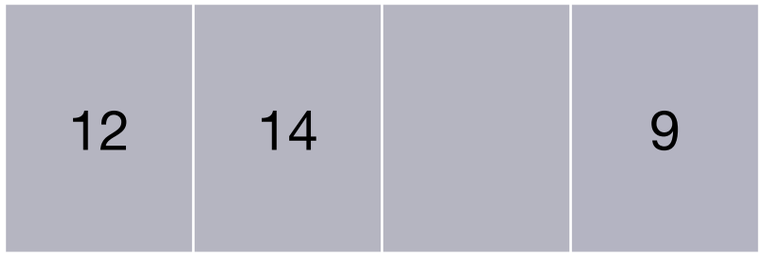
1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

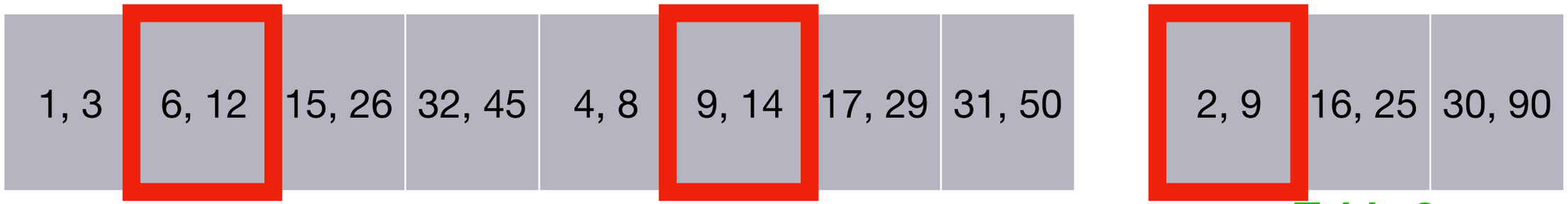


Table 1

Table 2

Refined SMJ - Phase 2

12	14	16, 25	9

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2

	14	16, 25	9

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2

15, 26	14	16, 25	9

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2

15, 26		16, 25	9

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2

15, 26	17, 29	16, 25	9

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2

26	17, 29	16, 25	9

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2

26	17, 29	25	9

Buffer Pool (4 Pages)

Hard Disk

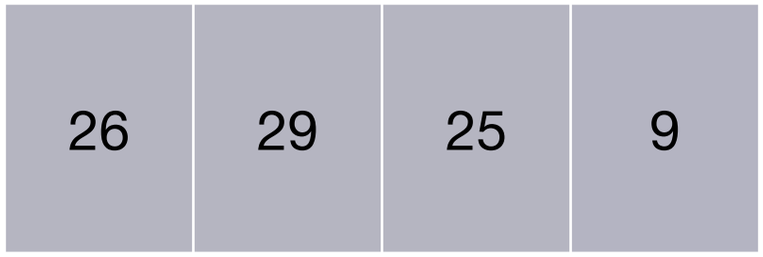
1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

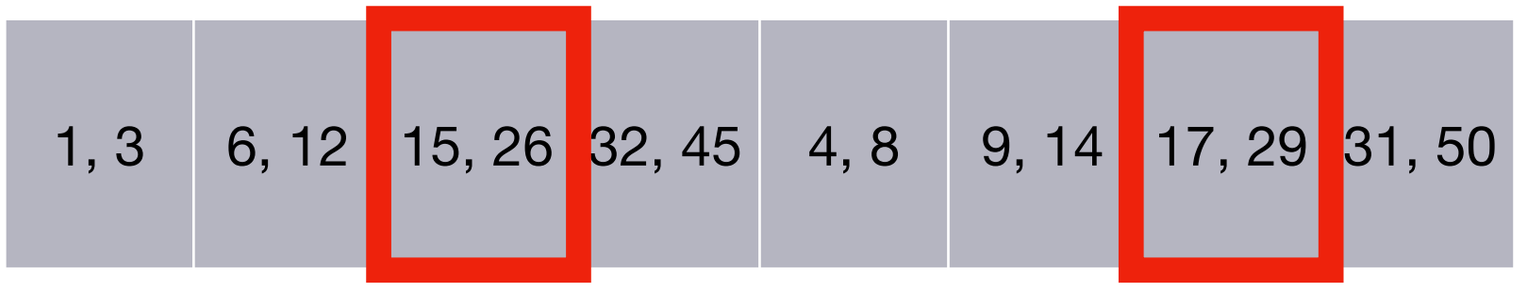


Table 1

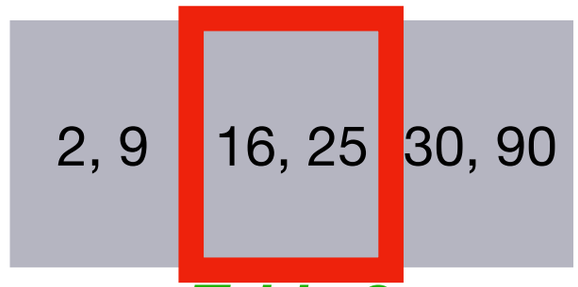
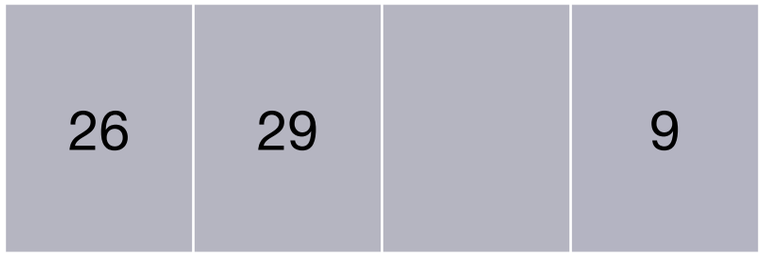


Table 2

Refined SMJ - Phase 2



Buffer Pool (4 Pages)

Hard Disk

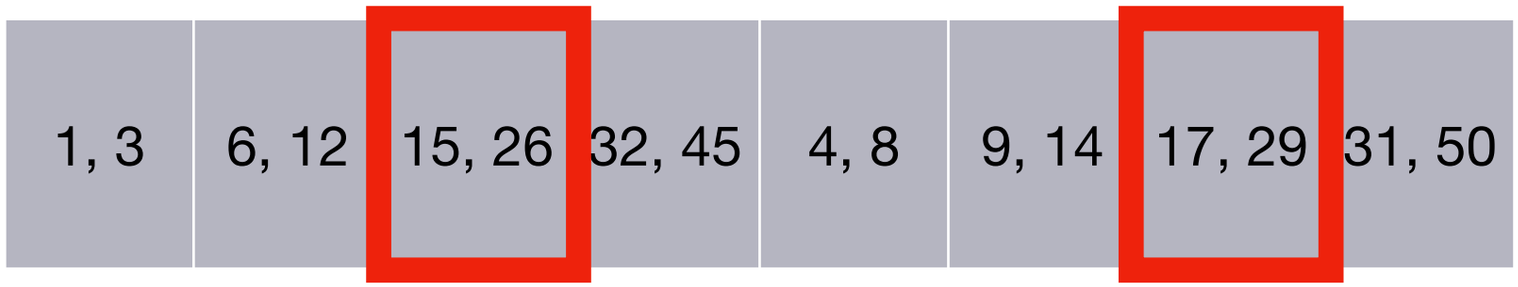


Table 1

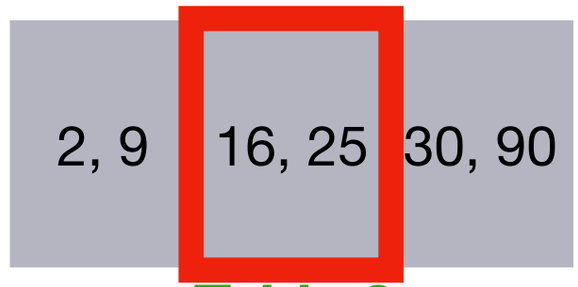


Table 2

Refined SMJ - Phase 2

26	29	30, 90	9

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined SMJ - Phase 2

	29	30, 90	9

Buffer Pool (4 Pages)

Hard Disk

1, 3	6, 12	15, 26	32, 45	4, 8	9, 14	17, 29	31, 50

Table 1

2, 9	16, 25	30, 90

Table 2

Refined Join Details

- Assume B buffer pages, tables with N and M pages
- **First phase**: load chunks of B pages, sort, write back
- **Second phase**: merge B-1 sorted chunks together
- Cost is **2*(M+N)** (Phase 1) **+ 1*(M+N)** (Phase 2)

How Much Memory?

- Again, B buffer pages, input sizes are M and N
- Have **(N+M)/B** sorted runs after first phase
- Need **B ≥ 1+(N+M)/B** to merge them in one step
- Rule of thumb if N>M: need **B ≥ 2*√(N)**

R-SMJ vs. Hash Join

	Hash Join	Refined Sort-Merge Join
Time	3 * Input Size	3 * Input Size
Memory	**> √(Smaller Table Size)**	> 2 * √(Larger Table Size)
Exploits Order		**Advantage**